



Table of Contents

PART I — FOUNDATIONS

Chapter 1: Project Layout	<i>Cargo.toml.md, lib.md, main.md</i>
Chapter 1: Project Layout	<i>Cargo.toml.md, lib.md, main.md</i>
Chapter 1: Project Layout	<i>Cargo.toml.md, lib.md, main.md</i>
Chapter 2: Primitives, Pattern Matching, Option	<i>boolean.md, env.md</i>
Chapter 2: Primitives, Pattern Matching, Option	<i>boolean.md, env.md</i>

PART II — CORE LANGUAGE

Chapter 3: Error Handling with Result	<i>error.md</i>
Chapter 4: Data Structures	<i>options.md, deployment_details.md</i>
Chapter 4: Data Structures	<i>options.md, deployment_details.md</i>

PART III — POLYMORPHISM

Chapter 5: Traits	<i>mod.md, stderr_io.md, mod.md, mod.md</i>
Chapter 5: Traits	<i>mod.md, stderr_io.md, mod.md, mod.md</i>
Chapter 5: Traits	<i>mod.md, stderr_io.md, mod.md, mod.md</i>
Chapter 5: Traits	<i>mod.md, stderr_io.md, mod.md, mod.md</i>
Chapter 6: Borrowing and Closures	<i>request_client.md, spinner.md</i>
Chapter 6: Borrowing and Closures	<i>request_client.md, spinner.md</i>

PART IV — APPLICATIONS

Chapter 7: Command-Line Parsing	<i>cli.md</i>
Chapter 9: Putting It All Together	<i>sample.md, deploy_naut.md, bitbucket.md</i>
Chapter 9: Putting It All Together	<i>sample.md, deploy_naut.md, bitbucket.md</i>
Chapter 9: Putting It All Together	<i>sample.md, deploy_naut.md, bitbucket.md</i>

PART V — QUALITY

Chapter 10: Testing	<i>cli_smoke.md, http_test.md, deploy_naut_test.md, bitbucket_test.md</i>
Chapter 10: Testing	<i>cli_smoke.md, http_test.md, deploy_naut_test.md, bitbucket_test.md</i>
Chapter 10: Testing	<i>cli_smoke.md, http_test.md, deploy_naut_test.md, bitbucket_test.md</i>
Chapter 10: Testing	<i>cli_smoke.md, http_test.md, deploy_naut_test.md, bitbucket_test.md</i>

Chapter 11: Documentation as Code

README.md

Chapter 12: Library Design

azure-sdk-guidelines-review.md

CHAPTER 1: PROJECT LAYOUT

 Source: [Cargo.toml.md](#)

This file is the **manifest** for the `nautpie` crate. Cargo reads it to figure out:

- What the crate is called and what version it is.
- What edition of Rust it targets.
- What other crates it depends on.
- What binary and library entry points to build.
- What lints to enforce.

This is the file you'd edit if you wanted to **add a new dependency**, **bump the version**, or **change the lint policy**.

The Big Picture

The file is divided into sections, each introduced by a `[section-name]` header:

Section	What it does
<code>[package]</code>	Crate metadata: name, version, edition, MSRV.
<code>[[bin]]</code>	Declares a binary target (the <code>nautpie</code> executable).
<code>[lib]</code>	Declares the library target.
<code>[dependencies]</code>	Runtime dependencies.
<code>[dev-dependencies]</code>	Test and example-only dependencies.
<code>[lints.clippy]</code>	Centralised clippy policy (added per Azure guidelines).
<code>[lints.rust]</code>	Centralised <code>rustc</code> lint policy.

Walkthrough

`[package]`

```
[package]
name = "nautpie"
version = "0.1.0"
edition = "2021"
rust-version = "1.85"
description = "DeployNaut / Bitbucket Pipelines deployment client (Rust port of nautpie.phar)"
license = "MIT"
```

Teaching points:

- **name** — the crate name. Used when other crates depend on this one (`use nautpie::...`).
- **version = "0.1.0"** — follows [Semantic Versioning](#). The leading `0` means "pre-1.0, anything goes." A `1.0` release would signal API stability.
- **edition = "2021"** — which *edition* of Rust this crate targets. Editions are opt-in language versions that allow breaking changes without bumping the major version. `2021` is the third stable edition (after 2015 and 2018).

- `rust-version = "1.85"` — the **MSRV** (Minimum Supported Rust Version). The crate promises to compile on any Rust version `>= 1.85`. CI must verify this. The Azure SDK guidelines require MSRV to be no more than 6 months old at release time.
- `description` and `license` — informational. Cargo uses them when publishing to crates.io.

Binary and library targets

```
[[bin]]
name = "nautpie"
path = "src/main.rs"

[lib]
name = "nautpie"
path = "src/lib.rs"
```

Teaching points:

- `[[bin]]` (**double brackets**) — declares a *list* of binaries. Even though there's only one, the double brackets are required syntax.
- `[lib]` (**single brackets**) — declares a single library. Most crates have at most one library.
- `path` — points to the source file. Defaults to `src/main.rs` and `src/lib.rs`, so these lines could be omitted. They're explicit here for clarity.
- **Same name for bin and lib.** This is allowed and even useful: when another crate depends on `nautpie`, it gets the library; when users install `cargo install nautpie`, they get the binary.

Dependencies

```
[dependencies]
clap = { version = "4.6", features = ["derive"] }
thiserror = "2"
serde = { version = "1", features = ["derive"] }
serde_json = "1"
owo-colors = { version = "4", features = ["supports-colors"] }
dotenvy = "0.15"
request = { version = "0.13", default-features = false, features = ["blocking", "json", "rustls"] }
base64 = "0.22"
url = "2"
chrono = "0.4"
ratatui = "0.29"
crossterm = "0.28"
```

Teaching points:

- `version = "x.y"` — the version requirement. `4.6` means `>=4.6.0, <5.0.0` (semver caret requirement, the default).
- `features = ["derive"]` — enable optional features for this crate. `clap`'s `derive` feature enables `#[derive(Parser)]` and similar.
- `default-features = false` — opt out of `request`'s default features (which include `native-tls`, `default-tls`, etc.). We instead enable only `blocking`, `json`, and `rustls`. This produces a smaller binary with no native TLS dependency.
- `request = { version = "0.13", ..., features = ["blocking"] }` — we deliberately use the **blocking** flavor of `request` because the binary is sync (matches PHP parity). Async `request` would require a tokio runtime.

```
[dev-dependencies]
assert_cmd = "2"
predicates = "3"
wiremock = "0.6"
tokio = { version = "1", features = ["macros", "rt-multi-thread"] }
```

Teaching points:

- `[dev-dependencies]` — only used for tests, examples, and benchmarks. Not included in the production binary.
- `assert_cmd` — runs the compiled binary as a subprocess and asserts on its output. Used in `tests/cli_smoke.rs`.

- `predicates` — composable assertion matchers (`predicate::str::contains(...)`).
- `wiremock` — in-process HTTP server for testing HTTP clients without a real network.
- `tokio` — needed by `wiremock` for async setup. We don't use it in production code.

Lint policy (Azure SDK guideline)

```
# Centralised lint policy. Mirrors what CI runs (`cargo clippy -D warnings`).
# Keeping it in Cargo.toml means `cargo clippy` locally uses the same rules.
[lints.clippy]
# We allow these pedantic lint because they produce too much noise on a
# CLI tool of this size. Each one has been considered; loosening them here
# keeps the clippy output actionable instead of full of false positives.
module_name_repetitions = "allow"
must_use_candidate = "allow"
missing_errors_doc = "allow"
missing_panics_doc = "allow"
similar_names = "allow"

# Rust lints: enforce documentation completeness on every public item.
[lints.rust]
missing_docs = "warn"
```

Teaching points:

- `[lints.clippy]` — a top-level table that controls clippy's behaviour. Each entry is a lint name and a level (`allow` , `warn` , `deny`).
- `module_name_repetitions = "allow"` — overrides a default-on pedantic lint that would flag `deployment_details::DeploymentDetails` (the module name repeated in the type). We have several such repetitions in this codebase and don't find them problematic.
- `must_use_candidate = "allow"` — silences a lint that flags functions whose return values should be used. For a CLI, this fires too often to be useful.
- `missing_errors_doc = "allow"` — the `missing_docs` lint would otherwise require every `Result`-returning function to document which error variants it can produce. This is enforced at a basic level (the function has a `# Errors` section where it matters) but we don't want it blocking the build.
- `missing_panics_doc = "allow"` — similar, but for `panic!` calls. We have only one (the `expect` in `main.rs`) and it's documented.
- `similar_names = "allow"` — silences a lint that flags near-duplicate identifiers. Useful when you have `Io` and `io` and `IO` in the same module.
- `[lints.rust]` — controls built-in `rustc` lints. The only entry is `missing_docs = "warn"`, which is the Azure guideline `[rust-documentation-warn-missing-docs]`.
- **Why "warn" instead of "deny"?** A `warn` is visible to developers without breaking the build. We could promote it to `deny` once we confirm no existing public items need attention.

Why Is It Written This Way?

- **Explicit over implicit.** Listing every dependency, target, and lint in the manifest makes the build **reproducible**: another developer (or CI) gets exactly the same setup.
- **MSRV declared.** Cargo's resolver will refuse to build with a Rust version older than `1.85`. This satisfies the Azure guideline `[rust-platform-msrv]`.
- **Feature flags trimmed.** Disabling `request`'s default features and re-enabling only what we need (blocking + rustls) keeps the binary lean and avoids pulling in native TLS code.
- **Dev-dependencies isolated.** `wiremock` and `assert_cmd` are not compiled into the release binary — they're only linked when running tests.
- **Centralised lints.** The lint policy lives in `Cargo.toml` (not just in CI commands) so that `cargo clippy` locally matches what CI enforces. This is one of the easier Azure guidelines to satisfy.

Key Rust / Cargo Concepts Used Here

Concept	Where	What it does
<code>[package]</code>	top	Crate metadata
<code>[[bin]]</code> vs <code>[lib]</code>	targets	Brackets tell Cargo which kind of target
<code>version = "x.y"</code>	deps	Semver requirement; <code>^x.y</code> is the default
<code>features = [...]</code>	deps	Enable optional functionality on a crate
<code>default-features = false</code>	request	Opt out of defaults; only enable what we list
<code>rust-version</code>	package	MSRV promise
<code>[dependencies]</code>	runtime	Crates linked into the final binary
<code>[dev-dependencies]</code>	tests/examples	Crates only linked for <code>cargo test</code> and <code>cargo run --example</code>
<code>[lints.clippy]</code>	lint policy	Centralised clippy configuration
<code>[lints.rust]</code>	lint policy	Centralised rustc lint configuration
<code>edition</code>	package	Which Rust edition's syntax to use
<code>description</code> , <code>license</code>	package	Used by crates.io when publishing

 Source: `src/lib.md`

When `Cargo.toml` lists both a `[[bin]]` and a `[lib]` section, you end up with **two crates from one project**: a binary (the `nautpie` executable) and a library (a reusable code module other Rust programs can depend on). `src/lib.rs` is the entry point of that library.

In this project, the binary is tiny — `main.rs` is only about 70 lines. The library is where the real work lives. Splitting them like this lets the integration tests (`tests/*.rs`) use the library code directly, and it would let other Rust programs embed `nautpie` as a dependency if they ever wanted to.

The Whole File

```

//! NautPie — DeployNaut / Bitbucket Pipelines deployment client.
//!
//! Rust port of the PHP `nautpie.phar` (Symfony Console). This crate exposes the
//! CLI dispatch surface and supporting helpers; the binary entrypoint is
//! [`main`] in `src/main.rs`.
//!
//! # Crate overview
//!
//! ... module map table ...
//!
//! # Quickstart
//!
//! ```no_run
//! use nautpie::error::ApiResponse;
//! use serde_json::json;
//! let resp = ApiResponse::ok(json!("hello"));
//! ```

#![warn(missing_docs)]
#![doc = include_str!("../README.md")]

pub mod boolean;
pub mod cli;
pub mod commands;
pub mod deployment_details;
pub mod env;
pub mod error;
pub mod http;
pub mod io;
pub mod options;
pub mod tui;

pub use error::{ApiResponse, Error};

```

That's the entire file. Every single line is doing important work.

Walkthrough

The doc comment block

```

//! NautPie — DeployNaut / Bitbucket Pipelines deployment client.
//! ...
//! # Module map
//! ...
//! # Quickstart
//! ```no_run
//! ...
//! ```

```

`//!` is an **inner doc comment** — it documents the **module itself**, not an item inside it. `cargo doc` will render this as the documentation page for the `nautpie` crate.

This particular comment block has grown over time. It now includes:

- A **one-line description** of what the crate does.
- A **"Crate overview"** section that points at the README.
- A **"Module map" table** listing every public module with a one-line summary.
- A **"Quickstart"** section with a `no_run` Rust code example showing the most basic usage (building an `ApiResponse`).

The `no_run` attribute on the code fence is important: it tells `cargo test` to **compile** the example (so it can't go stale) but **not run** it (so it doesn't require any external resources).

Crate-level attributes

```
#![warn(missing_docs)]
#![doc = include_str!("../README.md")]
```

Teaching points:

- `#![...]` is an **inner attribute** — the `!` means it applies to the enclosing item (the crate), not to a single item below. It always goes at the top of the file (or near the top).
- `#![warn(missing_docs)]` — turns on the `missing_docs` lint as a **warning**. This means every `pub fn`, `pub struct`, `pub enum`, `pub trait`, `pub mod`, and public field without a `///` doc comment will produce a compiler warning. This is one of the recommendations from the Azure SDK Rust guidelines (item [\[rust-documentation-warn-missing-docs\]](#)) and a huge help in keeping documentation complete.
- `#![doc = include_str!("../README.md")]` — the `include_str!` macro reads a file at compile time and embeds its contents as a `&'static str`. The `#![doc = ...]` attribute then treats that string as **module-level documentation**. The result: `cargo doc --open` shows the README as the crate overview, exactly the same way it appears on GitHub.
- The path `../README.md` is **relative to the file containing the attribute** (`src/lib.rs`), so `../README.md` means "go up one directory to the crate root, then into `README.md`".

Module declarations

```
pub mod boolean;
pub mod cli;
pub mod commands;
pub mod deployment_details;
pub mod env;
pub mod error;
pub mod http;
pub mod io;
pub mod options;
pub mod tui;
```

- `pub mod` means "this is a **public module** of the library." Anyone depending on this crate can write `nautpie::cli::Cli` and reach into the `cli` module.
- Each `mod` name corresponds to a file or folder in `src/`. For example, `pub mod http;` resolves to either `src/http.rs` or `src/http/mod.rs`. Here it's the folder form (the `http/mod.rs` we read earlier).
- The ordering is alphabetical, which is a common Rust convention.

Why do we need to declare modules here? In Rust, **modules are private by default**. Declaring them in `lib.rs` (or `main.rs`) brings them into scope. Declaring them as `pub` exposes them to outside consumers of the library.

Re-exports

```
pub use error::{ApiResponse, Error};
```

This is a **re-export**. Without it, consumers would have to write the full path:

```
use nautpie::error::{ApiResponse, Error};
```

With the re-export, they can write the shorter:

```
use nautpie::{ApiResponse, Error};
```

It's purely a convenience — it makes the most commonly-used types from the library available at the crate's top level. The original definitions are still in `src/error.rs`; the re-export just adds a shortcut.

The Module Tree

This single file creates the entire module hierarchy that the rest of the codebase uses. Here's the tree:

```
nautpie                (the crate root – this file)
├─ boolean              → src/boolean.rs
├─ cli                  → src/cli.rs
├─ commands             → src/commands/mod.rs
│   └─ bitbucket        → src/commands/bitbucket.rs
│   └─ deploy_naut      → src/commands/deploy_naut.rs
│   └─ sample           → src/commands/sample.rs
├─ deployment_details   → src/deployment_details.rs
├─ env                  → src/env.rs
├─ error                → src/error.rs                (ApiResponse, Error)
├─ http                 → src/http/mod.rs
│   └─ request_client   → src/http/request_client.rs
├─ io                   → src/io/mod.rs
│   └─ stderr_io        → src/io/stderr_io.rs
├─ options              → src/options.rs
├─ tui                  → src/tui/mod.rs
│   └─ spinner          → src/tui/spinner.rs
```

That's why `main.rs` can write things like `use nautpie::cli::Cli;` or `use nautpie::error::{ApiResponse, Error};` — every piece is wired up here.

Why Is It Written This Way?

- **Binary + library split.** Rust makes it trivial to have both at once: declare a binary in `Cargo.toml` with `[[bin]]` and a library with `[lib]`. The library has its own entry point (`src/lib.rs`); the binary has its own (`src/main.rs`). They share the source tree but produce two separate compilation artifacts.
- **Why not just put everything in `main.rs`?** Two big reasons: 1. **Integration tests** in `tests/*.rs` need to `use nautpie::something;` — they can only do that if the code is in the library, not the binary. 2. **Reusability.** If someone wants to call `nautpie` as a library function rather than spawning the binary, they can. The `examples/` directory in the crate root demonstrates this.
- **Why re-export `ApiResponse` and `Error`?** These are the two types a caller is most likely to want. Putting them at the crate root means shorter `use` statements in client code.
- `#![warn(missing_docs)]` — this is the single most important documentation improvement from the Azure SDK guidelines review. Without it, public items can quietly go undocumented. With it, every future PR that adds an undocumented `pub fn` will produce a compiler warning.
- `#![doc = include_str!("../README.md")]` — keeps `cargo doc` in sync with GitHub. Without it, the rendered documentation has a thin placeholder intro; with it, callers see the full README including examples and troubleshooting.
- **Doc comments** (`///`). The triple-slash comments at the top are **inner doc comments** — they document the **module itself**, not an item inside it.

Key Rust Concepts Used Here

Concept	Where	What it does
<code>//!</code>	doc comment	Inner doc comment — documents the enclosing module
<code>pub mod name;</code>	module decls	Declares a public module wired up to <code>src/name.rs</code> or <code>src/name/mod.rs</code>
<code>pub use path::Item;</code>	re-export	Re-exports an item so callers can use a shorter path
<code>lib.rs</code> vs <code>main.rs</code>	entry points	The library and binary entry points of a Cargo project
<code>#![warn(lint)]</code>	crate attribute	Promote a lint to a warning for the whole crate
<code>#![doc = "..."]</code>	crate attribute	Set the crate-level doc comment programmatically
<code>include_str!("path")</code>	macro	Embed a file's contents as a <code>&'static str</code> at compile time
<code>\`no_run   doc fence   Code compiles but doesn't run as part of cargo test`</code>		
<code>\`rust no_run`</code>	doc fence	Same, explicitly tagged as Rust

How `main.rs` Uses This

Every `use` statement at the top of `main.rs` resolves through this file:

```
use nautpie::cli::{normalise_action, Cli, CommandKind};
use nautpie::commands::bitbucket::Bitbucket;
use nautpie::commands::deploy_naut::{
    options_from_bitbucket_args, options_from_deploy_args, DeployNaut,
};
use nautpie::commands::Command;
use nautpie::env::load_dotenv;
use nautpie::error::{ApiResponse, Error};
use nautpie::http::{HttpClient, ReqwestHttpClient};
use nautpie::io::{Io, StderrIo};
```

Every one of these paths starts from the crate root defined right here.

 Source: `src/main.md`

This file is the **front door** of the `nautpie` binary. When you run `nautpie deploy:naut createDeployment`, the code inside `main()` is what gets executed first.

Think of `main.rs` as the **conductor of an orchestra**: it does not play any instruments itself (it does not actually talk to APIs), but it tells every other piece when to start, what to do, and how to report back.

The Big Picture

The whole program boils down to five steps:

1. Load environment variables from a `.env` file (if one exists).
2. Parse the command-line arguments using `clap`.
3. Set up the "output sink" (stderr) and the "HTTP client" (request).
4. Dispatch to the right subcommand (`deploy:naut` or `ci:bitbucket`) with the right action.
5. Print a single JSON line on stdout, then exit with code `0` (success) or `1` (failure).

That's the **contract** with the outside world: one JSON line on stdout, an exit code, and that's it. Everything else (status messages, warnings, progress bars) goes to stderr so it doesn't pollute the JSON the caller is parsing.

Walkthrough

The `main()` function

```
fn main() {
    let _ = load_dotenv();

    let cli = Cli::parse();
    let mut io = StderrIo::new();
    let mut http = ReqwestHttpClient::new().expect("request client build");

    let exit_code = dispatch(cli, &mut io, &mut http);
    std::process::exit(exit_code);
}
```

Every Rust program starts at a function called `main`. It takes no arguments and returns nothing (well, technically it returns `()` — Rust's name for "nothing"). Anything you want to return to the operating system must be passed to `std::process::exit()`.

A few teaching points:

- `let _ = load_dotenv();` — The underscore prefix `_` tells the compiler: "I know this returns a value, but I deliberately don't want to use it." Without that underscore, the compiler would warn you about an unused result. The `Result` returned by `load_dotenv()` is silently discarded because loading `.env` is **best-effort**: if there's no `.env` file, that's fine, we just use real environment variables.
 - `Cli::parse()` — `Cli` is a struct defined in `src/cli.rs`. The `#[derive(Parser)]` attribute from the `clap` crate generates all the argument-parsing code for you at compile time. `parse()` looks at `std::env::args()` (the actual `argv` from the shell) and produces a fully-populated `Cli` struct.
 - `let mut io = StderrIo::new();` — The `mut` keyword means **mutable**. By default, Rust variables are immutable (read-only). `io` needs to be mutable because we'll be calling methods like `set_options` on it that change its internal state.
 - `.expect("request client build")` — `ReqwestHttpClient::new()` returns a `Result`. If it's an `Err`, `expect` will **panic** (crash the program) with the given message. We use it here because if we can't even build an HTTP client, there's no point continuing. Azure SDK guidelines discourage `expect`, but a process-level init that can never realistically fail is an acceptable exception.
 - `std::process::exit(exit_code)` — terminates the program immediately with the given exit code. Unix convention: `0` means success, anything else means failure.
-

The `dispatch()` function

```
fn dispatch(cli: Cli, io: &mut dyn Io, http: &mut dyn HttpClient) -> i32 {
    let action_raw = match &cli.command {
        CommandKind::DeployNaut(args) => args.action.clone(),
        CommandKind::Bitbucket(args) => args.action.clone(),
    };

    match &cli.command {
        CommandKind::DeployNaut(args) => io.set_options(options_from_deploy_args(args)),
        CommandKind::Bitbucket(args) => io.set_options(options_from_bitbucket_args(args)),
    }

    let result = match cli.command {
        CommandKind::DeployNaut(_) => {
            let cmd = DeployNaut;
            cmd.run(&normalise_action(&action_raw), io, http)
        }
        CommandKind::Bitbucket(_) => {
            let cmd = Bitbucket;
            cmd.run(&normalise_action(&action_raw), io, http)
        }
    };

    finish(result, io)
}
```

Teaching points:

- `fn dispatch(cli: Cli, io: &mut dyn Io, http: &mut dyn HttpClient) -> i32` — Three parameters, returns an `i32` (the exit code). The `&mut dyn Io` and `&mut dyn HttpClient` types are interesting:
- `&mut` means "a mutable reference." We're lending the function the right to modify `io` and `http`, but ownership stays with `main()`.
- `dyn Io` means "any type that implements the `Io` trait." This is **dynamic dispatch** — Rust will look up which method to call at runtime through a vtable. We use this so tests can swap in a fake `Io` that records what was written.
- **The first `match`** extracts the raw action string (`sampleSuccess`, `createAccessToken`, etc.) by matching on the `CommandKind` enum.
- **The second `match`** puts the parsed CLI options onto the `io` object. This is a bit unusual — it's stashing data on the `io` object so the action functions can later pull it back out with `io.options()`. Why? Because the `Command` trait signature only takes `&mut dyn Io` and `&mut dyn HttpClient`, not the original CLI args. The `Io` trait acts as a **side-channel** for passing options.
- **`cli.command` is moved twice?** — No, Rust is smart about this. The first two matches use `&cli.command` (borrow), and only the third one moves it. That's why the first two don't have `mut`.
- `&normalise_action(&action_raw)` — `normalise_action` is a helper that converts any case style (`sampleSuccess`, `SAMPLESUCCESS`, `sample_success`, `Sample-Success`) into the canonical `lowerCamelCase` form. The `&` means we're passing a borrow, so the function can read the string without taking ownership.

The `finish()` function

```
fn finish(result: Result<ApiResponse, Error>, io: &mut dyn Io) -> i32 {
    match result {
        Ok(response) => {
            io.write_json(&response);
            0
        }
        Err(err) => {
            io.warning(&err.to_string());
            let body = serde_json::Value::String(err.to_string());
            let response = ApiResponse::error(1, "Bad Request", body);
            io.write_json(&response);
            1
        }
    }
}
```

Teaching points:

- **`Result<ApiResponse, Error>`** — Rust's built-in enum for "either success or failure." It has two variants: `Ok(value)` for success and `Err(error)` for failure. There's no exceptions in Rust — errors are values that you pass around.
- **`io.write_json(&response)`** — `write_json` is defined on the `Io` trait. Because we passed in `&mut dyn Io`, Rust calls the right implementation at runtime.
- **The error branch** does two things: prints a human-readable warning on stderr, then **also** prints a JSON line on stdout with status `1` and reason `"Bad Request"`. This is for **CI scripts that parse stdout**: they always get exactly one JSON line, whether things went well or badly. The warning on stderr is a bonus for humans watching the terminal.
- **The implicit return** — note the function has no `return` keyword and no semicolon on the last expression of each match arm. In Rust, the last expression in a block (without a semicolon) is the value of that block. So `0` and `1` are returned implicitly.

Why Is It Written This Way?

- **One JSON line, always.** The whole binary is designed so that a CI script can pipe stdout to `jq` and get a clean structured answer. That's why all progress messages go to stderr.
- **Trait objects (`dyn Io`, `dyn HttpClient`).** These let tests substitute fake implementations. The integration tests in `tests/` rely on this heavily.
- **A thin `main.rs`.** All the real logic lives in libraries (`src/lib.rs` and modules). This makes the code easier to test (you don't need to spawn a real process) and easier to embed in another program if you ever want to.
- **`finish()` is its own function.** It exists so the dispatcher has a single, clean exit point — every path through the program ends by calling `finish`, which handles both success and failure uniformly.

Key Rust Concepts Used Here

Concept	Where	What it does
<code>fn main()</code>	top of file	Required entry point of every Rust binary
<code>let mut</code>	in <code>main()</code>	Declares a mutable variable
<code>let _ = ...</code>	<code>load_dotenv()</code>	Discards a <code>Result</code> you intentionally don't use
<code>Result<T, E></code>	everywhere	Rust's success-or-error type
<code>match</code>	<code>dispatch</code> , <code>finish</code>	Pattern matching — like a <code>switch</code> but exhaustive
<code>&mut dyn Trait</code>	function params	Mutable reference to "some type implementing this trait"
<code>.expect(msg)</code>	<code>RequestHttpClient::new()</code>	Panics with <code>msg</code> if the <code>Result</code> is <code>Err</code>
<code>.parse()</code>	(called from <code>cli.rs</code>)	Parses a string into another type
<code>enum</code>	<code>CommandKind</code>	A sum type with a fixed set of variants
<code>String::clone()</code>	<code>args.action.clone()</code>	Allocates a new <code>String</code> with the same contents
<code>std::process::exit()</code>	end of <code>main()</code>	Terminates the program with an exit code

CHAPTER 2: PRIMITIVES, PATTERN MATCHING, OPTION

 Source: `src/boolean.md`

This is the smallest file in the codebase. It exports **one function** that does **one thing**: convert a string like `"yes"` or `"no"` into a real `bool`. Despite its size, it teaches some genuinely useful Rust patterns.

The Whole File

```
pub fn check_boolean(value: Option<&str>) -> Result<bool, Error> {
    match value.map(str::trim).map(str::to_ascii_lowercase) {
        None => Ok(false),
        Some(v) if v.is_empty() => Ok(false),
        Some(v) if matches!(v.as_str(), "false" | "no" | "0") => Ok(false),
        Some(v) if matches!(v.as_str(), "true" | "yes" | "1") => Ok(true),
        Some(v) => Err(Error::Generic(format!(
            "[Boolean] Value not accepted as boolean: {v}"
        ))),
    },
}
```

That's it. One function, one match, five arms.

Walkthrough

The function signature

```
pub fn check_boolean(value: Option<&str>) -> Result<bool, Error>
```

- `Option<&str>` — the input is "either a string slice, or nothing." This matches the way `clap` represents optional CLI flags: `Option<String>`. The caller can pass `Some("yes")`, `Some("")`, or `None`.
- `Result<bool, Error>` — output is "either a real boolean, or our error type."

The match

The body is a single `match` expression with five arms. Let's go through it one piece at a time.

```
value.map(str::trim).map(str::to_ascii_lowercase)
```

This is the most interesting line in the file, so let's break it down carefully.

- `Option::map(closure)` — a method on `Option<T>`. If the `Option` is `Some(x)`, it applies the closure to `x` and returns `Some(result)`. If the `Option` is `None`, it returns `None` unchanged. Think of it as "transform the inside of an `Option`, leaving `None` alone."
- `str::trim` — passed as a **function pointer**. The full signature of `str::trim` is `fn trim(&self) -> &str`. When we pass it to `.map()`, Rust uses it like a closure: `|s| s.trim()`. Same effect, shorter syntax.
- `str::to_ascii_lowercase` — same trick. Signature: `fn to_ascii_lowercase(&self) -> String`. So `value.map(str::to_ascii_lowercase)` would convert the inner string to lowercase.

The two `.map` calls are **chained**. Each one transforms the value if there is one:

1. First, trim whitespace from the string (e.g. `" yes "` → `"yes"`).
2. Then, lowercase the result (e.g. `"YES"` → `"yes"`).

After these two maps, we have:

- `None` — if the input was `None`
- `Some("yes")` — if the input was `Some("yes")`, `Some(" YES ")`, etc.
- `Some("")` — if the input was `Some("")`, `Some(" ")`, etc.

The match arms

Now we match on the normalised value. Each arm uses a **match guard** — an extra `if` condition after the pattern:

```
None => Ok(false),
Some(v) if v.is_empty() => Ok(false),
Some(v) if matches!(v.as_str(), "false" | "no" | "0") => Ok(false),
Some(v) if matches!(v.as_str(), "true" | "yes" | "1") => Ok(true),
Some(v) => Err(Error::Generic(format!(
    "[Boolean] Value not accepted as boolean: {v}"
))),
```

- `None => Ok(false)` — no input given? Default to `false`.
- `Some(v) if v.is_empty() => Ok(false)` — empty string after trimming? Default to `false`.
- `Some(v) if matches!(v.as_str(), "false" | "no" | "0") => Ok(false)` — explicit `false` synonyms. The pattern guard (`if matches!(...)`) refines the `Some(v)` arm so it only matches when the string is one of those three.
- `Some(v) if matches!(v.as_str(), "true" | "yes" | "1") => Ok(true)` — explicit `true` synonyms.
- `Some(v) => Err(...)` — the **catch-all arm**. Anything that survived the guards but isn't a recognised truthy/falsy string is an error.

Teaching points:

- **Pattern guards (`if condition`)** — they let you add extra conditions to a match arm without nesting more `match` or `if` statements. Very useful for matching on a range of values.
- `matches!(v.as_str(), "true" | "yes" | "1")` — the same `matches!` macro from `env.rs`. It returns `true` if the expression matches the pattern.

- `format!("{}", v)` — string interpolation. `{v}` is shorthand for `{}` when the variable's name is the same as the placeholder.
- **The order matters.** `Some(v) if v.is_empty()` must come **before** `Some(v) if matches!(v.as_str(), "false" | "no" | "0")` because both have a `Some(v)` pattern — but the empty-string check is more specific (the other patterns wouldn't match an empty string anyway, but ordering it first makes the intent clearer).

Why `str::trim` and not `value.trim()` ?

Both would work. Writing `str::trim` (passing the function reference) is idiomatic when you're chaining transformations through `.map()`. It reads like a pipeline: "take the value, map with trim, map with lowercase."

If you wrote `value.map(|s| s.trim())`, it would be exactly equivalent but longer.

Why Is It Written This Way?

- **PHP `CheckHelper::checkBoolean` parity.** The original PHP function was deliberately lenient: anything missing counts as `false`, empty counts as `false`, a few specific strings count as `true` or `false`, and anything else throws. The Rust port preserves every one of those cases.
- **`Option<&str>` over `&str`.** By taking `Option<&str>`, the caller can pass through `Option::None` directly from `clap` without writing `if let Some(...)` everywhere. The signature matches the shape of the data.
- **Match guards over nested `if`.** Five arms with guards is more readable than one arm with five nested `if-else` blocks. Each arm reads top-to-bottom as a separate case.
- **Custom error message.** `"[Boolean] Value not accepted as boolean: maybe"` is more helpful than a generic error. The `[Boolean]` prefix matches the PHP original's error format.

Key Rust Concepts Used Here

Concept	Where	What it does
<code>Option<&str></code>	param	"Either a string slice, or nothing"
<code>Result<bool, Error></code>	return	"Either a <code>bool</code> , or our error type"
<code>Option::map(closure)</code>	the chained maps	"Transform the inside of an <code>Option</code> "
Function pointer <code>str::trim</code>	first <code>.map</code>	Passing a function as a value
Method chaining <code>.map().map()</code>	the pipeline	Calling multiple methods in sequence
<code>match</code> with guards	the body	Pattern matching with extra <code>if</code> conditions
<code>matches!(expr, pattern)</code>	inside guards	"Does this match the pattern?"
Pattern OR <code> </code>	inside <code>matches!</code>	"Either this or that"
<code>format!("{}", v)</code>	error case	String interpolation
Implicit return	every arm	The last expression (no semicolon) is the value
<code>#[cfg(test)]</code>	tests	Only compile in test mode

 Source: `src/env.md`

This file is a thin wrapper around three small but important operations:

1. `load_dotenv()` — read a `.env` file from the current directory.

2. `check_envs()` — verify that a set of required environment variables are set, and return their values.
3. `is_production()` — check whether a given environment name refers to production.

It's deliberately tiny. Each function is a few lines, and there are no fancy abstractions.

Walkthrough

`load_dotenv()`

```
/// Load `.env` from the current directory. Missing files are ignored, matching
/// PHP `safeLoad`. Returns `true` if a `.env` was loaded, `false` otherwise.
pub fn load_dotenv() -> bool {
    dotenvy::dotenv().is_ok()
}
```

The whole function is one line. Let's unpack it:

- `dotenvy::dotenv()` — searches the current directory (and walks up the directory tree) for a file named `.env`. If found, it reads each line of the form `KEY=value` and sets the corresponding environment variable. It returns `Result<(), Error>`.
- `.is_ok()` — a method on `Result` that returns `true` if the `Result` is `Ok(_)`, `false` if it's `Err(_)`. We don't care about the specifics of success or failure — only whether anything happened.
- `-> bool` — the return type. The function tells its caller "did a `.env` file get loaded?" but the caller (in `main.rs`) **deliberately ignores even this**: `let _ = load_dotenv();`. Why? Because the PHP original's `safeLoad` was also fire-and-forget — it never threw on a missing `.env`.

Why is the missing-file case ignored? Because `.env` files are a **developer convenience**, not a deployment requirement. In CI or production, you'd set real environment variables on the machine. If `.env` is missing, the program should still work using real env vars.

`check_envs()`

```
pub fn check_envs(names: &[&str]) -> Result<Vec<String>, Error> {
    let mut out = Vec::with_capacity(names.len());
    for name in names {
        let value = std::env::var(name).map_err(|_| Error::MissingEnv((*name).into()))?;
        if value.is_empty() {
            return Err(Error::MissingEnv((*name).into()));
        }
        out.push(value);
    }
    Ok(out)
}
```

This function takes a **slice of string references** (`&[&str]`) and returns either a `Vec<String>` (one value per requested variable) or an `Error`.

Teaching points:

- `&[&str]` — this is a slice. The outer `&` means "borrowed reference." The inner `&str` means "borrowed string." So we're borrowing a list of borrowed strings. The caller (in `deploy_naut.rs`) writes `check_envs(&["DASH_USER", "DASH_TOKEN"])`.
- `Vec::with_capacity(names.len())` — creates an empty vector that has reserved space for `names.len()` elements. This avoids the vector needing to grow (and reallocate) as we push to it.
- `std::env::var(name)` — looks up an environment variable by name. Returns `Result<String, VarError>`. The `?` operator on the next line says: "if it's an `Err`, return that error early from this function."
- `.map_err(|_| Error::MissingEnv((*name).into()))?` — this is a **closure** (a tiny inline function). It converts the standard library's error type into our own `Error::MissingEnv`. The `|_|` means "I take one argument but I don't care to name it." The `(*name).into()` converts `&String` (since `name` is `&str`, dereferencing once gives `str`) to `String`.

- `if value.is_empty()` — PHP's `$_SERVER[$name]` returns `""` for set-but-empty values, while `getenv()` returns `false`. The Rust port treats both cases as "missing" because neither is useful.
- `out.push(value)` — appends to the vector. `value` is moved, not cloned.
- `Ok(out)` — successful return.

Why return all values in a `Vec`? Because the caller usually wants them as a tuple: `let creds = check_envs(&["DASH_USER", "DASH_TOKEN"]); let user = &creds[0]; let token = &creds[1];`. Returning a `Vec` keeps the function generic over how many env vars you ask for.

`is_production()`

```
pub fn is_production(env: &str) -> bool {
    matches!(env.to_ascii_lowercase().as_str(), "prod" | "production")
}
```

This is a one-liner but it teaches two nice patterns:

- `matches!(expr, pattern)` — a macro that returns `true` if the expression matches the pattern, `false` otherwise. Cleaner than writing `match env { ... }` for a single boolean check.
- `.to_ascii_lowercase().as_str()` — converts `&str` to a `String` (owned, lowercase), then `.as_str()` borrows it back as `&str`. We need this because we can't pattern-match against the result of a method call directly.
- `"prod" | "production"` — a **pattern OR**. Either match works.

The function is used in `deploy_naut.rs` to decide whether to apply the `bypass_and_start` and `redploy` flags. The reasoning: **you should never bypass or redeploy production deployments**. Those flags are silently ignored if `environment == "prod"` or `"production"`.

Why Is It Written This Way?

- **PHP parity.** The PHP original used `vlucas/phpdotenv` with `safeLoad`, which silently ignores missing files. The Rust port preserves that exact behavior so the migration is invisible.
- **Fail fast.** `check_envs` returns `Error::MissingEnv` at the **first** unset variable, not after collecting all of them. This produces clearer error messages: "X is missing" rather than "X, Y, and Z are missing."
- **Treat empty as missing.** This matches PHP's loose semantics where `""` and `null` are both "no value."
- **`is_production` is case-insensitive.** Users (and CI scripts) sometimes write `PROD`, sometimes `prod`, sometimes `Production`. All should be recognised.

Key Rust Concepts Used Here

Concept	Where	What it does
<code>-> bool</code>	<code>load_dotenv</code>	Function return type
<code>.is_ok()</code>	<code>load_dotenv</code>	"Is this <code>Result</code> an <code>Ok</code> ?"
<code>&[&str]</code>	<code>check_envs</code> param	A borrowed slice of borrowed strings
<code>Vec::with_capacity(n)</code>	<code>check_envs</code>	Pre-allocates space for <code>n</code> elements
<code>Result<T, E></code>	<code>check_envs</code> return	The standard success-or-failure type
<code>? operator</code>	<code>check_envs</code>	"If <code>Err</code> , return early with that error"
<code>.map_err(closure)</code>	<code>check_envs</code>	Convert one error type to another
Closure <code>\ _ </code>	<code>check_envs</code>	An inline function with a single unnamed argument
<code>.into()</code>	<code>check_envs</code>	Type conversion via <code>From</code> / <code>Into</code>
<code>matches!(expr, pattern)</code>	<code>is_production</code>	Pattern-match an expression in one line
<code>"a" \ "b"</code>	<code>is_production</code>	Pattern OR — match either
<code>.to_ascii_lowercase()</code>	<code>is_production</code>	Returns a new <code>String</code> in lowercase
<code>.as_str()</code>	<code>is_production</code>	Borrow a <code>String</code> back as <code>&str</code>
<code>#[cfg(test)]</code>	tests	Only compile in test mode

CHAPTER 3: ERROR HANDLING WITH RESULT

 Source: `src/error.md`

This file defines two things that are absolutely central to the program:

1. `Error` — every error the program can produce, as a Rust enum.
2. `ApiResponse` — the JSON envelope printed on stdout for every command, success or failure.

Together, they form the **public contract** between `nautpie` and whatever script is calling it. If you're going to understand one file in this codebase, make it this one.

The Big Picture

Every command invocation produces exactly one JSON line on stdout with this shape:

```
{"status": 200, "reason": "OK", "body": <whatever>}
```

- `status` — HTTP-like status code. `200` for success, anything else for failure.
- `reason` — short human-readable label, like `"OK"` or `"Bad Request"`.
- `body` — the actual payload. Could be a string, an object, an array, `null` — anything.

When something goes wrong, the program returns `Err(Error::SomeVariant)` from the action function. The dispatcher in `main.rs` catches it and converts it into an `ApiResponse::error(...)` that still has the same three-field shape, so CI scripts can parse stdout uniformly.

Walkthrough

The `Error` enum

```
use thiserror::Error;

#[derive(Debug, Error, PartialEq, Eq)]
pub enum Error {
    #[error("[Required:ENV] {0} is missing.")]
    MissingEnv(String),

    #[error("[Required:Option] {0} is missing.")]
    MissingOption(String),

    #[error("[Missing] Action or End Point.")]
    MissingAction,

    #[error("[Missing] End Point is not configured.")]
    MissingEndpoint,

    #[error("[Timeout] {0}")]
    Timeout(String),

    #[error("[HTTP {status}] {reason}")]
    HttpStatus {
        status: u16,
        reason: String,
        body: String,
    },

    #[error("[Json] {0}")]
    Json(String),

    #[error("{0}")]
    Generic(String),
}
```

This is a **sum type** (a Rust enum with data attached). Each variant represents a different kind of failure:

- **`MissingEnv(String)`** — a required environment variable wasn't set. The `String` is the variable's name. This is a **tuple variant**: it carries an unnamed field.
- **`MissingOption(String)`** — a required CLI option wasn't supplied. Same shape.
- **`MissingAction`** — a unit variant: no data at all, just the tag.
- **`MissingEndpoint`** — the API endpoint env var wasn't set.
- **`Timeout(String)`** — a polling operation exceeded its deadline. The `String` is the descriptive message.
- **`HttpStatus { status, reason, body }`** — a **struct variant**: it has named fields, like a tiny struct. The HTTP request returned a non-success status.
- **`Json(String)`** — JSON encoding or decoding failed.
- **`Generic(String)`** — a catch-all for anything else.

Teaching points:

- **`#[derive(Error)]`** — comes from the `thiserror` crate. It generates an `impl std::error::Error for Error` (and an `impl Display for Error`) automatically, so `Error` integrates with Rust's standard error-handling ecosystem.
- **`#[error("...")]`** — the format string for the error's `Display` implementation. `{0}` is the first field, `{name}` is the named field. When you `println!("{}", err)`, you get that formatted string.
- **`PartialEq, Eq`** — derives for equality. Two `Error` values can be compared with `==`. The tests at the bottom rely on this.
- **Tuple variants vs. struct variants** — Both hold data. Tuple variants are concise (`MissingEnv(String)`); struct variants are self-documenting (`HttpStatus { status, reason, body }`). Use struct variants when there are 2+ fields and the names matter.

Why `thiserror`?

In plain Rust, you'd write:

```
impl std::fmt::Display for Error {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        match self {
            Error::MissingEnv(name) => write!(f, "[Required:ENV] {name} is missing."),
            // ... 7 more arms ...
        }
    }
}

impl std::error::Error for Error {}
```

The `#[derive(Error)]` macro generates all of that for you from the `#[error("...")]` attributes. Less boilerplate, less chance of forgetting an arm.

The `ApiResponse` struct

```
#[derive(Debug, Clone, Serialize, serde::Deserialize)]
pub struct ApiResponse {
    #[serde(rename = "status")]
    pub status: u16,
    #[serde(rename = "reason")]
    pub reason: String,
    #[serde(rename = "body")]
    pub body: serde_json::Value,
}
```

This is the **envelope** that goes on stdout. Three fields:

- **status: u16** — an unsigned 16-bit integer (0–65535). Real HTTP status codes fit easily in this range.
- **reason: String** — a short label like `"OK"`, `"Bad Request"`, `"Internal Server Error"`.
- **body: serde_json::Value** — the actual payload. `serde_json::Value` is an **enum** that can be any JSON value: `Null`, `Bool`, `Number`, `String`, `Array`, or `Object`. This is how we get a "body of any shape" without writing a different struct for every action.

Teaching points:

- `#[derive(Serialize, Deserialize)]` — auto-generates code to convert this struct to and from JSON. Without these, you couldn't print it.
- `#[serde(rename = "status")]` — already redundant here because the field is already named `status`, but it's kept explicit so the JSON key matches even if the field is renamed in code.
- **Clone** — lets you duplicate an `ApiResponse` cheaply. Useful for tests.

The constructor methods

```
impl ApiResponse {
    pub fn ok(body: serde_json::Value) -> Self {
        Self {
            status: 200,
            reason: "OK".into(),
            body,
        }
    }

    pub fn error(status: u16, reason: &str, body: serde_json::Value) -> Self {
        Self {
            status,
            reason: reason.into(),
            body,
        }
    }

    pub fn to_line(&self) -> String {
        serde_json::to_string(self).unwrap_or_else(|_| {
            r#"{"status":1,"reason":"Bad Request","body":"json encode failed"}"#.to_string()
        })
    }
}
```

- `ok(body)` — convenience for the success case. Always produces `status=200, reason="OK"`.
- `error(status, reason, body)` — convenience for failures. The caller picks the status code and reason.
- `to_line(&self) -> String` — serializes to a single line of JSON (no newline at the end). The `unwrap_or_else` clause returns a fallback if serialization itself fails (essentially "couldn't even encode the error").
- `"OK".into()` — the `From<&str> for String` conversion. Shorter than `"OK".to_string()`.

Tests

The `#[cfg(test)]` block tests that `ApiResponse` round-trips correctly through JSON for all three body shapes (object, string, array). This is important because if the envelope shape ever changes, downstream CI scripts will silently break.

Why Is It Written This Way?

- **Single JSON line, always.** Whether the command succeeds or fails, the caller gets exactly one parseable JSON line on stdout. This is the **PHP parity contract** — the original PHP version did the same thing.
- **Error as an enum, not a string.** Rust strongly prefers typed errors. Every function returning `Result<T, Error>` lets the caller pattern-match on the specific failure: `match err { Error::MissingEnv(name) => ..., Error::Timeout(_) => ... }`. Compare with a stringly-typed error where you'd have to parse the message text.
- **`#[error("...")]` for messages.** The PHP original prefixed each error kind (`[Required:ENV]` , `[Timeout]` , etc.). The Rust port keeps those prefixes so the error text is byte-identical, making migration painless.
- **`body: serde_json::Value`** . Each action returns completely different data. Rather than write a generic `ApiResponse<T>` (which would force every caller to deal with generics), the body is `Value` — Rust's "any JSON" type.
- **thiserror over manual impl** . Less boilerplate, less chance of typos in the `Display` impls, automatic `std::error::Error` integration.

Key Rust Concepts Used Here

Concept	Where	What it does
<code>enum</code> with data	<code>Error</code>	A type with a fixed set of variants, each possibly carrying data
Tuple variant	<code>MissingEnv(String)</code>	Variant carrying unnamed fields
Struct variant	<code>HttpStatus { status, reason, body }</code>	Variant carrying named fields
Unit variant	<code>MissingAction</code>	Variant carrying no data
<code>#[derive(...)]</code>	everywhere	Auto-generates trait implementations
<code>thiserror::Error</code>	<code>Error</code>	Generates <code>Display</code> + <code>std::error::Error</code> impls
<code>serde::Serialize</code>	<code>ApiResponse</code>	Makes a type serializable to JSON
<code>serde::Deserialize</code>	<code>ApiResponse</code>	Makes a type deserializable from JSON
<code>serde_json::Value</code>	<code>body</code> field	A type that can hold any JSON value
<code>Into</code> / <code>From</code>	<code>"OK".into()</code>	Type conversion via trait
<code>unwrap_or_else(_ ...)</code>	<code>to_line</code>	"If this fails, run this fallback"
<code>Result<T, E></code>	callers	The standard success-or-failure return type
<code>#[cfg(test)]</code>	test module	"Only compile in test mode"

CHAPTER 4: DATA STRUCTURES

 Source: `src/options.md`

This file defines a single type, `Options`, that acts as a **key-value store** for parsed CLI options. It's a tiny abstraction, but it's central to how the action functions receive their inputs.

The Big Picture

Here's the chain of how data flows from the command line into an action function:

1. User types `nautpie deploy:naut createDeployment --stack=foo --environment=uat`.
2. `clap` parses that into a `DeployNautArgs` struct (in `cli.rs`).
3. `main.rs` calls `options_from_deploy_args(args)` (in `deploy_naut.rs`), which produces an `Options`.
4. `main.rs` stuffs the `Options` into the `Io` impl via `io.set_options(...)`.
5. The action function reads it back via `io.options()`.

The reason for step 4 — stuffing options through the `Io` trait — is so the `Command::run` trait method can take a uniform `&mut dyn Io` parameter without needing to thread the original CLI args through every layer.

This file defines the bag itself.

The Whole File

```
use std::collections::HashMap;

#[derive(Debug, Clone, Default)]
pub struct Options {
    values: HashMap<String, String>,
}

impl Options {
    pub fn new() -> Self {
        Self::default()
    }

    pub fn set(&mut self, key: impl Into<String>, value: impl Into<String>) {
        self.values.insert(key.into(), value.into());
    }

    pub fn get(&self, key: &str) -> Option<String> {
        self.values.get(key).cloned()
    }

    pub fn option(&self, name: &str) -> Option<String> {
        self.get(name)
    }
}
```

That's the whole file. Let's break down every line.

Walkthrough

The struct

```
#[derive(Debug, Clone, Default)]
pub struct Options {
    values: HashMap<String, String>,
}
```

- `HashMap<String, String>` — Rust's standard hash map, imported from `std::collections`. It maps string keys to string values. This is the underlying storage for all parsed CLI options.
- `#[derive(Debug)]` — so you can print it with `{:?}` for debugging.
- `#[derive(Clone)]` — so you can `.clone()` an `Options` value. The `io` trait's `options()` method returns `Option<Options>`, which means cloning is necessary when handing it back to callers.
- `#[derive(Default)]` — generates a `Default::default()` that produces an empty `Options`. This is what `Options::new()` calls.

Note the field is **not** `pub` — it's private. Callers must use the `set`, `get`, and `option` methods. This is good encapsulation: the internal representation can change (e.g., to a `BTreeMap` for deterministic ordering) without breaking callers.

`new()`

```
pub fn new() -> Self {
    Self::default()
}
```

A trivial constructor. Returns an empty `Options`. The `Self::default()` call uses the `Default` trait derived above. This pattern (`new()` calls `default()`) is idiomatic in Rust.

set()

```
pub fn set(&mut self, key: impl Into<String>, value: impl Into<String>) {
    self.values.insert(key.into(), value.into());
}
```

This is the most interesting method because of the parameter types.

- `&mut self` — mutable borrow. We're modifying the `Options`, so the caller must also have a mutable reference. (In practice, the only place that calls `set` is `options_from_deploy_args`, which builds a fresh `Options` and so has full ownership.)
- `key: impl Into<String>` — `impl Trait` syntax. This says: "I don't care what concrete type you pass, as long as it can be converted into a `String`." So `set("name", "value")` works because `&str` implements `Into<String>` (it allocates and copies). `set(String::from("name"), String::from("value"))` also works. `set(format!("k-{}", 1), "v")` works too. This is **maximum flexibility** for callers.
- `.insert(key.into(), value.into())` — convert both arguments into `String` and put them in the hash map. `HashMap::insert` will overwrite any existing value for the same key.

Why `impl Into<String>` and not just `String`? Because most callers in `deploy_naut.rs` are passing `Option<String>` results. They have to do `if let Some(v) = &args.url { opts.set("url", v.clone()); }`. Without the `Into` bound, they'd have to write `v.clone()` explicitly; with it, they could pass `v` directly (since `&String` implements `Into<String>`). It saves a small amount of boilerplate at every call site.

get()

```
pub fn get(&self, key: &str) -> Option<String> {
    self.values.get(key).cloned()
}
```

- `&self` — immutable borrow. We're reading, not modifying.
- `key: &str` — borrowed string slice. Callers don't need to pass owned `String`s.
- `self.values.get(key)` — returns `Option<&String>` (a reference to the value, or `None`).
- `.cloned()` — converts `Option<&String>` to `Option<String>` by cloning the inner value. This is necessary because returning a reference would tie the returned `Option<String>` to the lifetime of `self`, which is a hassle. Cloning is cheap for short strings.

option()

```
pub fn option(&self, name: &str) -> Option<String> {
    self.get(name)
}
```

A thin alias for `get()`. Why have both?

- `get` is the **generic, hash-map-flavoured** name.
- `option` is the **domain-flavoured** name — it's how the action functions in `deploy_naut.rs` read options.

It exists because the PHP original had a method called `option()` on its helper class. The Rust port keeps the same call style for parity: `opts.option("stack")` rather than `opts.get("stack")`. Functionally they're identical; the rename is purely cosmetic.

How It's Used

In `deploy_naut.rs`:

```
pub fn options_from_deploy_args(args: &DeployNautArgs) -> Options {
    let mut opts = Options::new();
    if let Some(v) = &args.url {
        opts.set("url", v.clone());
    }
    // ... many more like this ...
    opts
}
```

And in an action function:

```
let stack = opts
    .option("stack")
    .ok_or_else(|| Error::MissingOption("stack".into()))?;
```

The pattern is: `opts.option(...)` returns `Option<String>`, then `.ok_or_else(|| Error::MissingOption(...))` converts `None` to an error, and `?` propagates that error up to the caller.

Why Is It Written This Way?

- **Uniform access pattern.** Every action function reads its inputs the same way: `opts.option(...)`. They don't need to know which subcommand they're part of.
- **Late binding.** Options are stuffed into the `Io` impl in `main.rs`, not constructed inside the action. This decouples the action from the CLI parser — in principle, the same action could be triggered by a different input source (env vars, a config file) without changing its code.
- **Encapsulation.** The internal `HashMap` is private. Tomorrow's maintainer could swap it for a `BTreeMap` (sorted) or a `Vec<(String, String)>` (preserving insertion order) without touching any caller.
- **PHP parity.** The original `getOptions()` accessor on the PHP helper class behaved the same way: string in, string out, no validation. The Rust port mirrors that minimalism.

Key Rust Concepts Used Here

Concept	Where	What it does
<code>HashMap<K, V></code>	<code>values</code> field	Standard library hash map
<code>#[derive(Default)]</code>	struct	Generates <code>Default::default()</code>
<code>#[derive(Clone)]</code>	struct	Generates <code>Clone::clone()</code>
<code>impl Into<T></code>	<code>set</code> params	"Accepts anything convertible into <code>T</code> "
<code>.into()</code>	inside <code>set</code>	Performs the <code>Into</code> conversion
<code>Option<T></code>	return type	"Some value, or no value"
<code>.cloned()</code>	<code>get</code>	Convert <code>Option<&T></code> to <code>Option<T></code>
<code>Self::default()</code>	<code>new</code>	Calls the derived <code>Default</code> impl
<code>&mut self</code>	<code>set</code>	Mutable borrow — the method modifies state
<code>&self</code>	<code>get</code> , <code>option</code>	Immutable borrow — the method only reads
<code>impl Trait</code> in arg position	<code>set</code>	Generic over a concrete unknown type
Field privacy	<code>values</code>	No <code>pub</code> means the field is private

 Source: `src/deployment_details.md`

This file defines `DeploymentDetails`, a **builder-pattern struct** for constructing the JSON payload that gets POSTed to DeployNaut when you create a new deployment. It's the spiritual equivalent of the PHP `DeploymentDetails` class.

The Big Picture

When DeployNaut's API receives a `POST /project/{stack}/environment/{env}/deploys`, it expects a JSON body like:

```
{
  "ref": "abc123def456...",
  "ref_type": "sha",
  "title": "[CI] Deployment",
  "summary": "Branch:main",
  "bypass_and_start": true,
  "locked": false
}
```

The `DeploymentDetails` struct lets you build that body step by step:

```
let details = DeploymentDetails::new()
    .ref_("abc123def456...")
    .ref_type("sha")
    .title("[CI] Deployment")
    .summary("Branch:main")
    .bypass_and_start(true);

let json = details.values(); // → serde_json::Value
```

Each method returns `Self` (i.e., a modified copy of the struct), so the calls can be chained. This is the **builder pattern**, and it's one of the most popular idioms in Rust.

Walkthrough

Imports

```
use chrono::DateTime;
use serde_json::{Map, Value};
```

- `chrono::DateTime` — a date-and-time type from the `chrono` crate. Used to parse RFC3339 timestamps.
- `serde_json::{Map, Value}` — `Map` is the type of an object's key-value pairs (a wrapper around `BTreeMap` or `HashMap` depending on features); `Value` is the "any JSON" enum.

Constant

```
const DEFAULT_TITLE: &str = "[CI] Deployment";
```

A `const` is a value the compiler **inlines at every use site**. Unlike `let`, you can't have a mutable `const`, and the value must be known at compile time. Here it's just a default string.

The leading `[CI]` is a tag the DeployNaut UI shows in its dashboard. Production deployments get `[CD]` (continuous deployment) instead, but this struct defaults to `[CI]` to match the PHP original.

The struct

```
#[derive(Debug, Clone, PartialEq)]
pub struct DeploymentDetails {
    pub r#ref: Option<String>,
    pub ref_type: String,
    pub title: Option<String>,
    pub summary: Option<String>,
    pub bypass: bool,
    pub bypass_and_start: bool,
    pub schedule_start_unix: Option<i64>,
    pub schedule_end_unix: Option<i64>,
    pub locked: bool,
}
```

This is a plain data struct. Each field is `pub` so the constructor and methods (which live in the same module) can read and write them freely.

- `r#ref` — the raw-identifier escape again (the same trick we saw in `cli.rs`). `ref` is a reserved keyword in Rust, so we prefix it with `r#`. The actual JSON key will be `"ref"`.
- `Option<String>` for optional fields, plain `String` for required defaults, plain `bool` for booleans.
- `i64` for Unix timestamps (signed 64-bit integer). Unix epoch fits in `i64` until the year 292 billion, so we're safe.
- `#[derive(PartialEq)]` — generates `==` for the struct. Tests use this.
- `#[derive(Clone)]` — generates `.clone()`, useful for passing a copy of the payload around.

Default impl

```
impl Default for DeploymentDetails {
    fn default() -> Self {
        Self::new()
    }
}
```

This says: "the default `DeploymentDetails` is the same as a freshly-constructed one." It just delegates to `new()`. The derive macro couldn't auto-generate this for us because the defaults involve `Some(String::new())` and `Some("[CI] Deployment".into())` — values the macro can't infer.

`new()`

```
pub fn new() -> Self {
    Self {
        r#ref: Some(String::new()),
        ref_type: "sha".into(),
        title: Some(DEFAULT_TITLE.into()),
        summary: Some(String::new()),
        bypass: false,
        bypass_and_start: false,
        schedule_start_unix: None,
        schedule_end_unix: None,
        locked: false,
    }
}
```

Teaching points:

- `Some(String::new())` — wraps an empty string in `Some`. The struct starts with empty (but `Some`) optional fields so the builder can mutate them.
- `"sha".into()` — converts the `&str` literal to a `String`. The `into()` method works because `&str` implements `Into<String>`.
- `Some(DEFAULT_TITLE.into())` — the constant is `&str`, so we need `.into()` to make it `String` before wrapping in `Some`.
- `.into()` vs `String::from()` vs `.to_string()` — all three convert a `&str` to a `String`. They're interchangeable here; `into()` is just shorter.

Builder methods

Each builder method has the same shape:

```
pub fn ref_type(mut self, value: impl Into<String>) -> Self {
    self.ref_type = value.into();
    self
}
```

Teaching points:

- `mut self` — takes ownership of `self` (not a borrow). This is what enables the chaining pattern.
- `impl Into<String>` — same flexibility we saw in `Options::set()`. Callers can pass `&str`, `String`, or anything else convertible.
- `value.into()` — performs the conversion. Note we don't call `.to_string()` here because the compiler can't infer the target type from context; `into()` works because the destination type (`String`) is known.
- **Returns `self`** — returns the modified struct by value. The caller receives ownership and can call the next builder method.

`schedule_to_start` and `schedule_end`

```
pub fn schedule_to_start(mut self, time_str: impl AsRef<str>) -> Self {
    let raw = time_str.as_ref();
    self.schedule_start_unix = parse_unix(raw);
    self
}
```

- `impl AsRef<str>` — similar to `Into<String>` but a different trait. `AsRef` is for **borrowing** as a different type. Both `&str` and `String` implement `AsRef<str>` (the former is already a `&str`; the latter can be borrowed as one). This is more flexible than taking `&str` directly.
- `time_str.as_ref()` — converts whatever the caller passed into `&str`.

- `parse_unix(raw)` — calls a private helper at the bottom of the file that parses the time string.

redeploy

```
pub fn redeploy(mut self, yes: bool) -> Self {
    if yes {
        self.r#ref = Some(String::new());
        self.ref_type = "redploy".into();
    }
    self
}
```

The `if yes` block only runs when the flag is truthy. If you call `.redploy(false)`, nothing changes. If you call `.redploy(true)`, the ref is wiped and the `ref_type` becomes `"redploy"`.

This is the **PHP** `DeploymentDetails::redploy($yes)` semantics: when redeploying, you don't want to specify a commit SHA (you're re-running the previous one), so the ref is left empty and the API knows to use the last deployment.

values() — the serializer

```
pub fn values(&self) -> Value {
    let mut map = Map::new();
    let mut insert = |key: &str, val: Value| {
        if !matches!(val, Value::Null) {
            map.insert(key.into(), val);
        }
    };

    match &self.r#ref {
        Some(s) if !s.is_empty() => insert("ref", Value::String(s.clone())),
        _ => {}
    }

    insert("ref_type", Value::String(self.ref_type.clone()));
    insert("title", /* similar logic */);
    // ... and so on ...
    Value::Object(map)
}
```

This is the **rendering** method. It walks the struct, builds a `serde_json::Map<String, Value>` (the equivalent of a JSON object), and wraps it in `Value::Object(...)`.

Teaching points:

- `Map::new()` — creates an empty JSON object.
- The **insert closure** is a private helper that only inserts if the value is not `Value::Null`. This is how empty/null fields are stripped — it mirrors PHP's `array_filter($details, 'isNotNull')`.
- The **match for r#ref** has a **pattern guard** (`if !s.is_empty()`). If the ref is set to a non-empty string, insert it; otherwise, skip it entirely (don't even insert a null).
- `s.clone()` — clones the `&String` into a new owned `String` so we can move it into the `Value::String` constructor.
- `Value::Bool(...)`, `Value::String(...)`, `Value::Number(...).into()` — these are variants of the `serde_json::Value` enum. Each constructor takes the right type and wraps it.

parse_unix()

```
fn parse_unix(raw: &str) -> Option<i64> {
    if let Ok(n) = raw.parse::<i64>() {
        return Some(n);
    }
    DateTime::parse_from_rfc3339(raw)
        .ok()
        .map(|dt| dt.timestamp())
}
```

A private helper function (note: no `pub`, so it's only callable from within this module).

Teaching points:

- `raw.parse::<i64>()` — turbofish syntax to disambiguate the generic type. We're telling the compiler "parse this `&str` into an `i64`." Returns `Result<i64, ParseIntError>`.
- `if let Ok(n) = ...` — pattern-match on the `Result`. If it's `Ok`, bind the inner value to `n` and run the body; if it's `Err`, do nothing (and fall through to the next line).
- `DateTime::parse_from_rfc3339(raw)` — from `chrono`. Parses an RFC3339 timestamp string like `"2026-12-25T10:00:00Z"`. Returns `Result<DateTime<FixedOffset>, ParseError>`.
- `.ok()` — converts `Result<T, E>` to `Option<T>`. `Err` becomes `None`.
- `.map(|dt| dt.timestamp())` — if the `Option` is `Some(dt)`, apply the closure to get the Unix timestamp; if it's `None`, leave it as `None`. The chained `.ok().map(...)` is a very common Rust idiom for "try this, then transform if successful."

Why Is It Written This Way?

- **Builder pattern.** Lets you construct the payload incrementally, only setting the fields you care about. Compare with a constructor that takes 9 positional arguments.
- **PHP parity, with one fix.** The PHP `DeploymentDetails::summery()` (note the typo) meant the `summary` field was never persisted. The Rust port corrects the typo to `summary` so the field actually makes it into the payload.
- **values() strips empty fields.** PHP used `array_filter($details, 'isNotNull')` to drop null entries. The Rust version replicates this exactly so the wire format is identical.
- **Builder methods return `Self`, not `&mut Self`.** This is the **consuming builder** pattern: each method takes ownership and returns the modified value. It prevents two builders from sharing the same struct, which would lead to confusing bugs. The trade-off is that each call allocates, but for a small struct like this, the cost is negligible.

Key Rust Concepts Used Here

Concept	Where	What it does
<code>const NAME: &str = "..."</code>	top	Compile-time constant
<code>pub struct</code>	<code>DeploymentDetails</code>	Public data struct
<code>Option<String></code> / <code>Option<i64></code>	optional fields	"Some value, or nothing"
<code>bool</code>	flag fields	<code>true</code> / <code>false</code>
<code>r#name</code>	<code>r#ref</code>	Raw identifier — escape a reserved word
<code>impl Default for Struct</code>	manual impl	"Default value when <code>::default()</code> is called"
<code>mut self</code>	builder methods	Take ownership of <code>self</code> to modify it
<code>impl Into<String></code>	builder params	Accepts anything convertible to <code>String</code>
<code>impl AsRef<str></code>	schedule params	Accepts anything that can be borrowed as <code>&str</code>
<code>self</code> returned	every builder	Returns the modified struct by value
Closure <code>\ key, val\ </code>	<code>insert</code> helper	Inline function with two arguments
Pattern guard <code>if !s.is_empty()</code>	<code>r#ref</code> match	Refine a pattern with an extra condition
<code>serde_json::Value</code>	<code>values()</code> return	"Any JSON" type
<code>serde_json::Map</code>	inside <code>values()</code>	JSON object type
<code>Value::Object(map)</code>	wrap the map	Construct a JSON object value
<code>chrono::DateTime::parse_from_rfc3339</code>	<code>parse_unix</code>	Parse an RFC3339 timestamp
<code>.parse::<i64>()</code>	<code>parse_unix</code>	Parse a string into an integer
Turbofish <code>::<T></code>	<code>raw.parse::<i64>()</code>	Disambiguate a generic type
<code>if let Ok(n) = ...</code>	<code>parse_unix</code>	Match only the <code>Ok</code> arm
<code>.ok()</code>	<code>parse_unix</code>	<code>Result<T, E> → Option<T></code>
<code>.map(\ dt\ ...)</code>	<code>parse_unix</code>	Transform inside an <code>Option</code>
<code>#[derive(...)]</code>	struct	Auto-generate trait impls

CHAPTER 5: TRAITS

 Source: `src/io/mod.md`

This file defines the **interface** for the binary's output. Just like `http/mod.rs` defines how the program *sends*, this file defines how the program *speaks*. The `Io` trait abstracts over the destination: in production, it writes to stderr and stdout; in tests, it can record messages for later inspection.

The Big Picture

The CLI's contract with the outside world is:

- **stdout**: exactly one JSON line, always.

- **stderr**: human-readable status messages (warnings, success ticks, info).

The **Io** trait models this contract. Every action function takes `&mut dyn Io` and calls methods like `warning`, `success`, and `message` (for stderr) and `write_json` (for stdout).

This file defines:

1. **Io trait** — the interface.
2. **StderrIo re-export** — the production impl, defined in `stderr_io.rs`.
3. **Options re-export** — used by the `set_options` / `options` methods on **Io**.

Walkthrough

Imports and re-exports

```
use crate::error::ApiResponse;
use crate::options::Options;

pub mod stderr_io;
pub use stderr_io::StderrIo;
```

- `pub mod stderr_io;` — declares the submodule containing the production impl.
- `pub use stderr_io::StderrIo;` — re-exports at the `io::` level. Callers can write `use nautpie::io::StderrIo;` instead of `use nautpie::io::stderr_io::StderrIo;`.

The **Io** trait

```
pub trait Io {
    fn warning(&mut self, message: &str);
    fn success(&mut self, message: &str);
    fn message(&mut self, message: &str);
    fn write_json(&mut self, response: &ApiResponse);

    fn set_options(&mut self, _opts: Options) {}
    fn options(&self) -> Option<Options> { None }
}
```

Teaching points:

- **pub trait Io** — defines the interface. Concrete types (`StderrIo` in production, mocks in tests) implement it.
- **&mut self** on `warning`, `success`, `message`, `write_json`, `set_options` — these methods may modify state (e.g., write to a buffer). Mutable borrow required.
- **&self** on `options` — only reads. Immutable borrow.
- **fn warning(&mut self, message: &str)** — print a yellow-on-red warning badge. The default behaviour is impl-specific.
- **fn success(&mut self, message: &str)** — print a black-on-green success badge.
- **fn message(&mut self, message: &str)** — print a neutral info line.
- **fn write_json(&mut self, response: &ApiResponse)** — write the final JSON-line response to stdout. Note: this is the **only** method that goes to stdout, by design.
- **fn set_options(&mut self, _opts: Options)** — inject the parsed CLI options bag. The **default implementation** is `{}` (a no-op). This is one of Rust's most useful features: **default methods on traits**. If an implementor doesn't need options (a test mock, for example), they get a no-op for free.
- **fn options(&self) -> Option<Options> { None }** — retrieve the injected options. The default is `None`. Real implementors (like `StderrIo`) override this.

- `_opts: Options` — the underscore prefix on `_opts` is significant here. In the default implementation, the parameter is **named with a leading underscore** so the compiler doesn't warn about it being unused. This is a convention for "this parameter exists for the trait contract, but the default impl doesn't need it."

Why have a no-op default for `set_options` and `options` ?

Without default implementations, every `Io` implementor would be forced to write:

```
fn set_options(&mut self, _opts: Options) {}
fn options(&self) -> Option<Options> { None }
```

...even if they don't care about options. With the defaults, only the production impl (`StderrIo`) overrides them. Test mocks can ignore options entirely.

This is a key insight about Rust traits: they're not just contracts, they're **partial implementations**. You can write as much shared logic in the trait itself as you like, and let implementors override only what they need to.

`Options` and the side-channel pattern

Notice that `Io` carries an `Options` bag. This is unusual: why does the output sink hold input data?

The answer is **trait signature constraints**. The `Command::run` method takes `&mut dyn Io, &mut dyn HttpClient` — those are the only two parameters it gets. There's no slot for "the original CLI args."

So `main.rs` stuffs the parsed options into the `Io` impl via `set_options`, and the action functions read them back via `io.options()`. It's a **side-channel** that lets the action functions access data that wouldn't otherwise fit through the trait signature.

This pattern shows up in many places in Rust: when a trait is too narrow to carry all the data you need, you attach the data to one of the trait objects. It's a bit of a hack, but it's pragmatic.

Why Is It Written This Way?

- **PHP parity through abstraction.** The PHP original had a `SymfonyStyle` output wrapper with methods like `warning`, `success`, etc. The Rust port models this as a trait, which lets tests substitute mocks.
- **Default methods.** Putting `set_options` and `options` defaults in the trait means simple implementors don't have to write no-op stubs.
- **Side-channel via `Io`.** A pragmatic solution to the trait signature constraint. It works because the options are read once and discarded.
- **One trait, two consumers.** The production code uses `StderrIo` (real output). The tests use mocks that record what was written. Both implement the same trait.

Key Rust Concepts Used Here

Concept	Where	What it does
<code>pub mod name;</code>	top	Declare a submodule
<code>pub use path::Item;</code>	top	Re-export at a shorter path
<code>pub trait Io</code>	trait	Define an interface
<code>&mut self</code>	methods	Mutable borrow — method may modify state
<code>&self</code>	<code>options</code>	Immutable borrow — method only reads
<code>&str</code> parameter	string args	Borrowed string slice
Default method body <code>{}</code>	<code>set_options</code>	"This method does nothing by default"
Default method body <code>None</code>	<code>options</code>	"This method returns <code>None</code> by default"
<code>_opts</code>	parameter	Underscore prefix = unused parameter
Side-channel	<code>set_options</code> / <code>options</code>	Stash extra data through one of the trait objects
<code>Option<Options></code>	<code>options</code> return	"Some options, or no options"

 Source: `src/io/stderr_io.md`

This file implements the production `Io` trait. It writes to `stderr` (for human messages) and `stdout` (for the JSON envelope). Coloured badges make the warnings and successes visually distinct, just like the PHP original.

The Big Picture

Every command invocation produces output on two streams:

- **stdout**: the final JSON-line response (always exactly one line).
- **stderr**: status messages (warnings, success ticks, info, debug logs from the HTTP client).

`StderrIo` is the concrete type that does this writing. It also stores the parsed CLI options so action functions can pull them out later.

Walkthrough

Imports

```
use std::io::{self, Write};

use owo_colors::{OwoColorize, Stream};

use super::Io;
use crate::error::ApiResponse;
use crate::options::Options;
```

- `std::io::{self, Write}` — `io` brings in the `std::io` module (we use `io::stderr()` and `io::stderr().lock()`). `Write` is the trait that provides `.writeln!(...)`. The `self` in `io::{self, Write}` lets us refer to the module as `io` rather than `std::io`.

- `owo_colors::{OwoColorize, Stream}` — `OwoColorize` is the trait that gives us `.if_supports_color(...)`. `Stream::Stderr` is the stream we're targeting (we want colours only on stderr, not stdout).

The struct

```
pub struct StderrIo {
    opts: Option<Options>,
}
```

A tiny struct with one field:

- `opts: Option<Options>` — the parsed CLI options, stashed by `main.rs` via `set_options` and read back by action functions via `options`. Wrapped in `Option` because it might not have been set yet.

Note the field is **not** `pub` — it's private. The trait methods are the only way to access it.

`new()` and `Default`

```
impl StderrIo {
    pub fn new() -> Self {
        Self { opts: None }
    }
}

impl Default for StderrIo {
    fn default() -> Self {
        Self::new()
    }
}
```

A trivial constructor and a `Default` impl that delegates to `new()`. The `Default` impl is auto-implementable in many cases, but here we write it explicitly to delegate to `new()`.

Teaching points:

- `Self { opts: None }` — struct literal with one field set to `None`. Equivalent to `Self { opts: None }`.
- `impl Default for StderrIo` — implements the `Default` trait, so `StderrIo::default()` works.

The `io` impl

```
impl Io for StderrIo {
    fn warning(&mut self, message: &str) {
        let mut stderr = io::stderr().lock();
        let _ = writeln!(
            stderr,
            " {} {} ",
            "WARN".if_supports_color(Stream::Stderr, |t| t.on_yellow().red()),
            message
        );
    }

    fn success(&mut self, message: &str) {
        let mut stderr = io::stderr().lock();
        let _ = writeln!(
            stderr,
            " {} {} ",
            "OK".if_supports_color(Stream::Stderr, |t| t.on_green().black()),
            message
        );
    }

    fn message(&mut self, message: &str) {
        let mut stderr = io::stderr().lock();
        let _ = writeln!(stderr, " {} ", message);
    }

    fn write_json(&mut self, response: &ApiResponse) {
        println!("{}", response.to_line());
    }

    fn set_options(&mut self, opts: Options) {
        self.opts = Some(opts);
    }

    fn options(&self) -> Option<Options> {
        self.opts.clone()
    }
}
```

`warning()` — yellow-on-red badge

```
fn warning(&mut self, message: &str) {
    let mut stderr = io::stderr().lock();
    let _ = writeln!(
        stderr,
        " {} {} ",
        "WARN".if_supports_color(Stream::Stderr, |t| t.on_yellow().red()),
        message
    );
}
```

Teaching points:

- `io::stderr()` — get a handle to the standard error stream.
- `.lock()` — acquire an exclusive lock on the stream. This ensures that if multiple threads wrote to stderr simultaneously, their output wouldn't interleave. In this single-threaded CLI it doesn't matter much, but it's a good habit.
- `writeln!(stderr, "...")` — write a formatted line (with a trailing newline). Returns `Result<(), io::Error>`. We use `writeln!` rather than `println!` because we're writing to a specific handle (`stderr`), not the default stdout.
- `let _ = writeln!(...)` — discard the `Result`. We don't care about write errors (there's no useful recovery for stderr failures).
- `"WARN".if_supports_color(Stream::Stderr, |t| t.on_yellow().red())` — this is where the magic happens.
- `"WARN"` is a `&str`. The `OwoColorize` trait provides `.if_supports_color(...)`.

- `Stream::Stderr` — only apply colours if stderr supports them (i.e., is a TTY). When stderr is piped to a file or another program, no colour codes are emitted.
- `|t| t.on_yellow().red()` — the closure receives the coloured text and applies styling: yellow background, red foreground. The result is the colour-coded ANSI escape sequence string.

The output looks like:

```
WARN  Some warning message
```

With the `WARN` text in red on a yellow background, surrounded by spaces.

`success()` — black-on-green badge

Same pattern as `warning`, but with `on_green().black()`. The output:

```
OK    Some success message
```

`message()` — neutral info line

```
fn message(&mut self, message: &str) {
    let mut stderr = io::stderr().lock();
    let _ = writeln!(stderr, " {} ", message);
}
```

No colour. Just prints the message in a `message` format. Used for neutral informational output.

`write_json()` — the single stdout line

```
fn write_json(&mut self, response: &ApiResponse) {
    println!("{}", response.to_line());
}
```

This is the **only** method that writes to stdout. It uses `println!` (which writes to stdout by default) rather than `writeln!(stdout, ...)`. The `to_line()` method on `ApiResponse` produces a single line of JSON without a trailing newline; `println!` adds the newline.

This method is the keystone of the program's contract: **exactly one JSON line on stdout, always.**

`set_options()` and `options()`

```
fn set_options(&mut self, opts: Options) {
    self.opts = Some(opts);
}

fn options(&self) -> Option<Options> {
    self.opts.clone()
}
```

`set_options` stores the bag. `options` returns a clone (because `Options` derives `Clone` and the trait signature requires `Option<Options>`).

The `.clone()` is necessary because returning a reference would tie the returned value to `self`'s lifetime, which is awkward. Since `Options` is just a thin wrapper around a `HashMap<String, String>`, cloning is cheap.

Why Is It Written This Way?

- **PHP parity.** The original PHP used `<fg=red;bg=yellow>` and `<fg=black;bg=green>` Symfony tags for the warning and success badges. The Rust port uses ANSI escape codes (via `owo-colors`) to produce the same visual effect.
- **TTY-aware colouring.** `if_supports_color(Stream::Stderr, ...)` automatically disables colour when stderr is piped or redirected. This means CI logs don't get cluttered with escape codes.
- **stderr/stdout separation.** Status messages go to stderr; the JSON envelope goes to stdout. CI scripts can pipe `nautpie | jq` and get clean JSON without status messages polluting the stream.
- **Locked stderr.** Even though this CLI is single-threaded, locking the handle is a small habit that prevents bugs if the code is ever extended to use threads.

Key Rust Concepts Used Here

Concept	Where	What it does
<code>pub struct StderrIo { opts: Option<Options> }</code>	struct	Data struct with one private field
<code>Self { opts: None }</code>	<code>new</code>	Struct literal syntax
<code>impl Io for StderrIo</code>	top	Implement the <code>Io</code> trait
<code>io::stderr()</code>	methods	Get the stderr handle
<code>.lock()</code>	methods	Acquire an exclusive lock on the handle
<code>writeln!(handle, "...")</code>	methods	Write a formatted line with trailing newline
<code>let _ = writeln!(...)</code>	methods	Discard the <code>Result</code> from <code>writeln!</code>
<code>println!("{}", ...)</code>	<code>write_json</code>	Write to stdout
<code>owo-colors::OwoColorize</code>	coloured methods	Trait that adds colour methods to strings
<code>.if_supports_color(stream, \ t\ ...)</code>	coloured methods	Apply colour only if the stream supports it
Closure <code>\ t\ t.on_yellow().red()</code>	coloured methods	Inline function — apply colour attributes
<code>Stream::Stderr</code>	coloured methods	Enum variant identifying stderr
<code>impl Default for StderrIo</code>	bottom	Implement the <code>Default</code> trait
<code>Option<Options></code>	<code>opts</code> field	"Some options, or no options"
<code>.clone()</code>	<code>options</code>	Duplicate an <code>Options</code> value
<code>_opts</code> parameter	unused param	Underscore prefix = unused parameter
<code>&mut self</code>	mutating methods	Method may modify state
<code>&self</code>	<code>options</code>	Method only reads

 Source: `src/http/mod.md`

This file defines the **interface** that the action functions use to make HTTP requests. It's tiny — most of the implementation is in `request_client.rs` — but it teaches a critical Rust pattern: **trait-based dependency injection**.

The Big Picture

The action functions in `commands/deploy_naut.rs` and `commands/bitbucket.rs` all take a `&mut dyn HttpClient` parameter. They call methods like `set_endpoint`, `set_content_type`, and `request` on it.

This file defines:

1. **Response** — what an HTTP call returns.
2. **HttpClient trait** — the interface every HTTP client must satisfy.
3. **Re-exports** — `RequestHttpClient` (the production impl) is re-exported from here.

In production, the only implementor is `RequestHttpClient`. In tests, the same trait can be implemented by mock clients.

Walkthrough

Imports

```
use std::collections::HashMap;
use crate::error::Error;
```

- `HashMap<String, String>` — we'll use this for optional HTTP headers.
- `Error` — our crate's error type.

Module and re-exports

```
pub mod request_client;

pub use request_client::RequestHttpClient;
```

- `pub mod request_client;` — declares the submodule. The actual implementation lives in `request_client.rs` next to this file.
- `pub use request_client::RequestHttpClient;` — re-exports the production client at the `http::` level. Callers can write `use nautpie::http::RequestHttpClient;` instead of `use nautpie::http::request_client::RequestHttpClient;`. It's purely a convenience.

The `Response` struct

```
#[derive(Debug)]
pub struct Response {
    pub status: u16,
    pub reason: String,
    pub body: String,
}
```

A simple data struct with three fields:

- `status: u16` — the HTTP status code (200, 404, 500, etc.). `u16` is "unsigned 16-bit integer," range 0–65535.
- `reason: String` — the canonical reason phrase: `"OK"`, `"Not Found"`, `"Internal Server Error"`, etc.
- `body: String` — the response body as text. Note: it's `String`, not `serde_json::Value`. The client doesn't try to parse JSON — that's the caller's job. This keeps the client minimal and lets callers decide how to interpret the body.

`#[derive(Debug)]` lets us print the response for debugging.

The `HttpClient` trait

```
#[allow(dead_code)]
pub trait HttpClient {
    fn set_endpoint(&mut self, endpoint: &str);
    fn set_authorization(&mut self, auth: &str);
    fn set_content_type(&mut self, content_type: &str);
    fn set_username_and_password(&mut self, user: &str, password: &str);

    fn request(
        &mut self,
        method: &str,
        relative_url: &str,
        headers: Option<HashMap<String, String>>,
        body: Option<String>,
    ) -> Result<Response, Error>;
}
```

This is the **interface** every HTTP client must implement. It has five methods.

Teaching points:

- `pub trait HttpClient` — defines a trait. Any struct that wants to act as an HTTP client must implement these methods.
- `&mut self` on every method — the client is **stateful** (it stores the endpoint, content type, auth credentials). Each method mutates that state, so we take a mutable borrow.
- `set_endpoint(&mut self, endpoint: &str)` — store the base URL. Subsequent calls to `request` will resolve relative URLs against this.
- `set_authorization(&mut self, auth: &str)` — set a raw `Authorization` header value. Useful for tokens like `Bearer xyz`.
- `set_content_type(&mut self, content_type: &str)` — set the `Content-Type` header. Defaults to `application/json` but can be overridden for form-encoded requests.
- `set_username_and_password(&mut self, user: &str, password: &str)` — convenience method: builds the basic-auth header value (`Basic base64(user:password)`) and stores it.
- `request(&mut self, method, relative_url, headers, body) -> Result<Response, Error>` — the main entry point. Takes an HTTP method, a path (relative to the endpoint), optional extra headers, and an optional body. Returns a `Result` because requests can fail (network errors, timeouts, etc.).
- `#[allow(dead_code)]` — some of these methods (like `set_authorization`) may not be called by all clients. The trait defines the full interface even if some implementors don't need every method.

Why a trait?

Without the trait, every action function would have to know about `RequestHttpClient` specifically. Tests would need to either:

1. Spin up a real HTTP server (slow, brittle).
2. Refactor every action function to take an internal `Box<dyn ...>`.
3. Or just use the real client against `wiremock` (which is what we do).

With the trait, **dependency injection** is built into the type system. We can substitute any `HttpClient` impl at the call site. The integration tests use this to pass a `wiremock`-backed client instead of a real one.

Trait objects (`dyn HttpClient`)

When you write `&mut dyn HttpClient`, you're saying: "a mutable reference to some heap-allocated value that implements `HttpClient`." Rust resolves method calls at runtime through a **vtable** (a hidden lookup table).

The alternative would be generics:

```
fn request<H: HttpClient>(http: &mut H, ...)
```

Generics resolve at **compile time** (monomorphization) and have no runtime cost, but they require the implementor type to be known at the call site. Trait objects resolve at **runtime** and let us pass anything implementing the trait, at the cost of one indirection per call.

This codebase uses `dyn HttpClient` because the action functions are called from a single dispatch site (`main.rs`), and we want the flexibility to swap implementations at that site without touching every action.

Why Is It Written This Way?

- **PHP parity through abstraction.** The PHP original used Guzzle with a `MockHandler` for tests. The Rust port mimics that with a trait-based mock. The interface is small (5 methods) because that's all the actions need.
- **Mutable client state.** The PHP Guzzle client also had mutable config (base URL, headers, etc.). The Rust port preserves that interface, so the action functions can do `http.set_endpoint(...); http.set_content_type(...); http.request(...)` in sequence without re-passing config.
- **String body, not `serde_json::Value`.** The body is passed in and returned as `String`. Parsing and serializing is the caller's responsibility. This keeps the HTTP client focused on HTTP, not on JSON.
- **`#[allow(dead_code)]`** — the trait lists more methods than strictly needed today (e.g., `set_authorization`). Defining them now means callers can opt into them later without changing the trait.

Key Rust Concepts Used Here

Concept	Where	What it does
<code>pub mod name;</code>	top	Declare a submodule
<code>pub use path::Item;</code>	top	Re-export an item at a shorter path
<code>pub struct Response { ... }</code>	struct	Define a data struct
<code>#[derive(Debug)]</code>	struct	Auto-implement <code>Debug</code> for printing
<code>pub trait HttpClient</code>	trait	Define an interface
<code>&mut self</code>	trait methods	Method takes a mutable borrow
<code>&str</code> parameter	string args	Borrowed string slice
<code>Option<HashMap<...>></code>	<code>request</code> param	Optional extra headers
<code>Option<String></code>	<code>request</code> param	Optional request body
<code>Result<Response, Error></code>	<code>request</code> return	Success-or-failure
<code>dyn HttpClient</code>	callers	Trait object — runtime polymorphism
<code>#[allow(dead_code)]</code>	trait	Suppress unused-code warnings

 Source: `src/commands/mod.md`

This file is the **module root** of the `commands/` folder. It does two things:

1. **Declares the submodules** (`sample` , `deploy_naut` , `bitbucket`) so they can be referenced as `nautpie::commands::sample` , etc.
2. **Defines the `Command` trait** that every concrete command implementation must satisfy.

The trait is the **heart of the dispatch system**. In `main.rs` , you saw this:

```
let cmd = DeployNaut;
cmd.run(&normalise_action(&action_raw), io, http)
```

That `.run()` call works because `DeployNaut` implements the `Command` trait defined here.

Walkthrough

Module declarations

```
pub mod bitbucket;
pub mod deploy_naut;
pub mod sample;
```

Three submodules of `commands/`. Each one corresponds to a Rust file:

- `bitbucket.rs` → `ci:bitbucket` actions
- `deploy_naut.rs` → `deploy:naut` actions
- `sample.rs` → `sampleSuccess` / `sampleFail` smoke-test actions

These declarations are required so the modules can be referenced through `commands::sample`, etc. Without them, even though the files exist on disk, Rust wouldn't know they were part of the crate.

The `Command` trait

```
use crate::error::{ApiResponse, Error};
use crate::http::HttpClient;
use crate::io::Io;

#[allow(dead_code)]
pub trait Command {
    fn name(&self) -> &'static str;

    fn run(
        &self,
        action: &str,
        io: &mut dyn Io,
        http: &mut dyn HttpClient,
    ) -> Result<ApiResponse, Error>;
}
```

Teaching points:

- `pub trait Command` — defines a new trait. A trait is like an **interface** in other languages: it lists methods that any implementing type must provide. Concrete types say "I implement Command" with `impl Command for Foo { ... }`.
- `fn name(&self) -> &'static str` — every command must have a name (e.g., `"deploy:naut"` or `"ci:bitbucket"`). `&'static str` is a string slice with `'static` lifetime, meaning the string lives for the entire duration of the program — usually a string literal.
- `fn run(&self, action: &str, io: &mut dyn Io, http: &mut dyn HttpClient) -> Result<ApiResponse, Error>` — the main entry point. Three things to notice:
 - `&self` — borrowed self. The trait object doesn't own the data.
 - `&mut dyn Io`, `&mut dyn HttpClient` — **trait objects**. These can be any type implementing the trait. The `dyn` keyword is short for "dynamic dispatch" — at runtime, Rust looks up which concrete method to call through a vtable. This is how tests can substitute a fake `Io` or `HttpClient`.
 - `Result<ApiResponse, Error>` — returns either the successful response envelope or an error.

- `#[allow(dead_code)]` — suppresses the warning that `name()` is currently never called. The trait defines it for future use, but the dispatcher in `main.rs` doesn't actually call it.

What the implementors look like

In `deploy_naut.rs` :

```
pub struct DeployNaut;

impl Command for DeployNaut {
    fn name(&self) -> &'static str {
        "deploy:naut"
    }

    fn run(
        &self,
        action: &str,
        io: &mut dyn Io,
        http: &mut dyn HttpClient,
    ) -> Result<ApiResponse, Error> {
        let opts = io
            .options()
            .ok_or_else(|| Error::MissingOption("options".into()))?;
        match action.to_ascii_lowercase().as_str() {
            "samplesuccess" => Ok(sample::sample_success()),
            "samplefail" => sample::sample_fail(),
            "fetch" => do_fetch(&opts, http),
            // ... etc ...
            _ => Err(Error::MissingAction),
        }
    }
}
```

The `DeployNaut` struct is a **unit struct** (no fields). It exists only as a "tag" — a way to attach the `Command` impl to a concrete type. There's only ever one value of it: `DeployNaut`.

`Bitbucket` in `bitbucket.rs` works the same way.

Why a trait?

Without the `Command` trait, the dispatcher in `main.rs` would have to know about every concrete command type and `match` on them:

```
match command_name {
    "deploy:naut" => deploy_naut::DeployNaut.run(action, io, http),
    "ci:bitbucket" => bitbucket::Bitbucket.run(action, io, http),
    // ... etc ...
}
```

This is fine for two commands but gets unwieldy at ten. With the trait, you can store a `&dyn Command` and call `.run()` on it uniformly. The trait also enables generic code: `fn run_any(cmd: &impl Command, ...) { cmd.run(...); }`.

In this codebase the dispatch is small enough that the trait's main value is **discoverability** — it tells you "here are the two methods every command must implement."

Why Is It Written This Way?

- **One trait, many implementations.** This is the textbook use of a Rust trait. It defines a contract, and the concrete types (`DeployNaut` , `Bitbucket`) fulfill it.
- `&mut dyn Io` and `&mut dyn HttpClient`. Trait objects (`dyn Trait`) are how Rust does **runtime polymorphism**. They have a small cost (vtable lookup on every call) but enable dependency injection — perfect for tests.

- `#allow(dead_code)]` on the trait. The `name()` method isn't currently called, but defining it documents the contract and gives us a place to hang metadata later (e.g., help text per subcommand).
- The `_ => Err(Error::MissingAction)` arm. Every `match` in Rust must be **exhaustive**: every possible input value must be covered. The catch-all `_` ensures that an unknown action name produces a clean error rather than a panic.

Key Rust Concepts Used Here

Concept	Where	What it does
<code>pub mod name;</code>	top of file	Declares a submodule wired to <code>commands/name.rs</code>
<code>pub trait Name</code>	<code>Command</code>	Defines a trait — a contract of methods
<code>&self</code>	trait methods	Borrowed self
<code>&mut dyn Trait</code>	<code>run</code> params	Mutable reference to a trait object
<code>Result<T, E></code>	<code>run</code> return	Success-or-failure return type
<code>&'static str</code>	<code>name</code> return	String slice with <code>'static</code> lifetime
<code>impl Trait for Type</code>	implementors	Implement a trait for a concrete type
Unit struct <code>pub struct Name;</code>	<code>DeployNaut</code>	A struct with no fields — just a tag
<code>#[allow(dead_code)]</code>	trait	Suppress unused-code warnings
Catch-all <code>_</code>	<code>match</code> arms	Match any value not covered above

CHAPTER 6: BORROWING AND CLOSURES

 Source: `src/http/request_client.md`

This file is the **production implementation** of the `HttpClient` trait. It uses the `request` crate to actually make HTTP calls. While `http/mod.rs` defines the interface, this file is where the bytes hit the wire.

The Big Picture

`RequestHttpClient` is a **stateful** wrapper around `request::blocking::Client`. It holds:

- The base endpoint URL.
- The current `Authorization` header value.
- The current `Content-Type` value.

Each `request` call resolves the relative URL against the endpoint, applies the configured headers, sends the request, and returns a `Response`.

Walkthrough

Imports

```
use std::collections::HashMap;
use std::time::Duration;

use base64::Engine;
use request::header::{HeaderMap, HeaderName, HeaderValue, AUTHORIZATION, CONTENT_TYPE};
use request::Method;

use super::{HttpClient, Response};
use crate::error::Error;
```

- `Duration` — for the 120-second timeout.
- `base64::Engine` — provides the `.encode(...)` method for basic auth.
- `request::header::*` — typed HTTP header constants and types. `AUTHORIZATION` and `CONTENT_TYPE` are the two header constants we'll set. `HeaderMap` is a typed collection of headers.
- `request::Method` — an enum representing HTTP methods (`GET`, `POST`, etc.). More type-safe than passing strings around.
- `super::{HttpClient, Response}` — imports from the parent module (`http/mod.rs`).

Constants

```
const REQUEST_TIMEOUT: Duration = Duration::from_secs(120);
const DEFAULT_CONTENT_TYPE: &str = "application/json";
```

Two constants:

- `REQUEST_TIMEOUT: Duration` — 120 seconds. Same as the PHP `CURLOPT_TIMEOUT = 120` setting in the original.
- `DEFAULT_CONTENT_TYPE: &str` — JSON by default. Callers can override with `set_content_type` (e.g., for OAuth form-encoded requests).

The struct

```
pub struct RequestHttpClient {
    client: request::blocking::Client,
    endpoint: String,
    authorization: Option<String>,
    content_type: String,
}
```

Four fields:

- `client: request::blocking::Client` — the underlying `request` client. We use the **blocking** client (not `async`) to match the PHP version's synchronous behavior.
- `endpoint: String` — the base URL. Empty until `set_endpoint` is called.
- `authorization: Option<String>` — the `Authorization` header value, if any. `None` means "no auth header." Stored as a full string like `"Basic dXNlcjpwYXNz"` or `"Bearer xyz"`.
- `content_type: String` — the `Content-Type` header value. Defaults to `application/json`.

Teaching points:

- `request::blocking::Client` — the synchronous flavor of `request`. The `async` version is `request::Client` (without `blocking`).
- `Option<String>` for `authorization` because the absence of a value is meaningful: don't send the header at all.

`new()`

```
pub fn new() -> Result<Self, Error> {
    let client = request::blocking::Client::builder()
        .timeout(REQUEST_TIMEOUT)
        .redirect(request::redirect::Policy::limited(10))
        .danger_accept_invalid_certs(true)
        .danger_accept_invalid_hostnames(true)
        .build()
        .map_err(|e| Error::Generic(format!("[HttpClient] build failed: {e}")))?;
    Ok(Self {
        client,
        endpoint: String::new(),
        authorization: None,
        content_type: DEFAULT_CONTENT_TYPE.into(),
    })
}
```

This builds the underlying `request` client with specific configuration:

Teaching points:

- `Client::builder()` — the builder pattern. Each chained method configures one option.
- `.timeout(REQUEST_TIMEOUT)` — 120-second total timeout per request.
- `.redirect(request::redirect::Policy::limited(10))` — follow up to 10 redirects. Matches PHP's `CURLOPT_FOLLOWLOCATION = true`.
- `.danger_accept_invalid_certs(true)` — accept invalid TLS certificates. **This is a security trade-off** — it matches the PHP original's `CURLOPT_SSL_VERIFYPEER = false`, but in production you'd probably want to remove this. The naming `danger_*` is a hint from the `request` maintainers that these methods are risky.
- `.danger_accept_invalid_hostnames(true)` — same trade-off, for hostname verification.
- `.build()` — finalize the builder and return `Result<Client, Error>`.
- `.map_err(|e| Error::Generic(format!("[HttpClient] build failed: {e}")))` — convert request's build error into our error type.
- `DEFAULT_CONTENT_TYPE.into()` — convert `&str` to `String` for the struct field.

The `HttpClient` impl

```
impl HttpClient for RequestHttpClient {
    fn set_endpoint(&mut self, endpoint: &str) {
        self.endpoint = endpoint.to_string();
    }

    fn set_authorization(&mut self, auth: &str) {
        self.authorization = Some(auth.to_string());
    }

    fn set_content_type(&mut self, content_type: &str) {
        self.content_type = content_type.to_string();
    }

    fn set_username_and_password(&mut self, user: &str, password: &str) {
        let raw = format!("{user}:{password}");
        let encoded = base64::engine::general_purpose::STANDARD.encode(raw.as_bytes());
        self.authorization = Some(format!("Basic {encoded}"));
    }
    // ...
}
```

Teaching points:

- `impl HttpClient for RequestHttpClient` — implements the trait. The compiler verifies that every required method is present.
- `endpoint.to_string()` — convert `&str` to `String`. The struct needs an owned `String`.

- `Some(auth.to_string())` — wraps the new value in `Some(...)` since `authorization` is `Option<String>`.
- `base64::engine::general_purpose::STANDARD.encode(raw.as_bytes())` — encode the `user:password` string as base64. The `Engine` trait provides the `.encode(...)` method.

request() — the main entry point

```

fn request(
    &mut self,
    method: &str,
    relative_url: &str,
    headers: Option<HashMap<String, String>>,
    body: Option<String>,
) -> Result<Response, Error> {
    if self.endpoint.is_empty() {
        return Err(Error::MissingEndpoint);
    }

    let endpoint_path = url_path(&self.endpoint);
    let full = format!(
        "{}/{}/{}",
        endpoint_path.trim_end_matches('/'),
        relative_url.trim_start_matches('/')
    );

    let method = Method::from_bytes(method.to_ascii_uppercase().as_bytes())
        .map_err(|e| Error::Generic(format!("[HttpClient] bad method: {e}")))?;

    let mut header_map = HeaderMap::new();
    header_map.insert(
        CONTENT_TYPE,
        HeaderValue::from_str(&self.content_type)
            .map_err(|e| Error::Generic(format!("[HttpClient] bad content-type: {e}")))?,
    );

    if let Some(auth) = &self.authorization {
        if let Ok(value) = HeaderValue::from_str(auth) {
            header_map.insert(AUTHORIZATION, value);
        }
    }

    if let Some(extra) = headers {
        for (k, v) in extra {
            if let (Ok(name), Ok(value)) = (
                HeaderName::from_bytes(k.as_bytes()),
                HeaderValue::from_str(&v),
            ) {
                header_map.insert(name, value);
            }
        }
    }

    let url = format!("{}", base_url(&self.endpoint), full);
    eprintln!("[request] {method} {url}");
    let mut req = self.client.request(method, &url).headers(header_map);

    req = match body {
        Some(b) if self.content_type == "application/x-www-form-urlencoded" => req.body(b),
        Some(b) => req.body(b),
        None => req,
    };

    let response = req
        .send()
        .map_err(|e| Error::Generic(format!("[HttpClient] send failed: {e}")))?;

    let status = response.status().as_u16();
    let reason = response
        .status()
        .canonical_reason()
        .unwrap_or("")
        .to_string();
    let body_text = response
        .text()
        .map_err(|e| Error::Generic(format!("[HttpClient] read body failed: {e}")))?;

    if (400..=599).contains(&status) {
        return Err(Error::HttpStatus {

```

```

        status,
        reason,
        body: body_text,
    });
}

Ok(Response { status, reason, body: body_text })
}

```

This is the meatiest method. Let's break it down section by section.

1. Endpoint guard

```

if self.endpoint.is_empty() {
    return Err(Error::MissingEndpoint);
}

```

The first thing we do is **fail fast** if `set_endpoint` was never called. Without an endpoint, we don't know where to send requests.

2. URL resolution

```

let endpoint_path = url_path(&self.endpoint);
let full = format!(
    "{}/{}",
    endpoint_path.trim_end_matches('/'),
    relative_url.trim_start_matches('/')
);

```

`url_path` extracts just the path component of the endpoint (e.g., `/naut`). We append the relative URL to it. This mirrors PHP's behavior: `parse_url($endpoint, PHP_URL_PATH) . '/' . $relative`.

3. Method parsing

```

let method = Method::from_bytes(method.to_ascii_uppercase().as_bytes())
    .map_err(|e| Error::Generic(format!("[HttpClient] bad method: {e}")))?;

```

`Method::from_bytes` is a typed constructor: it parses bytes like `b"GET"` into a `Method::GET`. The conversion can fail (e.g., for invalid bytes), so it returns a `Result`. We convert that into our `Error::Generic`.

The `.to_ascii_uppercase()` allows callers to pass `"get"` or `"GET"` interchangeably.

4. Header construction

```

let mut header_map = HeaderMap::new();
header_map.insert(
    CONTENT_TYPE,
    HeaderValue::from_str(&self.content_type)
        .map_err(|e| Error::Generic(format!("[HttpClient] bad content-type: {e}")))?,
);

```

We always insert a `Content-Type` header. The value must be valid HTTP (no newlines, no non-ASCII), so we use `HeaderValue::from_str` which returns a `Result`.

```

if let Some(auth) = &self.authorization {
    if let Ok(value) = HeaderValue::from_str(auth) {
        header_map.insert(AUTHORIZATION, value);
    }
}

```

The `Authorization` header is only inserted if `authorization` is `Some(...)`. We silently skip it if the value is invalid HTTP (rather than failing).

```
if let Some(extra) = headers {
    for (k, v) in extra {
        if let (Ok(name), Ok(value)) = (
            HeaderName::from_bytes(k.as_bytes()),
            HeaderValue::from_str(&v),
        ) {
            header_map.insert(name, value);
        }
    }
}
```

Any caller-supplied extra headers are added too. Invalid header names or values are silently skipped.

5. Final URL

```
let url = format!("{}", base_url(&self.endpoint), full);
```

Combines the base URL (`https://example.com`) with the resolved path (`/naut/projects/...`).

`eprintln!("[request] {method} {url}");` is a debug print to stderr. It helps trace what's being sent.

6. Request body

```
req = match body {
    Some(b) if self.content_type == "application/x-www-form-urlencoded" => req.body(b),
    Some(b) => req.body(b),
    None => req,
};
```

Both `Some(b)` arms do the same thing (set the body). The split is for **future-proofing** — if we ever need different handling for form-encoded vs other content types, the structure is already there.

This is a common Rust pattern: use a `match` with two identical arms but a different guard, leaving the implementation flexible. You can change one arm without changing the other.

7. Sending

```
let response = req
    .send()
    .map_err(|e| Error::Generic(format!("[HttpClient] send failed: {e}"))?);
```

Actually sends the request. The error mapping converts any request error into our `Error::Generic`.

8. Reading the response

```
let status = response.status().as_u16();
let reason = response
    .status()
    .canonical_reason()
    .unwrap_or("")
    .to_string();
let body_text = response
    .text()
    .map_err(|e| Error::Generic(format!("[HttpClient] read body failed: {e}"))?);
```

- `response.status()` returns a `StatusCode`.
- `.as_u16()` converts to a plain `u16`.

- `.canonical_reason()` returns the standard reason phrase (e.g., `"OK"` for 200, `"Not Found"` for 404). `unwrap_or("")` gives an empty string if the status is non-standard.
- `.text()` reads the body as a `String`. Could fail if the body isn't valid UTF-8, but for our use case (JSON APIs), it always is.

9. Error handling for HTTP 4xx/5xx

```
if (400..=599).contains(&status) {
    return Err(Error::HttpStatus {
        status,
        reason,
        body: body_text,
    });
}
```

If the server returned a client or server error, we surface it as `Error::HttpStatus`. This matches PHP's behavior of throwing on non-2xx responses.

The `(400..=599).contains(&status)` syntax is an **inclusive range**: $400 \leq \text{status} \leq 599$.

10. Success return

```
Ok(Response { status, reason, body: body_text })
```

Wrap everything in our `Response` struct.

Default impl

```
impl Default for RequestHttpClient {
    fn default() -> Self {
        Self::new().expect("RequestHttpClient::new should succeed with default settings")
    }
}
```

A default constructor that **panics** if the underlying client can't be built. Useful for tests where you don't want to handle the `Result`.

Helper functions

url_path()

```
fn url_path(endpoint: &str) -> String {
    match url::Url::parse(endpoint) {
        Ok(u) => u.path().to_string(),
        Err(_) => endpoint.to_string(),
    }
}
```

Extracts the path component of a URL. If the endpoint is not a valid URL (e.g., a relative path), returns it as-is.

base_url()

```
fn base_url(endpoint: &str) -> String {
    match url::Url::parse(endpoint) {
        Ok(u) => {
            let host = u.host_str().unwrap_or("");
            let scheme = u.scheme();
            let port = u.port().map(|p| format!("{p}")).unwrap_or_default();
            format!("{scheme}://{host}{port}")
        }
        Err(_) => String::new(),
    }
}
```

Extracts `scheme://host[:port]` (e.g., `https://api.bitbucket.org`). If parsing fails, returns an empty string (which would produce invalid URLs, but tests are expected to use real URLs).

Tests

```
#[test]
fn url_path_extract_path() {
    assert_eq!(url_path("https://example.com/naut"), "/naut");
    assert_eq!(url_path("https://example.com/naut/"), "/naut/");
}

#[test]
fn base_url_extract_origin() {
    assert_eq!(base_url("https://example.com/naut"), "https://example.com");
    assert_eq!(base_url("https://api.bitbucket.org/2.0/"), "https://api.bitbucket.org");
}
```

Tests for the URL helper functions. Note that the tests don't make actual HTTP calls — they only verify the URL parsing.

Why Is It Written This Way?

- **PHP parity.** Each setting (`danger_accept_invalid_certs` , `danger_accept_invalid_hostnames` , redirects, timeout) mirrors the PHP `CURLOPT_*` constants. The behavior on the wire is identical.
- **Blocking, not async.** The PHP version was synchronous. The Rust port uses `request::blocking` for the same blocking semantics. Async would force the entire dispatcher to be async, which would complicate `main.rs`.
- **Header building is explicit.** We could use `request`'s builder API to set headers more declaratively, but doing it manually gives us full control over validation and skipping invalid headers.
- **The `eprintln!` debug log.** Useful during development, but should be removed (or gated) before this hits production.

Key Rust Concepts Used Here

Concept	Where	What it does
<code>request::blocking::Client</code>	struct field	Synchronous HTTP client
<code>Client::builder()</code>	<code>new</code>	Builder pattern for configuration
<code>.timeout(Duration)</code>	<code>new</code>	Configure request timeout
<code>.redirect(Policy::limited(n))</code>	<code>new</code>	Configure redirect-following
<code>Result<Self, Error></code>	<code>new</code> return	Success-or-failure constructor
<code>impl HttpClient for RequestHttpClient</code>	top	Implement the trait
<code>Method::from_bytes(...)</code>	<code>request</code>	Parse an HTTP method
<code>HeaderMap::new()</code>	<code>request</code>	Create an empty header map
<code>HeaderValue::from_str(...)</code>	<code>request</code>	Parse a header value (returns <code>Result</code>)
<code>HeaderName::from_bytes(...)</code>	<code>request</code>	Parse a header name (returns <code>Result</code>)
<code>header_map.insert(name, value)</code>	<code>request</code>	Insert a header
<code>req.body(b)</code>	<code>request</code>	Set the request body (returns updated builder)
<code>req.send()</code>	<code>request</code>	Actually send the HTTP request
<code>.status().as_u16()</code>	<code>request</code>	Extract HTTP status as a <code>u16</code>
<code>.canonical_reason()</code>	<code>request</code>	Get the standard reason phrase
<code>.text()</code>	<code>request</code>	Read response body as a <code>String</code>
<code>(400..=599).contains(&status)</code>	<code>request</code>	Inclusive range check
<code>base64::Engine::encode</code>	<code>set_username_and_password</code>	Base64-encode credentials
<code>url::Url::parse(...)</code>	helpers	Parse a URL string
<code>Duration::from_secs(n)</code>	<code>new</code>	Create a duration from seconds
<code>.expect(msg)</code>	<code>default</code>	Panic with message on <code>Err</code>
<code>eprintln!(...)</code>	<code>request</code>	Print to stderr (debug)
<code>match ... { ... }</code>	<code>request</code> body	Match on a value (with optional guards)
<code>if let Some(v) = ...</code>	<code>request</code>	Pattern-match only the <code>Some</code> arm
<code>unwrap_or("")</code>	<code>request</code>	Default value on <code>None</code>
<code>unwrap_or_default()</code>	<code>request</code>	Default to the type's <code>Default::default()</code>
<code>impl Default for Struct</code>	bottom	Implement the <code>Default</code> trait

 Source: `src/tui/spinner.md`

This is the **most algorithmically complex** file in the codebase. It implements a polling loop that:

1. Repeatedly calls a callback (which usually hits an HTTP endpoint).
2. Renders the current state to the terminal.
3. Stops when the callback signals completion, the callback returns an error, or a deadline expires.

It has two implementations: one for **TTY** environments (interactive terminals) using `ratatui`, and one for **non-TTY** (CI logs) using plain stderr lines.

The Big Picture

Two actions in `deploy_naut.rs` use this:

- `gitFetch` — polls `/project/{stack}/git/fetches/{id}` every 5 seconds until the git fetch completes.
- `checkDeploymentProgress` — polls `/project/{stack}/environment/{env}/deploys/{id}` until the deployment reaches the `Completed` state.

Both pass a deadline (2 minutes for git, 30 minutes for deploy), a label, and a closure that performs the polling HTTP request.

The `ProgressRunner` is the workhorse. It:

1. Checks if stderr is a TTY.
2. If yes: enters `run_tui` (ratatui-based).
3. If no: enters `run_fallback` (stderr-line-based).
4. Either way, polls until done / errored / timed out.

Walkthrough

Imports

```
use std::io::{self, IsTerminal, Write};
use std::time::{Duration, Instant};

use ratatui::backend::CrosstermBackend;
use ratatui::layout::{Constraint, Direction, Layout};
use ratatui::style::{Color, Modifier, Style};
use ratatui::text::{Line, Span};
use ratatui::widgets::{Block, Borders, Paragraph};
use ratatui::Terminal;

use crate::error::Error;
use crate::http::HttpClient;
```

- `std::io::{self, IsTerminal, Write}` — `io` (for `stderr()`), `IsTerminal` (a trait with `.is_terminal()`), `Write` (for `.writeln!`).
- `std::time::{Duration, Instant}` — `Duration` for time spans, `Instant` for measuring elapsed time.
- `ratatui::*` — the various pieces needed for TUI rendering.
- `crate::error::Error` and `crate::http::HttpClient` — our own types.

The `PollOutcome` enum

```
pub enum PollOutcome {
    Pending(String),
    Done,
}
```

The result of one polling iteration:

- `Pending(String)` — not done yet. The `String` is the current status (e.g., `"Queued"`, `"Deploying"`) for display.
- `Done` — operation completed.

This is a **simple enum** (no fields on `Done`, just a tuple variant on `Pending`).

The `PollFn` type alias

```
pub type PollFn = Box<dyn FnMut(&mut dyn HttpClient) -> Result<PollOutcome, Error>>;
```

A type alias for "a boxed, mutable closure that takes an HTTP client and returns a `PollOutcome` or error."

Teaching points:

- `type Alias = ...;` — defines a new name for an existing type. `PollFn` and `Box<dyn FnMut(...)>` are interchangeable.
- `Box<dyn Trait>` — a heap-allocated trait object. `dyn FnMut(...)` is a closure that mutates captured state.
- `FnMut` — closures come in three flavors:
- `Fn` — borrows captured state immutably.
- `FnMut` — borrows captured state mutably.
- `FnOnce` — consumes captured state.

Polling closures need to be `FnMut` because they're called multiple times. The HTTP client is mutated on each call.

The `ProgressRunner` struct

```
pub struct ProgressRunner {
    pub title: &'static str,
    pub label: String,
    pub poll_interval: Duration,
    pub tick_rate: Duration,
}
```

Configuration for one polling run.

Teaching points:

- `pub title: &'static str` — the title shown in the TUI box (e.g., `"gitFetch"`). `&'static str` is required because `ProgressRunner::new` stores string literals.
- `pub label: String` — a longer description, e.g., `"Git fetch for example-stack (#42)"`. Owned because it's computed dynamically.
- `pub poll_interval: Duration` — how often to call the poll function. Defaults to 5 seconds (PHP parity).
- `pub tick_rate: Duration` — how often to redraw the TUI. Defaults to 250 milliseconds (4 frames per second).
- All fields are `pub`. This is unusual — normally Rust prefers private fields with accessor methods. But here, the fields are intended to be tweaked after construction. For example, the tests access `runner.title` directly.

new()

```
pub fn new(title: &'static str, label: impl Into<String>) -> Self {
    Self {
        title,
        label: label.into(),
        poll_interval: Duration::from_secs(5),
        tick_rate: Duration::from_millis(250),
    }
}
```

Teaching points:

- **label: impl Into<String>** — accepts anything convertible to **String** (e.g., **&str**, **String**, **format!(...)**).
- **.into()** — performs the conversion.
- **Defaults** match PHP behavior (5-second polling) and a smooth UI (4 frames per second).

run() — the dispatcher

```
pub fn run(self, deadline: Duration, poll_fn: PollFn, http: &mut dyn HttpClient) -> Result<(), Error> {
    if io::stderr().is_terminal() {
        self.run_tui(deadline, poll_fn, http)
    } else {
        self.run_fallback(deadline, poll_fn, http)
    }
}
```

Picks the implementation based on whether stderr is a TTY.

Teaching points:

- **self** (consuming) — by taking **self** by value (not **&self**), the method **consumes** the runner. After **run()** is called, the runner is gone. This prevents reusing the same runner with different configs (which would be a likely source of bugs).
- **io::stderr().is_terminal()** — the **IsTerminal** trait, stable since Rust 1.70. Returns **true** if stderr is connected to a terminal (not piped to a file or another program).
- **Branch to either `run_tui` or `run_fallback`**. The two functions handle the same polling logic but render output differently.

run_tui() — the ratatui-based loop

```

fn run_tui(self, deadline: Duration, mut poll_fn: PollFn, http: &mut dyn HttpClient) -> Result<(), Error> {
    let backend = CrosstermBackend::new(io::stderr());
    let mut terminal = match Terminal::new(backend) {
        Ok(t) => t,
        Err(_) => return self.run_fallback(deadline, poll_fn, http),
    };
    let _ = ratatui::crossterm::terminal::enable_raw_mode();
    let _ = terminal.clear();

    let start = Instant::now();
    let mut last_status = String::new();
    let mut next_poll = Instant::now();
    let mut next_tick = Instant::now();

    loop {
        let elapsed = start.elapsed();
        if elapsed >= deadline {
            let _ = ratatui::crossterm::terminal::disable_raw_mode();
            return Err(Error::Timeout(format!(
                "{} timed out after {}s",
                self.label,
                elapsed.as_secs()
            )));
        }

        if Instant::now() >= next_poll {
            match poll_fn(http) {
                Ok(PollOutcome::Pending(status)) => { last_status = status; }
                Ok(PollOutcome::Done) => {
                    let _ = ratatui::crossterm::terminal::disable_raw_mode();
                    return Ok(());
                }
                Err(e) => {
                    let _ = ratatui::crossterm::terminal::disable_raw_mode();
                    return Err(e);
                }
            }
            next_poll = Instant::now() + self.poll_interval;
        }

        if Instant::now() >= next_tick {
            let _ = terminal.draw(|f| {
                let chunks = Layout::default()
                    .direction(Direction::Vertical)
                    .margin(1)
                    .constraints([Constraint::Length(3), Constraint::Length(3)].as_ref())
                    .split(f.area());

                let title = Paragraph::new(Line::from(vec![Span::styled(
                    self.title,
                    Style::default().fg(Color::Cyan).add_modifier(Modifier::BOLD),
                )]))
                    .block(Block::default().borders(Borders::ALL).title(" nautpie "));
                f.render_widget(title, chunks[0]);

                let body = Paragraph::new(vec![
                    Line::from(format!("Label : {}", self.label)),
                    Line::from(format!("Elapsed : {}s", elapsed.as_secs())),
                    Line::from(format!("Status : {}", last_status)),
                ])
                    .block(Block::default().borders(Borders::ALL));
                f.render_widget(body, chunks[1]);
            });
            next_tick = Instant::now() + self.tick_rate;
        }

        std::thread::sleep(Duration::from_millis(50));
    }
}

```

```
}
}
```

This is the meatiest function. Let's break it down.

Terminal bring-up

```
let backend = CrosstermBackend::new(io::stderr());
let mut terminal = match Terminal::new(backend) {
    Ok(t) => t,
    Err(_) => return self.run_fallback(deadline, poll_fn, http),
};
```

Teaching points:

- `CrosstermBackend::new(io::stderr())` — ratatui uses a "backend" abstraction for terminal I/O. `CrosstermBackend` wraps a `Write` implementor (here, `stderr`) and turns it into a TTY-renderable surface.
- `Terminal::new(backend)` — wrap the backend in a `Terminal` that can be drawn to. Returns `Result`.
- The fallback `Err(_) => return self.run_fallback(...)` — if terminal initialization fails (e.g., `TERM` env var missing on CI), gracefully fall back to the non-TUI mode.

Raw mode

```
let _ = ratatui::crossterm::terminal::enable_raw_mode();
```

Teaching points:

- **Raw mode** disables line buffering and echo so the TUI can take over the entire screen. It must be **disabled** before returning, otherwise the terminal stays in a broken state.
- `let _ = ...` — we ignore the result. Raw mode might fail in some environments; if it does, we just continue without it.

Time state

```
let start = Instant::now();
let mut last_status = String::new();
let mut next_poll = Instant::now();
let mut next_tick = Instant::now();
```

Four pieces of mutable state:

- `start` — when the loop began. Used to compute elapsed time.
- `last_status` — the most recent status string from the polling callback. Displayed in the TUI.
- `next_poll` — when the next poll should run. We check `Instant::now() >= next_poll` to decide.
- `next_tick` — when the next TUI redraw should happen. Same pattern.

This is a classic **fixed-timestep loop** pattern from game development: separate the "polling rate" (5s) from the "render rate" (250ms).

The main loop

```
loop {
    let elapsed = start.elapsed();
    if elapsed >= deadline {
        // timeout handling
    }

    if Instant::now() >= next_poll {
        // poll
    }

    if Instant::now() >= next_tick {
        // draw
    }

    std::thread::sleep(Duration::from_millis(50));
}
```

Three phases per iteration:

1. **Check deadline.** If we've exceeded it, disable raw mode and return `Timeout`.
2. **Poll if it's time.** Run the callback; update `last_status` or return early on `Done` / `Err`.
3. **Draw if it's time.** Re-render the TUI frame.
4. **Sleep 50ms.** Prevents the loop from spinning at 100% CPU.

Polling

```
if Instant::now() >= next_poll {
    match poll_fn(http) {
        Ok(PollOutcome::Pending(status)) => { last_status = status; }
        Ok(PollOutcome::Done) => {
            let _ = ratatui::crossterm::terminal::disable_raw_mode();
            return Ok(());
        }
        Err(e) => {
            let _ = ratatui::crossterm::terminal::disable_raw_mode();
            return Err(e);
        }
    }
    next_poll = Instant::now() + self.poll_interval;
}
```

Teaching points:

- `poll_fn(http)` — invokes the closure passed in by the caller. It mutably borrows `http`, makes a request, and returns a `Result<PollOutcome, Error>`.
- **The three-arm match** handles all three possible outcomes.
- **On every exit, we disable raw mode.** This is critical — leaving the terminal in raw mode would corrupt subsequent output.
- `next_poll = Instant::now() + self.poll_interval` — schedule the next poll 5 seconds from now.

Drawing

```
if Instant::now() >= next_tick {
    let _ = terminal.draw(|f| {
        let chunks = Layout::default()
            .direction(Direction::Vertical)
            .margin(1)
            .constraints([Constraint::Length(3), Constraint::Length(3)].as_ref())
            .split(f.area());

        let title = Paragraph::new(Line::from(vec![Span::styled(
            self.title,
            Style::default().fg(Color::Cyan).add_modifier(Modifier::BOLD),
        )]))
        .block(Block::default().borders(Borders::ALL).title(" nautpie "));
        f.render_widget(title, chunks[0]);

        let body = Paragraph::new(vec![
            Line::from(format!("Label : {}", self.label)),
            Line::from(format!("Elapsed : {}s", elapsed.as_secs())),
            Line::from(format!("Status : {}", last_status)),
        ])
        .block(Block::default().borders(Borders::ALL));
        f.render_widget(body, chunks[1]);
    });
    next_tick = Instant::now() + self.tick_rate;
}
```

This is **ratatui boilerplate**. The pattern is:

1. Split the screen into chunks (here: two vertical sections of 3 lines each).
2. Create a `Paragraph` widget with the text to display.
3. Wrap it in a `Block` with a border.
4. Render the widget into the chunk.

Teaching points:

- `terminal.draw(|f| { ... })` — the closure receives a `Frame` (`f`), which represents the current screen. We draw onto it.
- `f.area()` — the full area of the frame. We split it.
- `Constraint::Length(3)` — each chunk is 3 lines tall.
- `[Constraint::Length(3), Constraint::Length(3)].as_ref()` — the constraints array. `.as_ref()` converts `&[Constraint; 2]` to `&[Constraint]`.
- `Span::styled(text, style)` — a styled piece of text. `Style::default().fg(Color::Cyan).add_modifier(Modifier::BOLD)` produces a bold cyan text style.
- `Paragraph::new(line).block(border).render(area)` — the standard pattern for a framed text widget.

The sleep

```
std::thread::sleep(Duration::from_millis(50));
```

A 50ms sleep throttles the loop. Without it, the loop would spin as fast as the CPU allows, wasting energy and heat.

`run_fallback()` — the non-TUI loop

```
fn run_fallback(self, deadline: Duration, mut poll_fn: PollFn, http: &mut dyn HttpClient) -> Result<(), Error> {
    let stderr = io::stderr();
    let mut handle = stderr.lock();
    let _ = writeln!(handle, "[nautpie] {} ...", self.label);

    let start = Instant::now();
    let mut next_poll = Instant::now();
    loop {
        let elapsed = start.elapsed();
        if elapsed >= deadline {
            // timeout
        }

        if Instant::now() >= next_poll {
            let _ = writeln!(handle, "[nautpie] Waiting... ({}s)", elapsed.as_secs());
            match poll_fn(http) {
                Ok(PollOutcome::Pending(status)) => { let _ = writeln!(handle, "[nautpie] Status: {status}"); }
                Ok(PollOutcome::Done) => return Ok(()),
                Err(e) => return Err(e),
            }
            next_poll = Instant::now() + self.poll_interval;
        }

        std::thread::sleep(Duration::from_millis(200));
    }
}
```

Same logic as `run_tui`, but:

- **No terminal setup or raw mode** — just lock `stderr` once and use it.
- **No TUI rendering** — just `writeln!` to print lines like `[nautpie] Waiting... (30s)` and `[nautpie] Status: Queued`.
- **200ms sleep** instead of 50ms — less responsive but cheaper on CI.

The output looks like:

```
[nautpie] Git fetch for example-stack (#42) ...
[nautpie] Waiting... (0s)
[nautpie] Waiting... (5s)
[nautpie] Status: In Progress
[nautpie] Waiting... (10s)
[nautpie] Status: Complete
```

This matches PHP's behavior: a line every 5 seconds with the current state.

`run_with_progress()` — convenience wrapper

```
pub fn run_with_progress<F>(
    title: &'static str,
    label: impl Into<String>,
    deadline: Duration,
    http: &mut dyn HttpClient,
    poll_fn: F,
) -> Result<(), Error>
where
    F: FnMut(&mut dyn HttpClient) -> Result<PollOutcome, Error> + 'static,
{
    let runner = ProgressRunner::new(title, label);
    let boxed: PollFn = Box::new(poll_fn);
    runner.run(deadline, boxed, http)
}
```

A free function that wraps the boilerplate:

1. Build a `ProgressRunner` with the given title and label.
2. Box the closure into a `PollFn`.
3. Run it.

Teaching points:

- The `where F: FnMut(...) + 'static` — generic over any closure-like type. The `'static` bound means the closure can't borrow non-`'static` data (it must own everything it captures).
- `Box::new(poll_fn)` — heap-allocate the closure. The `Box<dyn FnMut(...)>` is what `PollFn` is.

In `deploy_naut.rs`, callers write:

```
run_with_progress(
    "gitFetch",
    format!("Git fetch for {stack} ({#fetch_id})"),
    Duration::from_secs(GIT_TIMEOUT_SECS),
    http,
    move |http| poll_status(http, &poll_url_for_closure, "status"),
)?;
```

The `move` keyword is required because the closure outlives the calling function.

Tests

The tests verify:

1. **Default constructor values** are correct.
2. **Timeout works** — a never-Done poll produces `Error::Timeout`.
3. **Immediate Done works** — a poll that returns `Done` succeeds immediately.

Each test creates a `NoopClient` (a fake `HttpClient` impl) so no real HTTP calls happen.

Why Is It Written This Way?

- **TTY-aware UX.** On a developer's machine, you get a smooth live-updating TUI. On CI, you get simple log lines. The same code path picks the right one based on `is_terminal()`.
- **Fixed-timestep loop.** Decoupling poll rate (5s) from render rate (250ms) means we don't make HTTP calls faster than necessary while still keeping the UI smooth.
- **Graceful degradation.** If `Terminal::new` fails (e.g., on a broken CI runner), we fall back to the stderr-line variant. The user gets progress either way.
- **Two return paths share cleanup.** Both `run_tui` and the early-exit cases in the polling match disable raw mode before returning. This is critical for not corrupting the terminal.
- **PHP parity.** The 5-second polling interval matches PHP's `sleep(5)` loop.

Key Rust Concepts Used Here

Concept	Where	What it does
<code>enum PollOutcome</code>	type	Two-variant result enum
<code>type Alias = ...;</code>	<code>PollFn</code>	Type alias for readability
<code>Box<dyn FnMut(...)></code>	<code>PollFn</code>	Heap-allocated mutable closure
<code>FnMut</code>	<code>PollFn</code>	Closure flavor that mutates captured state
<code>+ 'static</code>	<code>run_with_progress</code> bound	Closure can't borrow non- <code>'static</code> data
<code>pub struct</code>	<code>ProgressRunner</code>	Data struct with public fields
<code>&'static str</code>	<code>title</code> field	String slice with <code>'static</code> lifetime
<code>Duration::from_secs(n)</code>	defaults	Create a duration from seconds
<code>Duration::from_millis(n)</code>	sleep	Create a duration from milliseconds
<code>Instant::now()</code>	timing	Current monotonic time
<code>Instant::now() + Duration</code>	scheduling	Future instant
<code>start.elapsed()</code>	timing	Time elapsed since <code>start</code>
<code>std::thread::sleep(...)</code>	loop	Pause for a duration
<code>io::stderr().is_terminal()</code>	dispatcher	"Is stderr a TTY?"
<code>CrosstermBackend::new(...)</code>	<code>run_tui</code>	ratatui backend over stderr
<code>Terminal::new(backend)</code>	<code>run_tui</code>	ratatui terminal wrapper
<code>terminal.draw(f ...)</code>	TUI render	Render a frame
<code>Layout::default().direction(...)</code>	TUI	Split the screen
<code>Constraint::Length(n)</code>	TUI	Fixed-size constraint
<code>f.area()</code>	TUI	The full screen area
<code>Paragraph::new(...)</code>	TUI	Text widget
<code>Block::default().borders(Borders::ALL)</code>	TUI	Bordered box
<code>Style::default().fg(Color::Cyan)</code>	TUI	Text style
<code>Span::styled(...)</code>	TUI	A styled piece of text
<code>Line::from(...)</code>	TUI	A line of text
<code>enable_raw_mode()</code>	<code>run_tui</code>	Disable line buffering/echo
<code>disable_raw_mode()</code>	exits	Restore terminal state
<code>move</code> closure	callers	Move captured vars into the closure
<code>impl Into<String></code>	<code>label</code> param	Accept anything convertible to <code>String</code>

Concept	Where	What it does
Generic <code><F: FnMut(...) + 'static></code>	<code>run_with_progress</code>	Generic over closure type
<code>Result<(), Error></code>	<code>return</code>	Success-or-failure
<code>Error::Timeout(...)</code>	<code>timeout</code>	Construct the timeout error variant
<code>Box::new(poll_fn)</code>	<code>run_with_progress</code>	Heap-allocate the closure
<code>impl Trait</code> for params	<code>run_with_progress</code>	"Accepts anything implementing this"
<code>where F: Trait</code>	<code>run_with_progress</code>	Trait bound on a generic

CHAPTER 7: COMMAND-LINE PARSING

 Source: `src/cli.md`

This file uses the `clap` crate to define the CLI surface. When you type `nautpie --help`, the help text you see comes from the structures defined here. When you type `nautpie deploy:naut createDeployment --stack=foo`, the parsing of that string is done by the code here.

The single most important idea in this file is that **`clap` generates the parser at compile time** from the structs and attributes you write. There's no separate "schema" file and no runtime parsing — you write Rust structs and `clap` figures out the rest.

The Big Picture

This file defines:

1. `normalise_action()` — a helper that converts any case style (`sampleSuccess`, `Sample-Success`, `SAMPLE_SUCCESS`) into the canonical `lowerCamelCase`.
2. `Cli` — the top-level struct that says "this program takes a subcommand."
3. `CommandKind` — an enum with two variants: `DeployNaut` and `Bitbucket`. Each variant holds its own argument struct.
4. `DeployNautArgs` — every flag the `deploy:naut` subcommand accepts.
5. `BitbucketArgs` — every flag the `ci:bitbucket` subcommand accepts.

When `Cli::parse()` runs (in `main.rs`), `clap` walks the arguments and produces a fully-populated `Cli` value.

Walkthrough

normalise_action()

```
pub fn normalise_action(raw: &str) -> String {
    let mut words: Vec<String> = Vec::new();
    let mut current = String::new();
    let mut prev_is_lower = false;
    for c in raw.chars() {
        if c == '_' || c == '-' {
            if !current.is_empty() {
                words.push(std::mem::take(&mut current));
            }
            prev_is_lower = false;
            continue;
        }
        let upper = c.is_ascii_uppercase();
        if upper && prev_is_lower {
            words.push(std::mem::take(&mut current));
        }
        current.push(c);
        prev_is_lower = c.is_ascii_lowercase();
    }
    if !current.is_empty() {
        words.push(current);
    }
    // ... join logic ...
}
```

This is a small string-processing function, but it teaches several Rust concepts:

- **&str** — a borrowed string slice. We take a reference (**&**) so we don't have to take ownership of the caller's string.
- **String::new()** — creates an empty, growable string on the heap. The opposite would be **""** (a string literal, which is immutable and stored in the program's read-only data segment).
- **Vec<String>** — a growable list of owned strings. The **Vec!** macro (with the **!**) creates an empty vec.
- **std::mem::take(&mut current)** — a handy trick. It replaces **current** with the **default value** (here, an empty **String**) and returns the old value. So we can move the accumulated characters out of **current** into the **words** vector without cloning them. This is more efficient than **current.clone()**.
- **c.is_ascii_uppercase()** — a method on **char** (Rust's 32-bit Unicode scalar type). It returns **true** if the character is in the ASCII range A–Z.
- **continue** — skips the rest of the loop body and goes to the next iteration.

The algorithm walks the input character by character. When it sees an underscore or hyphen, that's a word boundary. When it sees an uppercase letter preceded by a lowercase letter (**aB** → split between **a** and **B**), that's another word boundary. Everything else accumulates into **current**.

After collecting the words, it joins them: the first word is lowercased (**sample**), and each subsequent word is capitalised (**Success**), giving **sampleSuccess**.

cli and the Parser derive

```
#[derive(Debug, Parser)]
#[command(
    name = "nautpie",
    version,
    about = "DeployNaut / Bitbucket Pipelines deployment client"
)]
pub struct Cli {
    #[command(subcommand)]
    pub command: CommandKind,
}
```

- `#[derive(Debug, Parser)]` — These are **derive macros**. `Debug` makes the struct printable for debugging. `Parser` is `clap`'s magic — it generates a `Cli::parse()` function and a `Cli::try_parse_from(...)` function for you.
- `#[command(...)]` — attributes that configure `clap`. `name`, `version`, and `about` populate the `--help` text.
- `#[command(subcommand)]` — tells `clap` that this field is a subcommand, so the user must type one of the variants of `CommandKind`.

`CommandKind` enum

```
#[derive(Debug, Subcommand)]
pub enum CommandKind {
    /// DeployNaut API actions.
    #[command(name = "deploy:naut")]
    DeployNaut(DeployNautArgs),

    /// Bitbucket Pipelines actions.
    #[command(name = "ci:bitbucket")]
    Bitbucket(BitbucketArgs),
}
```

This is one of Rust's most powerful features: an **enum with data attached**. Each variant carries a different type:

- `DeployNaut(DeployNautArgs)` — when the user types `deploy:naut`, the parsed arguments go here.
- `Bitbucket(BitbucketArgs)` — when the user types `ci:bitbucket`, the parsed arguments go here.

`#[command(name = "deploy:naut")]` is the literal string the user types. Without it, `clap` would default to the variant name (`deploy-naut` with hyphens), but we want to match the PHP original's colon syntax.

`SharedArgs` trait

```
#[allow(dead_code)]
pub trait SharedArgs {
    fn action(&self) -> &str;
}
```

This is a tiny trait with one method. Both `DeployNautArgs` and `BitbucketArgs` implement it (see below). It's a way of saying: "any struct that represents a subcommand's arguments has an `action` method."

```
impl SharedArgs for DeployNautArgs {
    fn action(&self) -> &str {
        &self.action
    }
}
```

This says: "the `DeployNautArgs` struct implements `SharedArgs`, and the `action` method returns a reference to its `action` field."

Why bother? It lets generic code write `args.action()` without caring whether `args` is `DeployNautArgs` or `BitbucketArgs`. Today the code uses each type directly, so `SharedArgs` is currently more aspirational than load-bearing — but it's there for future flexibility.

The `#[allow(dead_code)]` attribute suppresses the warning that the trait is currently unused (only its methods are called through concrete types).

The argument structs

```
#[derive(Debug, clap::Args)]
pub struct DeployNautArgs {
    /// Command action (e.g. `sampleSuccess`, `createDeployment`).
    #[arg(value_name = "ACTION")]
    pub action: String,

    /// [Optional] URL
    #[arg(long)]
    pub url: Option<String>,

    /// [Optional] Git commit SHA
    #[arg(long)]
    pub commit: Option<String>,

    // ... many more ...
}
```

Teaching points:

- `#[derive(clap::Args)]` — marks this as a "leaf" group of arguments that doesn't have its own subcommands.
- `pub action: String` — the positional argument (no `--` prefix). `value_name = "ACTION"` is just the placeholder name shown in `--help`.
- `pub url: Option<String>` — `Option<String>` means "either `Some(value)` or `None`." This is how `clap` represents optional arguments. The user might not have supplied `--url`, in which case the field is `None`.
- `#[arg(long)]` — the long flag form: `--url`, `--commit`, etc. There are also `#[arg(short)]` for `-u` and `#[arg(long, short)]` for both.
- The `///` comments are **doc comments**. `clap` picks them up and shows them in `--help`. The first line becomes the description.

One subtle thing: `r#ref`

```
/// [Optional] Deployment Reference
#[arg(long)]
pub r#ref: Option<String>,
```

Notice the strange `r#` prefix. In Rust, `ref` is a **keyword** (used for taking references in patterns). You can't use a reserved word as a field name directly. The `r#` prefix is called a **raw identifier** — it tells the compiler "I know this looks like a keyword, but treat it as a plain identifier."

Why did the original authors need `ref` as a field name? Because that's exactly the option name in DeployNaut's API: `ref` is the Git reference (branch, tag, or commit SHA) being deployed.

Tests

The `#[cfg(test)]` block at the bottom only gets compiled when you run `cargo test`. It's removed from release builds entirely (`cfg(test)` is `false` in production).

The tests use `Cli::try_parse_from(["nautpie", "deploy:naut", "sampleSuccess"])` — passing the arguments as a slice of strings instead of reading them from `std::env::args()`. This is how you test a `clap` parser without actually invoking the binary.

Why Is It Written This Way?

- **Derive macros over manual parsing.** `clap` generates a robust, well-tested parser at compile time. Writing it by hand would be hundreds of lines and full of edge cases.
- **Case-insensitive action names.** The PHP version accepted `sampleSuccess`, `SampleSuccess`, or `SAMPLESUCCESS` interchangeably (because PHP method calls are case-insensitive). Rust method calls are case-sensitive, so we normalise the input to one canonical form before

dispatch. The `normalise_action()` function exists entirely to preserve that user-facing behavior.

- **Option everywhere.** PHP allowed arguments to be missing without complaint — they'd just be `null`. In Rust, "missing" is best modelled as `Option<T>::None`. Every optional flag uses this type so the action functions can pattern-match cleanly.
- **Two argument structs sharing a trait.** Most flags differ between subcommands (`deploy:naut` has `--ref_type`, `ci:bitbucket` doesn't). Rather than one giant struct with all flags, we have two focused ones. The `SharedArgs` trait is the thin glue between them.

Key Rust Concepts Used Here

Concept	Where	What it does
<code>derive(Parser)</code>	<code>Cli</code>	Auto-generates argument parsing
<code>derive(Subcommand)</code>	<code>CommandKind</code>	Auto-generates subcommand dispatch
<code>derive(clap::Args)</code>	<code>DeployNautArgs</code>	Marks a leaf argument group
<code>derive(Debug)</code>	every struct/enum	Makes the type printable with <code>{:?}</code>
<code>enum</code> with data	<code>CommandKind</code>	A type that can be one of several variants, each holding its own data
<code>Option<T></code>	optional flags	"Some value, or no value at all"
<code>&str</code>	function params	A borrowed view into a string
<code>String</code>	fields	An owned, growable string
<code>Vec<T></code>	inside <code>normalise_action</code>	A growable list
<code>r#ref</code>	field name	Raw identifier — escape a reserved word
<code>#[arg(...)]</code>	field attributes	Configures how <code>clap</code> parses that field
<code>#[command(...)]</code>	struct/enum attributes	Configures how <code>clap</code> treats the whole group
<code>#[cfg(test)]</code>	tests module	"Only compile this in test mode"
<code>impl Trait for Type</code>	<code>SharedArgs</code> impls	Implementing a trait for a specific type
<code>trait</code>	<code>SharedArgs</code>	A set of methods a type must implement
Doc comments <code>///</code>	field comments	Documentation that <code>clap</code> and <code>rustdoc</code> both read

CHAPTER 9: PUTTING IT ALL TOGETHER

 Source: `src/commands/sample.md`

This is the smallest `commands/` file. It implements the two **trivially-simple** actions: `sampleSuccess` and `sampleFail`. They exist to:

1. Verify that the CLI dispatch system works end-to-end.
2. Give CI scripts a way to test the binary without making any real network calls.

If you've ever wired up a CLI framework before, you've probably seen this pattern: a "hello world" subcommand that exercises the parser and dispatcher without doing anything substantive.

Walkthrough

Constants

```
#[allow(dead_code)]
pub const SUCCESS_MESSAGE: &str = "[Action:Success] Response successful.";
#[allow(dead_code)]
pub const FAIL_MESSAGE: &str = "[Action:Fail] Has failed.";
```

Two `const` strings. Like the `DEFAULT_TITLE` we saw in `deployment_details.rs`, these are inlined by the compiler at every use site.

`#[allow(dead_code)]` suppresses the warning that the constants are technically unused — they're declared for symmetry and in case future code wants to reference them.

The `[Action:Success]` and `[Action:Fail]` prefixes match the PHP original's error/message format exactly.

sample_success()

```
pub fn sample_success() -> ApiResponse {
    ApiResponse::ok(json!(SUCCESS_MESSAGE))
}
```

This is the entire implementation. It:

1. Wraps the `SUCCESS_MESSAGE` constant in a `serde_json::json!()` macro call.
2. Passes the resulting JSON value to `ApiResponse::ok(...)`.
3. Returns the `ApiResponse`.

Teaching points:

- `json!(...)` **macro** — from `serde_json`. It looks like JSON syntax but runs at compile time (or rather, at runtime, but it produces a `serde_json::Value`). It can contain variables: `json!({ "name": user_name })`.
- `ApiResponse::ok(...)` — the constructor we saw in `error.rs`. It produces `{ status: 200, reason: "OK", body: <whatever> }`.

So when you run:

```
nautpie deploy:naut sampleSuccess
```

...the program prints:

```
{"status":200,"reason":"OK","body":"[Action:Success] Response successful."}
```

...and exits with code 0.

sample_fail()

```
pub fn sample_fail() -> Result<ApiResponse, Error> {
    Err(Error::Generic(FAIL_MESSAGE.into()))
}
```

Returns `Err(Error::Generic(...))`. The dispatcher in `main.rs` catches this and:

1. Prints the error message as a warning on stderr.
2. Prints a JSON envelope on stdout with `status: 1, reason: "Bad Request", body: <error>`.
3. Exits with code 1.

The return type is `Result<ApiResponse, Error>` rather than just `Error` because Rust forces every function to have a single return type. Returning a `Result` makes it composable with other code that returns `Result`s.

Teaching points:

- `Err(Error::Generic(...))` — constructs the `Error::Generic` variant (the catch-all error) wrapping the `FAIL_MESSAGE`.
- `FAIL_MESSAGE.into()` — `FAIL_MESSAGE` is a `&str`; `Error::Generic` takes a `String`. The `Into` trait handles the conversion.

How it's wired into `DeployNaut::run()`

In `deploy_naut.rs`, the dispatch is:

```
match action.to_ascii_lowercase().as_str() {
    "samplesuccess" => Ok(sample::sample_success()),
    "samplefail" => sample::sample_fail(),
    // ... other actions ...
}
```

Note how `sample_success()` already returns `ApiResponse`, so we wrap it in `Ok(...)`. `sample_fail()` already returns `Result<ApiResponse, Error>`, so we use it as-is.

Tests

```
#[test]
fn success_message_round_trip() {
    let r = sample_success();
    assert_eq!(r.status, 200);
    assert_eq!(r.reason, "OK");
    assert_eq!(r.body, json!(SUCCESS_MESSAGE));
}

#[test]
fn fail_returns_error() {
    let err = sample_fail().unwrap_err();
    assert!(matches!(err, Error::Generic(ref m) if m == FAIL_MESSAGE));
}
```

Two quick tests that verify:

- `sample_success()` produces the expected envelope.
- `sample_fail()` produces the expected error.

Teaching points:

- `matches!(err, Error::Generic(ref m) if m == FAIL_MESSAGE)` — pattern-match with a guard. `ref m` means "bind a reference to the inner `String` as `m`." The guard `if m == FAIL_MESSAGE` checks the bound value. Note: this works because `Error` derives `PartialEq`, so `m == FAIL_MESSAGE` is allowed.

Why Is It Written This Way?

- **Tiny smoke tests.** The whole point of these actions is that they're tiny. They prove the CLI works without exercising any real API.
- **PHP parity.** The PHP original had `doSampleSuccess` and `doSampleFail` methods on its helper class. The Rust port keeps the same names and the same prefix-tagged messages.
- **No network calls.** Everything happens in memory. CI scripts can run `nautpie deploy:naut sampleSuccess` to verify the binary is callable.
- **Real `Result` plumbing.** Even though these functions are trivial, they exercise the same `Result<ApiResponse, Error>` plumbing as the real actions. So they're a meaningful test of the dispatcher.

Key Rust Concepts Used Here

Concept	Where	What it does
<code>pub const NAME: &str = "..."</code>	constants	Compile-time string constant
<code>#[allow(dead_code)]</code>	constants	Suppress unused-code warnings
<code>json!(...)</code> macro	<code>sample_success</code>	Build a <code>serde_json::Value</code> from JSON-like syntax
<code>ApiResponse::ok(...)</code>	<code>sample_success</code>	Construct a success response
<code>Result<T, E></code>	<code>sample_fail</code> return	Success-or-failure return type
<code>Err(variant(field))</code>	<code>sample_fail</code> body	Construct an error value
<code>.into()</code>	<code>FAIL_MESSAGE.into()</code>	Convert <code>&str</code> to <code>String</code>
<code>matches!(expr, pat)</code>	test	Pattern-match an expression in one line
<code>ref</code> pattern	test	Bind a reference to the inner value
Pattern guard <code>if m == ...</code>	test	Refine a pattern with an extra condition
<code>unwrap_err()</code>	test	Assert that a <code>Result</code> is <code>Err</code> and extract it
<code>assert_eq!(a, b)</code>	tests	Verify equality, panic with diff otherwise

 Source: `src/commands/deploy_naut.md`

This file is the **largest in the project** — about 350 lines. It implements the `deploy:naut <action>` subcommand, which talks to DeployNaut's REST API.

If `cli.rs` is the **front door** (parsing what the user typed) and `main.rs` is the **conductor** (deciding what to run), this file is the **stage** where most of the actual work happens.

The Big Picture

`deploy:naut` supports these actions:

Action	What it does
<code>sampleSuccess</code>	Returns a fixed success message. No network.
<code>sampleFail</code>	Returns a fixed failure. No network.
<code>fetch</code>	GETs a path under the API endpoint.
<code>createDeployment</code>	POSTs a new deployment payload.
<code>gitFetch</code>	Asks DeployNaut to refresh its git cache, then polls until done.
<code>getDeployments</code>	Lists recent deployments, optionally filtered by commit.
<code>lastDeployment</code>	Returns the most recent deployment.
<code>checkDeploymentProgress</code>	Polls a single deployment until it completes.

Every action goes through:

1. `setup_deploy_naut_env(http)` — verify the required env vars are set and configure the HTTP client.
2. Build a URL or payload.
3. Call `http.request(...)`.
4. Decode the JSON response and return an `ApiResponse`.

Walkthrough

Constants

```
pub const DEPLOY_TIMEOUT_SECS: u64 = 1800; // 30 minutes
pub const GIT_TIMEOUT_SECS: u64 = 120;    // 2 minutes
```

Two `u64` (unsigned 64-bit integer) constants. They're `u64` because the `Duration::from_secs(...)` constructor takes a `u64`. These are the **deadlines** for polling operations: 30 minutes for a deployment, 2 minutes for a git fetch.

`pub` means they're visible to other modules. The integration tests use them to shorten the deadline for faster test runs.

The `DeployNaut` struct and `Command` impl

```
pub struct DeployNaut;

impl Command for DeployNaut {
    fn name(&self) -> &'static str { "deploy:naut" }

    fn run(&self, action: &str, io: &mut dyn Io, http: &mut dyn HttpClient)
        -> Result<ApiResponse, Error>
    {
        let opts = io.options()
            .ok_or_else(|| Error::MissingOption("options".into()))?;
        match action.to_ascii_lowercase().as_str() {
            "samplesuccess" => Ok(sample::sample_success()),
            "samplefail"    => sample::sample_fail(),
            "fetch"         => do_fetch(&opts, http),
            "createdeployment" => do_create_deployment(&opts, io, http, None),
            "gitfetch"      => do_git_fetch(&opts, http),
            "getdeployments" => do_get_deployments(&opts, http),
            "lastdeployment" => do_last_deployment(&opts, http),
            "checkdeploymentprogress" => do_check_deployment_progress(&opts, http),
            _ => Err(Error::MissingAction),
        }
    }
}
```

Teaching points:

- `io.options().ok_or_else(|| Error::MissingOption("options".into()))?` — three Rust idioms in one line:
- `io.options()` returns `Option<Options>`.
- `.ok_or_else(|| ...)` converts it to `Result<Options, Error>`. The closure form (`ok_or_else`) is preferred over `ok_or` when constructing the error is non-trivial, because the closure only runs on `None`.
- `?` returns early with the error if it's `Err`.
- `action.to_ascii_lowercase().as_str()` — converts the action string to lowercase ASCII, then borrows it as `&str` so we can pattern-match in the `match`. We could also use `eq_ignore_ascii_case` for each arm, but lowercase-then-match is cleaner.
- Each arm returns a `Result<ApiResponse, Error>` but with different shapes:
- `sample_success()` returns `ApiResponse` directly, so we wrap in `Ok(...)`.

- `sample_fail()` already returns `Result<ApiResponse, Error>`.
- `do_fetch(...)` etc. return `Result<ApiResponse, Error>` directly.

`options_from_deploy_args()`

```
pub fn options_from_deploy_args(args: &DeployNautArgs) -> Options {
    let mut opts = Options::new();
    if let Some(v) = &args.url { opts.set("url", v.clone()); }
    if let Some(v) = &args.commit { opts.set("commit", v.clone()); }
    // ... etc ...
    opts
}
```

This converts a `DeployNautArgs` (from `cli.rs`) into an `Options` bag. It's called once in `main.rs` after `clap` parses the args.

Teaching points:

- `if let Some(v) = &args.url` — pattern-match only the `Some` variant. If `args.url` is `None`, the block is skipped silently.
- `v.clone()` — we need an owned `String` to put into the `Options` hash map; `args.url` is `Option<String>` and we're borrowing it as `&String`, so we have to clone.
- **Repeated `if let`** — verbose but explicit. The alternative (a helper macro or iterator chain) would be cleverer but harder to read.

There's also `options_from_bitbucket_args()` which does the same thing for the Bitbucket subcommand.

`setup_deploy_naut_env()`

```
pub fn setup_deploy_naut_env(http: &mut dyn HttpClient)
    -> Result<(String, String, String), Error>
{
    let endpoint = std::env::var("NAUT_ENDPOINT")
        .ok()
        .filter(|v| !v.is_empty())
        .ok_or(Error::MissingEndpoint)?;
    http.set_endpoint(&endpoint);
    http.set_content_type("application/json");
    let creds = check_envs(&["DASH_USER", "DASH_TOKEN"])?;
    let dash_user = creds[0].clone();
    let dash_token = creds[1].clone();
    http.set_username_and_password(&dash_user, &dash_token);
    Ok((endpoint, dash_user, dash_token))
}
```

This is the **shared setup** for every action that talks to DeployNaut. It:

1. Reads `NAUT_ENDPOINT` (returns `MissingEndpoint` if unset or empty).
2. Tells the HTTP client what the base endpoint is.
3. Resets the content type to `application/json`.
4. Reads `DASH_USER` and `DASH_TOKEN` (returns `MissingEnv` if either is unset).
5. Tells the HTTP client what the basic-auth credentials are.
6. Returns the three values so callers don't need to re-read them.

Teaching points:

- `std::env::var("NAUT_ENDPOINT").ok()` — convert `Result<String, VarError>` to `Option<String>`. `.ok()` discards the error info, which is fine here.
- `.filter(|v| !v.is_empty())` — if the value is an empty string, treat it as `None`. This matches `check_envs` semantics.
- `.ok_or(Error::MissingEndpoint)?` — convert `Option<String>` to `Result<String, Error>`, then return early on `None`. This is the same pattern as `io.options().ok_or_else(...)` above.
- `let creds = check_envs(...)?` — the `?` operator propagates the error. If `DASH_USER` or `DASH_TOKEN` is missing, we bail out.

- **Returning a 3-tuple** — `Result<(String, String, String), Error>`. Callers destructure: `let (endpoint, user, token) = setup_deploy_naut_env(http)?;`

`do_fetch()` — the simplest real action

```
pub fn do_fetch(opts: &Options, http: &mut dyn HttpClient) -> Result<ApiResponse, Error> {
    let url = opts.option("url").ok_or_else(|| Error::MissingOption("url".into()))?;
    let resp = http.request("GET", &url, None, None)?;
    let body: Value = serde_json::from_str(&resp.body)
        .map_err(|e| Error::Json(format!("decode failed: {e}")))?;
    Ok(ApiResponse {
        status: resp.status,
        reason: resp.reason,
        body,
    })
}
```

The pattern is repeated in every action:

1. Read a required option, return `MissingOption` if absent.
2. Call `http.request(...)`.
3. Decode the JSON body.
4. Construct an `ApiResponse` from the status, reason, and body.

Teaching points:

- `serde_json::from_str(&resp.body)` — parse the response body as JSON into a `Value`. Returns `Result<Value, serde_json::Error>`.
- `.map_err(|e| Error::Json(format!("decode failed: {e}")))` — convert the parse error into our `Error::Json` variant. The closure captures `e` and uses `format!` to build the message.
- **Returning a struct literal** — `Ok(ApiResponse { status: ..., reason: ..., body })`. This is the same as calling a constructor but explicitly named fields. It works because the struct's fields are all `pub`.

`do_create_deployment()` — the most complex action

This is the heart of the `deploy:naut` subcommand. It builds a `DeploymentDetails` payload, optionally applies some overrides (used by `do_deploy_package` in `bitbucket.rs`), POSTs it, and optionally polls for completion.

```
pub fn do_create_deployment(
    opts: &Options,
    io: &mut dyn Io,
    http: &mut dyn HttpClient,
    overrides: Option<DeploymentOverrides>,
) -> Result<ApiResponse, Error> {
    setup_deploy_naut_env(http)?;
    let stack = opts.option("stack").ok_or_else(|| Error::MissingOption("stack".into()))?;
    let environment = opts.option("environment").ok_or_else(|| Error::MissingOption("environment".into()))?;
    // ... build details ...
    let relative_url = format!("project/{stack}/environment/{environment}/deploys");
    let body_str = serde_json::to_string(&details.values())
        .map_err(|e| Error::Json(format!("encode failed: {e}")))?;
    let response = http.request("POST", &relative_url, None, Some(body_str))?;
    // ... handle should_wait ...
}
```

Teaching points:

- `Option<DeploymentOverrides>` — the action accepts **either** options from the CLI **or** explicit overrides (used when `do_deploy_package` synthesises ref/title/summary from the Bitbucket download URL).

- **The override chain** — for each field, the code first checks `overrides`, then falls back to `opts`: `rust if let Some(r) = overrides.as_ref().and_then(|o| o.ref_.clone()) { details = details.ref_(r); } else if let Some(r) = opts.option("ref") { details = details.ref_(r); }` This is verbose but explicit. The alternative is to merge the two sources into one struct first.
- `.and_then(|o| o.ref_.clone())` — `overrides.as_ref()` is `Option<&DeploymentOverrides> . and_then(|o| ...)` is `Option::and_then`, which is like `map` but the closure returns another `Option`. The closure clones the inner `Option<String>` to convert from `&Option<String>` to `Option<String>`. So if `overrides` is `None`, we get `None`; if `overrides` is `Some(o)` but `o.ref_` is `None`, we get `None`; if both are `Some`, we get `Some(value)`.
- `format!("project/{stack}/environment/{environment}/deploys")` — inline variable interpolation in a format string. Equivalent to `format!("project/{}/environment/{}/deploys", stack, environment)` but more concise.
- `Some(body_str)` — pass the serialized JSON as the request body. The HTTP client knows to send this as the request body because it's `Some`.

`do_git_fetch()` and the polling pattern

```
pub fn do_git_fetch(opts: &Options, http: &mut dyn HttpClient) -> Result<ApiResponse, Error> {
    let stack = opts.option("stack").ok_or_else(|| Error::MissingOption("stack".into()))?;
    setup_deploy_naut_env(http)?;
    let relative_url = format!("project/{stack}/git/fetches");
    let response = http.request("POST", &relative_url, None, None)?;

    if response.status != 202 {
        return Err(Error::HttpStatus { /* ... */ });
    }

    let body = decode_body(&response.body)?;
    let fetch_id = body.get("data").and_then(|d| d.get("id")).and_then(|i| i.as_i64())
        .ok_or_else(|| Error::Generic("[Error:GitFetch] missing data.id".into()))?;

    let poll_url = format!("project/{stack}/git/fetches/{fetch_id}");
    let poll_url_for_closure = poll_url.clone();
    let label = format!("Git fetch for {stack} ({fetch_id})");
    run_with_progress(
        "gitFetch",
        label,
        Duration::from_secs(GIT_TIMEOUT_SECS),
        http,
        move |http| poll_status(http, &poll_url_for_closure, "status"),
    );
    // ... final GET ...
}
```

This is the **two-step polling pattern**: trigger an async operation, then poll until it completes.

Teaching points:

- **HTTP 202 (Accepted)** — DeployNaut's git-fetch endpoint returns 202 with a job ID. We poll a separate URL until the job is done.
- `.get("data").and_then(|d| d.get("id"))` — chained `Option::and_then` for nested JSON lookup. This is the standard idiom for "navigate a JSON tree without panicking."
- `poll_url_for_closure = poll_url.clone()` — the `move` closure needs to own its captured variables. We clone the URL so the outer `poll_url` is still usable after the closure moves in.
- `move |http| poll_status(http, &poll_url_for_closure, "status")` — a **move closure**: the `move` keyword moves the captured variables (`poll_url_for_closure`) into the closure. Without `move`, the closure would borrow them, but closures that outlive the calling function (which polling closures do) must own their data.
- `run_with_progress(...)` — see `tui/spinner.rs`. It runs the polling loop with a progress UI.

do_get_deployments()

```
pub fn do_get_deployments(opts: &Options, http: &mut dyn HttpClient) -> Result<ApiResponse, Error> {
    let stack = opts.option("stack").ok_or_else(|| Error::MissingOption("stack".into()))?;
    let environment = opts.option("environment").ok_or_else(|| Error::MissingOption("environment".into()))?;
    let start_date = opts.option("startDate").unwrap_or_else(|| "-1 year".into());

    setup_deploy_naut_env(http)?;
    let response = fetch_deployments(http, &stack, &environment, &start_date)?;

    let mut deployments = response.body.get("data")
        .and_then(|d| d.as_array())
        .cloned()
        .unwrap_or_default();

    if let Some(commit) = opts.option("commit") {
        let commit_key = if commit.len() == 7 { "short_sha" } else { "sha" };
        let accepted_states = ["New", "Submitted", "Approved", "Queued", "Deploying", "Completed"];
        deployments.retain(|item| {
            item.get("attributes")
                .and_then(|a| a.get(commit_key))
                .and_then(|v| v.as_str())
                == Some(commit.as_str())
            && item.get("attributes")
                .and_then(|a| a.get("state"))
                .and_then(|v| v.as_str())
                .map(|s| accepted_states.contains(&s))
                .unwrap_or(false)
        });
    }

    deployments.sort_by(|a, b| { /* ... */ });

    Ok(ApiResponse { status: response.status, reason: response.reason, body: json!({ "data": deployments }) })
}
```

This action lists deployments from the past N days and optionally filters them by commit SHA.

Teaching points:

- `opts.option("startDate").unwrap_or_else(|| "-1 year".into())` — if the user didn't supply `--startDate`, default to the string `"-1 year"`. The `unwrap_or_else` form is used because the default requires allocation (the `.into()`).
- `.cloned().unwrap_or_default()` — three operations chained:
- `.get("data").and_then(|d| d.as_array())` returns `Option<&Vec<Value>>`.
- `.cloned()` converts to `Option<Vec<Value>>`.
- `.unwrap_or_default()` gives us `Vec<Value>`, defaulting to an empty vector if the key was missing.
- `deployments.retain(|item| ...)` — like `Vec::retain` in many languages: keep only the items where the closure returns `true`.
- **The `commit_key` ternary** — if the commit is 7 chars long, it's a "short SHA" (the truncated form some platforms show); otherwise it's a full 40-char SHA. DeployNaut stores both.
- `accepted_states.contains(&s)` — array `.contains` checks for membership. Returns `true` if `s` is one of the listed states.

do_last_deployment() — composition over duplication

```
pub fn do_last_deployment(opts: &Options, http: &mut dyn HttpClient) -> Result<ApiResponse, Error> {
    let mut response = do_get_deployments(opts, http)?;
    let first = response.body
        .get("data")
        .and_then(|d| d.as_array())
        .and_then(|arr| arr.first().cloned())
        .unwrap_or(Value::Null);
    response.body = first;
    Ok(response)
}
```

This is beautifully small. It calls `do_get_deployments` (which already sorts newest-first) and grabs the first item.

Teaching points:

- `arr.first().cloned()` — `first()` returns `Option<&Value>`. `.cloned()` makes it `Option<Value>`. We need an owned `Value` so we can move it into `response.body`.
- `.unwrap_or(Value::Null)` — if the array is empty, default to JSON `null`. This is the cleanest "missing data" representation in JSON.
- **Composition over duplication.** Rather than reimplementing the listing + sorting logic, this function **delegates** to `do_get_deployments` and trims the result. This is the **single source of truth** principle.

`do_check_deployment_progress()`

```
pub fn do_check_deployment_progress(opts: &Options, http: &mut dyn HttpClient) -> Result<ApiResponse, Error> {
    let stack = opts.option("stack").ok_or_else(|| Error::MissingOption("stack".into()))?;
    let environment = opts.option("environment").ok_or_else(|| Error::MissingOption("environment".into()))?;
    let deploy_id_str = opts.option("deploy_id").ok_or_else(|| Error::MissingOption("deploy_id".into()))?;
    let deploy_id: i64 = deploy_id_str.parse()
        .map_err(|_| Error::Generic("[Required:Option] deploy_id must be an integer".into()))?;
    check_deployment_progress(deploy_id, &stack, &environment, &mut StderrShim, http)?;
    Ok(ApiResponse::ok(json!("Deployment progress check complete")))
}
```

Teaching points:

- `deploy_id_str.parse()` — parse the string into `i64`. The turbofish isn't needed because the `let deploy_id: i64 = ...` annotation on the left tells the compiler the target type.
- `.map_err(|_| ...)` — convert `ParseIntError` into our error. The closure ignores the parse error.
- `&mut StderrShim` — passes a no-op `Io` impl. The polling code in `check_deployment_progress` doesn't actually use `Io` (it only renders progress through the TUI), so we pass a dummy that ignores all messages.

Helper functions

`fetch_deployments()`

```
pub fn fetch_deployments(http, stack, environment, start_date) -> Result<ApiResponse, Error> {
    let since_unix = strtotime_or_now(start_date);
    let query = format!("datestarted_from_unix={since_unix}");
    let relative_url = format!("project/{stack}/environment/{environment}/deploys?{query}");
    let response = http.request("GET", &relative_url, None, None)?;
    if response.status != 200 { return Err(Error::HttpStatus { /* ... */ }); }
    let body = decode_body(&response.body)?;
    Ok(ApiResponse { status: response.status, reason: response.reason, body })
}
```

Just a GET with a query parameter. Note the explicit check for status 200 — DeployNaut returns other codes if the date is in the future or the stack doesn't exist.

`check_deployment_progress()`

The polling core. Same pattern as `do_git_fetch` but with a longer deadline (`DEPLOY_TIMEOUT_SECS = 1800`).

StderrShim

```
struct StderrShim;
impl Io for StderrShim {
    fn warning(&mut self, _: &str) {}
    fn success(&mut self, _: &str) {}
    fn message(&mut self, _: &str) {}
    fn write_json(&mut self, _: &ApiResponse) {}
}
```

A no-op **Io** implementation. All methods take **_** (underscore) arguments — the underscore tells the compiler "I don't care about this argument." All bodies are empty (**{}**) — the methods do nothing.

This is needed because **check_deployment_progress** requires an **&mut dyn Io** parameter, but we don't have a real **Io** to pass at that point in the call stack. The shim satisfies the type system without doing anything.

strtotime_or_now()

```
fn strtotime_or_now(s: &str) -> i64 {
    if let Ok(n) = s.parse::<i64>() { return n; }
    if let Ok(dt) = DateTime::parse_from_rfc3339(s) { return dt.timestamp(); }
    match s.trim().to_ascii_lowercase().as_str() {
        "-1 year" | "last year" => (Utc::now() - chrono::Duration::days(365)).timestamp(),
        "-1 month" | "last month" => (Utc::now() - chrono::Duration::days(30)).timestamp(),
        "-1 day" | "yesterday" => (Utc::now() - chrono::Duration::days(1)).timestamp(),
        _ => Utc::now().timestamp(),
    }
}
```

A partial re-implementation of PHP's **strtotime**. Supports the formats used by the tests:

- Numeric Unix epoch (e.g., **"1700000000"**).
- RFC3339 timestamp (e.g., **"2026-12-25T10:00:00Z"**).
- Common relative strings: **"-1 year"**, **"last year"**, **"-1 month"**, etc.
- Anything else: fall back to "now."

Teaching points:

- **chrono::Duration::days(365)** — a **Duration** of 365 days. Subtracting it from **Utc::now()** gives us a moment in the past.
- **Utc::now()** — the current UTC time as a **DateTime<Utc>**.
- **.timestamp()** — extract the Unix epoch seconds as **i64**.

decode_body() and poll_status()

Tiny helpers. **decode_body** parses JSON or returns **Value::Null** on empty input. **poll_status** extracts the "status" field from the response and returns whether the operation is done.

Why Is It Written This Way?

- **Every action is its own function.** Even though they share setup logic, each action is isolated. This makes them easy to test individually.
- **DeploymentOverrides is passed explicitly.** Rather than smuggling overrides through env vars or globals, the Bitbucket command builds an **Options** bag that already has the right values, then optionally passes overrides. This keeps the data flow explicit.
- **PHP parity.** Each action corresponds to a method on the PHP **DeployNautCommand** class. The HTTP shape, the JSON keys, the env vars — all preserved.
- **Composition over duplication.** **do_last_deployment** reuses **do_get_deployments**. **do_deploy_package** (in **bitbucket.rs**) reuses **do_create_deployment**. The codebase favors small, composable functions.

Key Rust Concepts Used Here

Concept	Where	What it does
<code>pub struct Name;</code>	<code>DeployNaut</code>	Unit struct — just a tag for trait impl
<code>impl Command for DeployNaut</code>	top	Implement the <code>Command</code> trait
<code>match action.to_ascii_lowercase().as_str()</code>	dispatch	Match on a normalised string
<code>ok_or_else(\ _\\ ...)</code>	option reads	Convert <code>Option</code> to <code>Result</code> with a lazy error
<code>?</code>	everywhere	Early-return on <code>Err</code>
<code>.ok()</code>	env vars	<code>Result<T, E> → Option<T></code>
<code>.filter(\ v\\ ...)</code>	env vars	Refine an <code>Option</code>
<code>Option<T>::and_then</code>	JSON traversal	Chain nested lookups safely
<code>Vec::retain</code>	deployments filter	Keep only items matching a predicate
<code>Vec::sort_by</code>	deployments sort	Sort with a custom comparator
<code>unwrap_or_else(\ _\\ ...)</code>	default value	Default to a lazy computed value
<code>unwrap_or_default()</code>	missing data	Default to the type's <code>Default::default()</code>
<code>String::parse::<i64>()</code>	deploy_id	Parse a string into an integer
<code>.map_err(\ _\\ ...)</code>	parse errors	Convert error types
<code>.cloned()</code>	cloned JSON	<code>Option<T> → Option<T></code>
<code>move</code> closure	polling closure	Move captured variables into the closure
<code>Duration::from_secs(n)</code>	timeouts	Create a duration from seconds
<code>format!("{}", var)</code>	URLs	String interpolation
<code>json!({ "data": deployments })</code>	response body	Construct a JSON value
<code>#[allow(dead_code)]</code>	constants	Suppress unused-code warnings
<code>StderrShim</code>	private struct	No-op <code>Io</code> impl for internal use

 Source: `src/commands/bitbucket.md`

This file implements the `ci:bitbucket <action>` subcommand, which integrates with Bitbucket Pipelines. It mirrors the PHP `BitbucketCommand` class.

While `deploy_naut.rs` is large and self-contained, this file is **smaller** but **more dependent** — `do_deploy_package` calls back into `deploy_naut.rs` to do the actual deployment after getting a Bitbucket download URL.

The Big Picture

`ci:bitbucket` supports three actions:

Action	What it does
<code>createAccessToken</code>	POSTs to Bitbucket's OAuth endpoint with <code>grant_type=client_credentials</code> to get a temporary access token.
<code>createTag</code>	POSTs a tag reference to a Bitbucket repo. The tag string is truncated to match a PHP quirk.
<code>deployPackage</code>	Combines <code>createAccessToken</code> + building a download URL + calling <code>do_create_deployment</code> .

The `do_deploy_package` action is the one that ties Bitbucket and DeployNaut together: it gets a token, builds a download URL, and then **calls into** the `deploy_naut` module to perform the actual deployment.

Walkthrough

Constant

```
const BITBUCKET_OAUTH_DEFAULT: &str = "https://bitbucket.org/site";
```

The default Bitbucket OAuth endpoint. Used unless `BB_OAUTH_ENDPOINT` is set in the environment (which lets tests point at a `wiremock` server).

The `Bitbucket` struct and `Command` impl

```
pub struct Bitbucket;

impl Command for Bitbucket {
    fn name(&self) -> &'static str { "ci:bitbucket" }

    fn run(&self, action: &str, io: &mut dyn Io, http: &mut dyn HttpClient)
        -> Result<ApiResponse, Error>
    {
        let opts = io.options().ok_or_else(|| Error::MissingOption("options".into()))?;
        match action.to_ascii_lowercase().as_str() {
            "createaccesstoken" => do_create_access_token(&opts, http),
            "createtag"         => do_create_tag(&opts, http),
            "deploypackage"     => do_deploy_package(&opts, io, http),
            _                   => Err(Error::MissingAction),
        }
    }
}
```

Same structure as `DeployNaut::run()`. The match arms route to the three action functions.

do_create_access_token()

```
pub fn do_create_access_token(_opts: &Options, http: &mut dyn HttpClient) -> Result<ApiResponse, Error> {
    let consumer_key = std::env::var("BB_CONSUMER_KEY").map_err(|_| Error::MissingEnv("BB_CONSUMER_KEY".into()))?;
    let consumer_secret = std::env::var("BB_CONSUMER_SECRET").map_err(|_| Error::MissingEnv("BB_CONSUMER_SECRET".into()))

    let endpoint = std::env::var("BB_OAUTH_ENDPOINT").ok().filter(|v| !v.is_empty())
        .unwrap_or_else(|| BITBUCKET_OAUTH_DEFAULT.to_string());
    http.set_endpoint(&endpoint);
    http.set_content_type("application/x-www-form-urlencoded");
    http.set_username_and_password(&consumer_key, &consumer_secret);

    let resp = http.request("POST", "oauth2/access_token/", None, Some("grant_type=client_credentials".into()));
    if resp.status != 200 {
        return Err(Error::HttpStatus { status: resp.status, reason: resp.reason, body: resp.body });
    }
    let body: Value = serde_json::from_str(&resp.body).map_err(|e| Error::Json(format!("decode failed: {e}")))?;
    Ok(ApiResponse { status: resp.status, reason: resp.reason, body })
}
```

This is the OAuth client-credentials flow. Bitbucket returns:

```
{ "access_token": "...", "scopes": "...", "expires_in": 3600 }
```

Teaching points:

- **_opts: &Options** — the underscore prefix means "I don't use this parameter." The **_opts** is there because the trait signature requires it.
- **std::env::var(...).map_err(|_| Error::MissingEnv("NAME".into()))?** — the standard pattern for "env var or error." **map_err** converts the standard library's **VarError** into our **Error::MissingEnv**. The **?** propagates the error.
- **.ok().filter(|v| !v.is_empty()).unwrap_or_else(...)** — three operations chained:
- **.ok()** — convert **Result** to **Option**.
- **.filter(...)** — keep the value only if it's non-empty (so an unset-but-empty var doesn't override the default).
- **.unwrap_or_else(...)** — fall back to the default.
- **http.set_content_type("application/x-www-form-urlencoded")** — the access token request is a form submission, not JSON.
- **http.set_username_and_password(&consumer_key, &consumer_secret)** — sets the HTTP basic-auth header. Bitbucket's OAuth expects the consumer key and secret as basic-auth credentials.
- **Some("grant_type=client_credentials".into())** — passes the form-encoded body. The **.into()** converts the **&str** to a **String** so it matches the **Option<String>** parameter type.
- **The trailing if resp.status != 200** — explicitly check for success. Other status codes (401, 403, etc.) become **Error::HttpStatus**.

do_create_tag() — the truncation quirk

```
pub fn do_create_tag(opts: &Options, http: &mut dyn HttpClient) -> Result<ApiResponse, Error> {
    let commit = opts.option("commit").ok_or_else(|| Error::MissingOption("commit".into()))?;
    if commit.len() != 40 {
        return Err(Error::Generic("[Action:DeployPackage] Requires 40-char commit".into()));
    }
    let mut tag = opts.option("tag").ok_or_else(|| Error::MissingOption("tag".into()))?;

    let (owner, slug) = match (std::env::var("BITBUCKET_REPO_OWNER").ok(), std::env::var("BITBUCKET_REPO_SLUG").ok()) {
        (Some(o), Some(s)) if !o.is_empty() && !s.is_empty() => (o, s),
        _ => return Err(Error::MissingEnv("BITBUCKET_REPO_OWNER/BITBUCKET_REPO_SLUG".into())),
    };

    let endpoint = /* ... */;
    http.set_endpoint(&endpoint);

    // PHP `strrpos($tag, $commit)` — substring truncation:
    // if the tag string contains the full commit, keep up to 7 chars of the commit.
    if let Some(pos) = tag.rfind(&commit) {
        let cap = std::cmp::min(7, commit.len());
        tag.truncate(pos + cap);
    }

    let url = format!("repositories/{owner}/{slug}/refs/tags");
    let payload = json!({
        "name": tag,
        "target": { "hash": commit },
    });
    let body_str = serde_json::to_string(&payload).map_err(|e| Error::Json(format!("encode failed: {e}")))?;
    let resp = http.request("POST", &url, None, Some(body_str));
    let body: Value = serde_json::from_str(&resp.body).map_err(|e| Error::Json(format!("decode failed: {e}")))?;
    Ok(ApiResponse { status: resp.status, reason: resp.reason, body })
}
```

This is the most idiosyncratic function in the file because of the **PHP truncation quirk**.

Teaching points:

- **if commit.len() != 40** — Bitbucket SHA-1s are exactly 40 hex characters. Anything else is rejected.
- **let mut tag = ...** — **mut** because we'll truncate it later. Without **mut**, you can't reassign or call mutating methods on a variable.
- **match (a, b)** — Rust's tuple pattern matching. The first arm matches when both env vars are **Some(non-empty)**; the second arm catches everything else (including both **None**, one **None** one **Some**, or both **Some** but one empty). The **if !o.is_empty() && !s.is_empty()** is a **match guard** that refines the first arm.
- **tag.rfind(&commit)** — find the last occurrence of **commit** in **tag**. Returns **Option<usize>** (the byte position, or **None**).
- **tag.truncate(pos + cap)** — keep the first **pos + cap** bytes of the tag, drop the rest. This is the **PHP substr(\$tag, 0, strrpos(\$tag, \$commit) + min(7, strlen(\$commit)))** behavior.
- **std::cmp::min(7, commit.len())** — **commit.len()** is 40, so **min(7, 40) == 7**. The **min** is defensive: if a future version supports shorter commits, this still works.
- **json!({ "name": tag, "target": { "hash": commit } })** — the **json!** macro supports nested object syntax. It produces **{"name": "...", "target": {"hash": "..."} }**.

Why the truncation?

The PHP original had this line:

```
$tag = substr($tag, 0, strrpos($tag, $commit) + min(7, strlen($commit)));
```

The intent (according to the comments in the source) was: "if the tag string contains the full 40-char commit, truncate to the first 7 chars of the commit so the tag is shorter." This was probably for some legacy reason — modern tags don't need this. The Rust port preserves the exact behavior for PHP parity.

do_deploy_package() — composition

```
pub fn do_deploy_package(opts: &Options, _io: &mut dyn Io, http: &mut dyn HttpClient) -> Result<ApiResponse, Error> {
    let commit = opts.option("commit").ok_or_else(|| Error::MissingOption("commit".into()))?;
    if commit.len() != 40 { return Err(Error::Generic("...".into())); }

    let _stack = opts.option("stack").ok_or_else(|| Error::MissingOption("stack".into()))?;
    let _environment = opts.option("environment").ok_or_else(|| Error::MissingOption("environment".into()))?;

    let branch = std::env::var("BITBUCKET_BRANCH").ok();
    let owner = std::env::var("BITBUCKET_REPO_OWNER").map_err(|_| Error::MissingEnv("BITBUCKET_REPO_OWNER".into()))?;
    let slug = std::env::var("BITBUCKET_REPO_SLUG").map_err(|_| Error::MissingEnv("BITBUCKET_REPO_SLUG".into()))?;
    let endpoint = std::env::var("BB_ENDPOINT").map_err(|_| Error::MissingEndpoint)?;

    let token_response = do_create_access_token(opts, http)?;
    eprintln!("[bitbucket] token_response: {:?}", token_response.body);
    let access_token = token_response.body.get("access_token")
        .and_then(|v| v.as_str())
        .ok_or_else(|| Error::Generic("[Action:DeployPackage] access_token missing from response".into()))?
        .to_string();

    let download_link = format!(
        "{}/repositories/{}/{/}/downloads/{}.tar.gz?access_token={}",
        endpoint.trim_end_matches('/'),
        owner, slug, commit, access_token
    );

    let overrides = DeploymentOverrides {
        ref_: Some(download_link),
        ref_type: Some("package".into()),
        title: Some(format!("[CD:Package] {commit}")),
        summary: Some(format!("Branch:{", branch.as_deref().unwrap_or("")),
    };

    do_create_deployment(opts, _io, http, Some(overrides))
}
```

This is the **most cross-cutting action** in the codebase. It:

1. Validates that all required env vars and CLI options are present.
2. Calls `do_create_access_token` to get a temporary token (this mutates the `http` client's endpoint, content type, and basic auth).
3. Builds a download URL of the form `https://api.bitbucket.org/2.0/repositories/{owner}/{slug}/downloads/{commit}.tar.gz?access_token={token}`.
4. Constructs a `DeploymentOverrides` that points `ref` at the download URL and sets `ref_type` to `"package"`.
5. Calls `do_create_deployment` (from `deploy_naut.rs`) with those overrides.

Teaching points:

- `let _stack = ...` — the leading underscore tells the compiler "I know this is unused; I'm just doing this for the side effect of validating that it exists." Without the underscore, the compiler would warn that the variable is never used.
- `do_create_access_token(opts, http)?` — calls the previous action function **directly**, not through the dispatcher. This is a deliberate shortcut: the dispatcher is for CLI-level routing; within the implementation, we just call functions.
- `eprintln!(...)` — debug output on stderr. This is here to help diagnose OAuth issues during development. In a more polished codebase, you'd probably remove it or gate it behind a debug flag.
- `endpoint.trim_end_matches('/')` — strip a trailing slash from the endpoint before appending the URL. This prevents double-slashes.
- `DeploymentOverrides { ref_: ..., ref_type: ..., title: ..., summary: ... }` — struct literal with the `Some(...)` wrappers. Note the field is `ref_` (with trailing underscore) because `ref` is a Rust keyword.
- `format!("[CD:Package] {commit}")` — `[CD:Package]` is the dashboard title prefix; `{commit}` interpolates the commit SHA. The result looks like `[CD:Package] abc123def456...`.
- `do_create_deployment(opts, _io, http, Some(overrides))` — calls into the other command module. The `Some(overrides)` tells it to use our synthesised ref/title/summary instead of whatever the user supplied on the CLI.

The PHP three-line-vs-one-response divergence

The original PHP `do_deploy_package` printed **three** separate JSON lines:

1. The token response.
2. The deployment response.
3. The final combined response.

The Rust port, by decision D6, prints **only one** — the final combined response. This was a deliberate design choice: it's cleaner and the downstream CI scripts can be updated to expect a single line. The comment in the source notes this.

Why Is It Written This Way?

- **PHP parity, with one fix.** The PHP version's tag-truncation logic was probably a bug, but it's preserved exactly because changing it would alter observable behavior. The port is meant to be **byte-compatible** with the PHP version.
 - **Composition over duplication.** `do_deploy_package` reuses `do_create_access_token` and `do_create_deployment`. It doesn't reinvent either.
 - **Single response.** Despite the PHP version producing three lines, the Rust port produces one. This was an intentional design choice to make CI scripts easier to write.
 - **Defensive validation.** The function checks `commit.len() == 40` even though `do_create_deployment` would also reject it. Failing fast gives a clearer error message.
-

Key Rust Concepts Used Here

Concept	Where	What it does
<code>pub struct Name;</code>	<code>Bitbucket</code>	Unit struct
<code>impl Command for Bitbucket</code>	top	Implement the <code>Command</code> trait
<code>match (a, b)</code>	<code>do_create_tag</code>	Match on a tuple
Match guard <code>if !o.is_empty() ...</code>	<code>do_create_tag</code>	Refine a pattern
<code>mut tag</code>	<code>do_create_tag</code>	Mutable variable for in-place truncation
<code>String::rfind(&commit)</code>	<code>do_create_tag</code>	Find last occurrence of substring
<code>String::truncate(n)</code>	<code>do_create_tag</code>	Truncate string to first <code>n</code> bytes
<code>std::cmp::min(a, b)</code>	<code>do_create_tag</code>	Smaller of two integers
<code>eprintln!(...)</code>	<code>do_deploy_package</code>	Print to stderr (debug)
<code>endpoint.trim_end_matches('/')</code>	<code>do_deploy_package</code>	Strip trailing characters
<code>_stack, _environment</code>	<code>do_deploy_package</code>	Discarded bindings (side-effect only)
<code>DeploymentOverrides</code> struct	<code>do_deploy_package</code>	Override fields in the deployment payload
<code>Some(field)</code> in struct literal	<code>do_deploy_package</code>	Wrap fields in <code>Option::Some</code>
Cross-module function call	<code>do_deploy_package</code>	Reuses <code>do_create_deployment</code>
<code>.to_string()</code>	<code>do_deploy_package</code>	Convert <code>&str</code> to owned <code>String</code>
<code>#[allow(dead_code)]</code>	trait	Suppress unused-code warnings

CHAPTER 10: TESTING

 Source: `tests/cli_smoke.md`

This file contains **integration tests** that run the actual `nautpie` binary as a subprocess. Unlike unit tests (which live inside the source files in `#[cfg(test)] mod tests`), integration tests live in the `tests/` directory and treat the crate as an **external consumer**.

The Big Picture

These tests verify the **most fundamental contract** of the binary:

1. `--help` lists both subcommands.
2. `sampleSuccess` exits 0 with the documented JSON envelope.
3. `sampleFail` exits 1 with the documented JSON envelope.
4. Missing required options produce a clean error envelope (not a panic).
5. Case-insensitive action dispatch works.

If these five tests pass, the binary is at least minimally functional.

Walkthrough

Imports

```
use assert_cmd::Command;
use predicates::prelude::*;
```

- `assert_cmd::Command` — a wrapper around `std::process::Command` that makes it easy to run a binary and assert on its output. We use `Command::cargo_bin("nautpie")` to find and run the compiled binary in the workspace.
- `predicates::prelude::*` — a crate that provides **composable assertions**. `predicate::str::contains(...)` is a predicate that checks whether a string contains a substring.

Test 1: `--help`

```
#[test]
fn help_lists_both_subcommands() {
    Command::cargo_bin("nautpie")
        .unwrap()
        .arg("--help")
        .assert()
        .success()
        .stdout(predicate::str::contains("deploy:naut"))
        .stdout(predicate::str::contains("ci:bitbucket"));
}
```

Teaching points:

- `Command::cargo_bin("nautpie")` — finds the compiled `nautpie` binary in `target/debug/`. `cargo test` builds the binary before running integration tests, so this always exists when tests run.
- `.unwrap()` — panics if the binary can't be located. In practice this never fails for in-tree tests.
- `.arg("--help")` — adds a CLI argument. Chainable.
- `.assert()` — runs the command and returns an `Assert` object. From here on, every method call asserts something about the result.
- `.success()` — asserts the exit code was 0.
- `.stdout(predicate::str::contains("deploy:naut"))` — asserts that stdout contains the literal string `"deploy:naut"`. The double `.stdout(...)` calls accumulate multiple assertions on stdout.

Test 2: success envelope

```
#[test]
fn sample_success_exits_zero_and_emits_json_envelope() {
    let output = Command::cargo_bin("nautpie")
        .unwrap()
        .args(["deploy:naut", "sampleSuccess"])
        .assert()
        .success()
        .get_output()
        .stdout
        .clone();

    let stdout = String::from_utf8(output).expect("stdout utf-8");
    assert_eq!(
        stdout.lines().count(),
        1,
        "stdout must be a single JSON line: {stdout:?}"
    );

    let value: serde_json::Value =
        serde_json::from_str(stdout.trim()).expect("stdout must be valid JSON");
    assert_eq!(value["status"], 200);
    assert_eq!(value["reason"], "OK");
    assert_eq!(value["body"], "[Action:Success] Response successful.");
}
```

This is the most thorough test in the file. It verifies:

1. The binary exits 0.
2. Stdout has **exactly one line** (the JSON-line contract).
3. The JSON parses cleanly.
4. The status is 200.
5. The reason is **"OK"**.
6. The body matches the documented success message.

Teaching points:

- `.args(["deploy:naut", "sampleSuccess"])` — pass multiple arguments as an array.
- `.get_output().stdout.clone()` — `get_output()` returns an `Output` struct (containing `stdout`, `stderr`, status). Cloning the `stdout` bytes is necessary because `Output` is borrowed by `Assert`.
- `String::from_utf8(output).expect("stdout utf-8")` — convert raw bytes to a UTF-8 `String`. `expect` panics with the given message if conversion fails.
- `stdout.lines().count()` — count newline-separated lines. The strict `== 1` check enforces the contract.
- `serde_json::from_str(stdout.trim())` — parse the trimmed stdout as JSON. `trim()` removes the trailing newline.
- `value["status"]` — JSON object indexing by string key. The result is a `Value` that compares with `assert_eq!`.

Test 3: failure envelope

```
#[test]
fn sample_fail_exits_one_and_emits_error_envelope() {
    let output = Command::cargo_bin("nautpie")
        .unwrap()
        .args(["deploy:naut", "sampleFail"])
        .assert()
        .failure() // exit code is non-zero
        .get_output()
        .stdout
        .clone();

    let stdout = String::from_utf8(output).expect("stdout utf-8");
    assert_eq!(stdout.lines().count(), 1, ...);

    let value: serde_json::Value = serde_json::from_str(stdout.trim()).expect(...);
    assert_eq!(value["status"], 1);
    assert_eq!(value["reason"], "Bad Request");
    let body = value["body"].as_str().expect("body should be a string");
    assert!(body.contains("[Action:Fail] Has failed."));
}
```

The mirror image of test 2, but for the failure path:

- `.failure()` — asserts the exit code was non-zero.
- `value["body"].as_str()` — `as_str()` returns `Option<&str>`. `.expect("body should be a string")` panics if the body is not a string.
- `body.contains(...)` — substring check (PHP `str_contains` style).

Test 4: missing required options

```
#[test]
fn missing_action_returns_error_envelope() {
    let output = Command::cargo_bin("nautpie")
        .unwrap()
        .args(["deploy:naut", "createDeployment"])
        .assert()
        .failure()
        .get_output()
        .stdout
        .clone();

    let stdout = String::from_utf8(output).expect("stdout utf-8");
    let value: serde_json::Value = serde_json::from_str(stdout.trim()).expect(...);
    assert_eq!(value["status"], 1);
    assert_eq!(value["reason"], "Bad Request");
}
```

Verifies that `createDeployment` without `--stack` or `--environment` produces a JSON error envelope (not a panic, not a stack trace on stderr).

Test 5: case-insensitive dispatch

```
#[test]
fn sample_success_accepts_case_insensitive_action() {
    for variant in ["Sample-Success", "SAMPLE_SUCCESS", "samplesuccess"] {
        let output = Command::cargo_bin("nautpie")
            .unwrap()
            .args(["deploy:naut", variant])
            .assert()
            .success()
            .get_output()
            .stdout
            .clone();
        let stdout = String::from_utf8(output).expect("stdout utf-8");
        let value: serde_json::Value = serde_json::from_str(stdout.trim()).expect(...);
        assert_eq!(value["status"], 200, "variant: {variant}");
        assert_eq!(value["body"], "[Action:Success] Response successful.");
    }
}
```

This is the **PHP parity test**. The original PHP version accepted `sampleSuccess`, `SampleSuccess`, and even `SAMPLE_SUCCESS` interchangeably (because PHP's method dispatch is case-insensitive). The Rust port uses `normalise_action()` to convert all of these to `sampleSuccess`.

Teaching points:

- The `for variant in [...]` loop — runs the same assertions three times with different inputs.
- `assert_eq!(value["status"], 200, "variant: {variant}")` — the optional third argument is a format string used in the panic message. This makes test failures easier to diagnose.

Why Is It Written This Way?

- **End-to-end tests for the contract.** These tests verify the **observable behavior** of the binary — what's printed, what exit code is returned. They don't care about internals.
- **assert_cmd over raw std::process::Command.** `assert_cmd` adds ergonomic assertion chaining (`.success()`, `.stdout(...)`) and handles the cargo binary lookup automatically.
- **Strict one-line stdout check.** The binary's contract with downstream CI scripts is "exactly one JSON line." Verifying this in tests prevents regressions where someone accidentally adds an extra `println!` in `main.rs`.
- **JSON parsing in tests.** Rather than comparing the raw stdout string (which would be brittle to key ordering and whitespace), the tests parse it as JSON and compare specific fields.

Key Rust Concepts Used Here

Concept	Where	What it does
<code>#[test]</code>	every test	Marks a function as a test
<code>Command::cargo_bin("nautpie")</code>	every test	Locate the compiled binary
<code>.arg(...)</code> / <code>.args(...)</code>	every test	Add CLI arguments
<code>.assert()</code>	every test	Run the command and return an <code>Assert</code>
<code>.success()</code>	success tests	Assert exit code 0
<code>.failure()</code>	failure tests	Assert exit code non-zero
<code>.stdout(predicate)</code>	help test	Assert on stdout content
<code>.get_output().stdout</code>	several tests	Get the captured stdout bytes
<code>String::from_utf8(...)</code>	every test	Convert bytes to a UTF-8 string
<code>.expect(msg)</code>	every test	Panic with message on <code>Err</code>
<code>.lines().count()</code>	envelope tests	Count newline-separated lines
<code>serde_json::from_str(...)</code>	envelope tests	Parse a string as JSON
<code>value["status"]</code>	envelope tests	Index into a JSON value
<code>.as_str()</code>	failure test	Downcast a JSON value to <code>&str</code>
<code>.contains(...)</code>	failure test	Substring check
<code>for variant in [...]</code>	case test	Iterate over test inputs
Assert format string	case test	Custom panic message with the failing variant
<code>tests/</code> directory	whole file	Integration tests live here (not in <code>src/</code>)

 Source: `tests/http_test.md`

This file contains integration tests for the production `ReqwestHttpClient`. It uses `wiremock` to spin up a real HTTP server in-process, then runs the actual `request` client against it. This is the Rust equivalent of the PHP `MockHandler` test pattern.

The Big Picture

The tests verify that `ReqwestHttpClient` correctly:

1. Sends GETs and parses status / reason / body.
2. Sends POSTs with JSON bodies.
3. Sends POSTs with form-urlencoded bodies.
4. Maps 4xx HTTP responses to `Error::HttpStatus`.
5. Base64-encodes basic-auth credentials.
6. Returns `MissingEndpoint` if `set_endpoint` was never called.
7. Merges extra headers into the request.

Each test sets up its own mock server, mounts its own expectations, and runs the client against it.

Walkthrough

Imports

```
use std::collections::HashMap;

use base64::Engine;
use wiremock::matchers::{body_string, header, method, path};
use wiremock::{Mock, MockServer, ResponseTemplate};

use nautpie::error::Error;
use nautpie::http::{HttpClient, RequestHttpClient};
```

- `wiremock::*` — the mock HTTP server. Key types:
- `MockServer` — the server itself. Started with `MockServer::start().await`.
- `Mock::given(...).respond_with(...)` — registers an expected request and the response to return.
- `MockServer::uri()` — the base URL of the running mock server.
- `wiremock::matchers::*` — composable matchers for request properties:
- `method("GET")` — match by HTTP method.
- `path("/foo")` — match by URL path.
- `header("name", "value")` — match by header.
- `body_string("...")` — match by exact body string.
- `base64::Engine` — used to verify the basic-auth encoding.

Helpers

`build_client()`

```
fn build_client(server: &MockServer) -> RequestHttpClient {
    let mut c = RequestHttpClient::new().expect("client builds");
    c.set_endpoint(&server.uri());
    c
}
```

A trivial constructor: build the client and point it at the mock server's URI.

Teaching points:

- `&MockServer` — borrowed reference. We only need to call `.uri()` on it.
- `.expect("client builds")` — panic if `RequestHttpClient::new()` returns `Err`.

start_server()

```
fn start_server<F>(setup: F) -> MockServer
where
    F: for<'a> std::ops::FnOnce(
        &'a MockServer,
    )
    -> std::pin::Pin<Box<dyn std::future::Future<Output = ()> + 'a>>,
{
    let rt = tokio::runtime::Builder::new_current_thread()
        .enable_all()
        .build()
        .expect("tokio runtime builds");
    let server = rt.block_on(async { MockServer::start().await });
    rt.block_on(async { setup(&server).await });
    drop(rt);
    server
}
```

This is one of the **trickier** pieces of the test infrastructure.

Teaching points:

- The **F** generic with HRTB — `for<'a> FnOnce(&'a MockServer) -> Pin<Box<dyn Future<Output = ()> + 'a>>` is a **higher-ranked trait bound**. It says: "F is a function that, for ANY lifetime 'a, can produce a future borrowing the `MockServer` for that lifetime." This is necessary because the `wiremock::Mock::mount(...)` function is async and borrows the server.
- `std::pin::Pin<Box<dyn Future>>` — a pinned, heap-allocated, type-erased future. We need `Pin` because the future might contain self-referential state. We need `Box<dyn Future>` because the concrete future type is unknown to us.
- `tokio::runtime::Builder::new_current_thread()` — builds a single-threaded tokio runtime. We need this because `wiremock` is async, but our `request::blocking` client is sync. The two need to coexist, so we start a small tokio runtime just for the wiremock setup.
- `rt.block_on(async { MockServer::start().await })` — runs an async block on the runtime. `MockServer::start()` returns a future; `block_on` waits for it to complete.
- `rt.block_on(async { setup(&server).await })` — runs the user's setup closure (which mounts the mocks).
- `drop(rt)` — explicit drop of the runtime. We want the runtime to be gone before we make HTTP calls with the sync `request::blocking` client.
- `server` — returning the `MockServer` is fine even though the runtime is dropped. The server runs in its own thread.

Tests

GET round-trip

```
#[test]
fn get_round_trip_returns_status_reason_body() {
    let body = serde_json::json!({"meta": {"whoami": "marco", "now": "2026"}}).to_string();
    let server = start_server(|s| {
        let body = body.clone();
        Box::pin(async move {
            Mock::given(method("GET"))
                .and(path("/project/example/environment/uat/deloys"))
                .respond_with(ResponseTemplate::new(200).set_body_string(body))
                .mount(s)
                .await;
        })
    });

    let mut client = build_client(&server);
    let resp = client
        .request("GET", "project/example/environment/uat/deloys", None, None)
        .expect("get succeeds");

    assert_eq!(resp.status, 200);
    assert_eq!(resp.reason, "OK");
    let parsed: serde_json::Value = serde_json::from_str(&resp.body).unwrap();
    assert_eq!(parsed["meta"]["whoami"], "marco");
}
```

Teaching points:

- `serde_json::json!({...}).to_string()` — build a JSON value with the `json!` macro, then serialize to a string.
- `Mock::given(method("GET")).and(path(...)).respond_with(...)` — fluent chain that mounts an expectation. `and(...)` composes matchers with logical AND.
- `ResponseTemplate::new(200).set_body_string(body)` — the mock response: status 200 with the given body.
- `.mount(s).await` — register the mock against the server. The `.await` is required because `mount` is async.
- `Box::pin(async move { ... })` — wrap the async block in a pinned boxed future. The `move` keyword moves captured variables (`body`, `s`) into the future.
- `let body = body.clone();` — clone inside the outer closure so the inner `move` closure can take ownership.
- `client.request("GET", "relative/url", None, None).expect("get succeeds")` — make the actual HTTP call. `None, None` means no extra headers, no body.
- `serde_json::from_str(&resp.body).unwrap()` — parse the response body as JSON. `.unwrap()` panics if parsing fails (acceptable in a test).

POST JSON

```
#[test]
fn post_json_sends_body() {
    let server = start_server(|s| {
        Box::pin(async move {
            Mock::given(method("POST"))
                .and(path("/project/example/environment/uat/deploys"))
                .and(header("content-type", "application/json"))
                .and(body_string(r#"{"ref_type":"sha","ref":"abc"}"#))
                .respond_with(ResponseTemplate::new(201).set_body_string(r#"{"id":"42"}"#))
                .mount(s)
                .await;
        })
    });

    let mut client = build_client(&server);
    let resp = client
        .request(
            "POST",
            "project/example/environment/uat/deploys",
            None,
            Some(r#"{"ref_type":"sha","ref":"abc"}"#.into()),
        )
        .expect("post succeeds");

    assert_eq!(resp.status, 201);
    assert_eq!(resp.body, r#"{"id":"42"}"#);
}
```

This test mounts a mock that **only matches** if:

- The method is POST.
- The path is `/project/example/environment/uat/deploys`.
- The `Content-Type` header is `application/json`.
- The body is **exactly** `{"ref_type":"sha","ref":"abc"}`.

If the client sends anything different, the test fails with a "no matching mock" error. This is what makes wiremock powerful: it asserts on the **shape of the request**, not just the response.

Teaching points:

- `body_string(...)` — exact-match on the request body. There's also `body_partial_json(...)` for partial matching.
- `Some(string.into())` — the `Option<String>` parameter type requires `.into()` to convert `&str` to `String`.

POST form-urlencoded

```
#[test]
fn post_form_urlencoded_sends_body() {
    let server = start_server(|s| {
        Box::pin(async move {
            Mock::given(method("POST"))
                .and(path("/site/oauth2/access_token"))
                .and(header("content-type", "application/x-www-form-urlencoded"))
                .and(body_string("grant_type=client_credentials"))
                .respond_with(
                    ResponseTemplate::new(200).set_body_string(r#"{"access_token":"tok"}"#),
                )
                .mount(s)
                .await;
        })
    });

    let mut client = build_client(&server);
    client.set_content_type("application/x-www-form-urlencoded");
    let resp = client
        .request(
            "POST",
            "site/oauth2/access_token",
            None,
            Some("grant_type=client_credentials".into()),
        )
        .expect("form post succeeds");

    assert_eq!(resp.status, 200);
    assert!(resp.body.contains("access_token"));
}
```

Same pattern, but verifying that an `application/x-www-form-urlencoded` body is sent correctly. The client is configured with `set_content_type("application/x-www-form-urlencoded")` before the request.

4xx → `Error::HttpStatus`

```
#[test]
fn four_xx_returns_http_status_error() {
    let server = start_server(|s| {
        Box::pin(async move {
            Mock::given(method("GET"))
                .and(path("/meta"))
                .respond_with(ResponseTemplate::new(400).set_body_string(r#"{"error":"bad"}"#))
                .mount(s)
                .await;
        })
    });

    let mut client = build_client(&server);
    let err = client
        .request("GET", "meta", None, None)
        .expect_err("400 should error");

    match err {
        Error::HttpStatus { status, reason, body } => {
            assert_eq!(status, 400);
            assert_eq!(reason, "Bad Request");
            assert_eq!(body, r#"{"error":"bad"}"#);
        }
        other => panic!("expected HttpStatus, got {other:?}"),
    }
}
```

Teaching points:

- `.expect_err(...)` — assert that a `Result` is `Error` and return the error. Mirror of `.expect()` which asserts `Ok`.

- Pattern matching on `Error::HttpStatus { status, reason, body }` — destructures the struct variant. Each field is bound by name.
- `panic!("expected HttpStatus, got {other:?}")` — `{:?}` is the `Debug` formatter. Prints the value for diagnostic purposes.

Basic auth header

```
#[test]
fn basic_auth_header_is_base64_encoded_user_password() {
    let raw = "marco:password";
    let expected = format!(
        "Basic {}",
        base64::engine::general_purpose::STANDARD.encode(raw.as_bytes())
    );
    let server = start_server(move |s| {
        let expected = expected.clone();
        Box::pin(async move {
            Mock::given(method("GET"))
                .and(path("/meta"))
                .and(header("authorization", expected))
                .respond_with(ResponseTemplate::new(200).set_body_string("{}"))
                .mount(s)
                .await;
        })
    });

    let mut client = build_client(&server);
    client.set_username_and_password("marco", "password");
    let resp = client
        .request("GET", "meta", None, None)
        .expect("auth succeeds");
    assert_eq!(resp.status, 200);
}
```

This test computes the expected `Authorization` header value (base64 of `marco:password`), then mounts a mock that matches only if the header matches. Then it makes a request with `set_username_and_password("marco", "password")` and verifies the request succeeded.

Teaching points:

- `base64::engine::general_purpose::STANDARD.encode(...)` — the standard base64 alphabet (not URL-safe).
- `move |s| { ... }` — outer closure captures `expected` by move.
- `let expected = expected.clone();` — clone inside the outer closure because the inner `move` closure will take ownership.
- `header("authorization", expected)` — match a specific header value. Lowercase header name (`authorization`) because HTTP headers are case-insensitive.

Missing endpoint

```
#[test]
fn missing_endpoint_returns_error() {
    let mut client = ReqwestHttpClient::new().unwrap();
    let err = client
        .request("GET", "meta", None, None)
        .expect_err("no endpoint should error");
    assert!(matches!(err, Error::MissingEndpoint));
}
```

The simplest test in the file: build a client, never call `set_endpoint`, make a request, expect `MissingEndpoint`.

Teaching points:

- `assert!(matches!(err, Error::MissingEndpoint))` — `matches!` macro returns `true` if the expression matches the pattern. `assert!` then checks that the result is `true`.

Extra headers

```
#[test]
fn extra_headers_are_merged() {
    let server = start_server(|s| {
        Box::pin(async move {
            Mock::given(method("GET"))
                .and(path("/meta"))
                .and(header("x-custom", "v1"))
                .respond_with(ResponseTemplate::new(200))
                .mount(s)
                .await;
        })
    });

    let mut client = build_client(&server);
    let mut extra = HashMap::new();
    extra.insert("x-custom".into(), "v1".into());
    let resp = client
        .request("GET", "meta", Some(extra), None)
        .expect("extra headers accepted");
    assert_eq!(resp.status, 200);
}
```

Verifies that the `headers` parameter on `request(...)` is actually added to the outgoing request.

Teaching points:

- `HashMap::new()` — type-inferred as `HashMap<String, String>` from the `insert` calls.
- `"x-custom".into()` — `&str` → `String` conversion via the `Into` trait.

Why Is It Written This Way?

- **Real HTTP server, real request.** Unlike a pure mock, `wiremock` runs an actual HTTP server and `request::blocking` makes actual TCP connections. This catches issues that pure mocks would miss (e.g., header case sensitivity, TLS edge cases).
- **Request-shape assertions.** The `Mock::given(...)` chain asserts that the **outgoing request** matches specific criteria. This is critical: it ensures the client is sending what we expect, not just that it handles responses correctly.
- **`expect_err` and `matches!`.** Tests assert on the **specific error variant**, not just "it failed." This catches regressions where the error type changes.
- **Helper for the awkward async boundary.** The `start_server` helper isolates the complexity of running an async setup on a sync test runner. The actual tests are short and focused.

Key Rust Concepts Used Here

Concept	Where	What it does
<code>tests/</code> directory	whole file	Integration tests live here
<code>#[test]</code>	every test	Marks a function as a test
<code>for<'a> FnOnce(&'a ...) -> ...</code>	<code>start_server</code>	Higher-ranked trait bound
<code>Pin<Box<dyn Future + 'a>></code>	<code>start_server</code>	Pinned, heap-allocated, type-erased future
<code>tokio::runtime::Builder</code>	<code>start_server</code>	Build a tokio runtime
<code>rt.block_on(future)</code>	<code>start_server</code>	Run a future to completion
<code>wiremock::MockServer::start()</code>	<code>start_server</code>	Start an in-process mock server
<code>Mock::given(method(...))</code>	every test	Start a request matcher chain
<code>.and(matcher)</code>	every test	Compose matchers with logical AND
<code>.respond_with(template)</code>	every test	Specify the mock response
<code>.mount(server)</code>	every test	Register the mock
<code>serde_json::json!(...)</code>	GET test	Build a JSON value
<code>.to_string()</code>	GET test	Serialize JSON to a string
<code>.expect_err(...)</code>	several tests	Assert <code>Result</code> is <code>Err</code> and return it
<code>matches!(expr, pattern)</code>	missing-endpoint test	"Does this expression match the pattern?"
<code>assert!(expr)</code>	missing-endpoint test	Assert that an expression is <code>true</code>
Pattern destructuring	4xx test	<code>Error::HttpStatus { status, reason, body }</code>
<code>panic!("{}", x)</code>	4xx test	Panic with a debug-formatted message
<code>HashMap::new()</code>	extra-headers test	Create a new hash map (type-inferred)
<code>move</code> closure	several tests	Move captured vars into the closure
<code>.clone()</code> inside <code>move</code> closure	several tests	Clone outer variables for the inner closure
<code>into()</code>	multiple	Type conversion via <code>Into</code>

 Source: `tests/deploy_naut_test.md`

This file contains integration tests for the `deploy:naut` actions. It's the **largest** of the test files (~490 lines) because `deploy:naut` has the most actions and the most complex behaviors.

The Big Picture

The tests cover:

Action	Tests
<code>fetch</code>	success returns the meta envelope; 400 returns <code>Error::HttpStatus</code> .
<code>createDeployment</code>	success; 400; missing ref; with overrides; prod ignores bypass_and_start.
<code>getDeployments</code>	success returns sorted data.
<code>lastDeployment</code>	returns the first record (sorted desc).
<code>gitFetch</code>	success returns final completed body; non-202 errors.
<code>checkDeploymentProgress</code>	polls until <code>Completed</code> .

The file also defines a `CaptureIo` struct — a custom `Io` impl used in some tests.

Walkthrough

Imports

```
use std::collections::HashMap;
use std::sync::{Mutex, MutexGuard};

use serde_json::json;

use nautpie::commands::deploy_naut::{
    do_check_deployment_progress, do_create_deployment, do_fetch, do_get_deployments,
    do_git_fetch, do_last_deployment, DeploymentOverrides,
};
use nautpie::error::{ApiResponse, Error};
use nautpie::http::{HttpClient, ReqwestHttpClient};
use nautpie::io::{Io, StderrIo};
use nautpie::options::Options;
```

The same imports as `bitbucket_test.rs` , plus the `deploy_naut` actions and `serde_json::json` .

Fixtures

```
const FIXTURE_GET_DEPLOYMENTS: &str = include_str!("fixtures/getDeployments.json");
const FIXTURE_CREATE_SUCCESS: &str = include_str!("fixtures/createDeploymentSuccess.json");
const FIXTURE_CREATE_ERROR: &str = include_str!("fixtures/createDeploymentError.json");
```

Three JSON files embedded at compile time. They are unchanged from the PHP repo.

Helpers

The helpers (`start_server` , `client_at` , `opts` , `lock_env` , `EnvLock` , `setup_deploy_env`) are essentially the same as in `bitbucket_test.rs` . The only difference is `setup_deploy_env` :

```
fn setup_deploy_env(server_uri: &str) -> EnvLock {
    let g = lock_env();
    std::env::set_var("NAUT_ENDPOINT", server_uri);
    std::env::set_var("DASH_USER", "marco");
    std::env::set_var("DASH_TOKEN", "token123");
    g
}
```

`deploy:naut` actions only need `NAUT_ENDPOINT`, `DASH_USER`, and `DASH_TOKEN`. We don't set the Bitbucket-specific env vars.

Teaching points:

- **"Bare host:port (no path component)" comment in the source** — the comment explains that the wiremock server's URI has no path. So request URLs are rooted at `/` and match mock paths directly. This is a subtle gotcha: if the URI had a path like `/naut`, the request URL would be `/naut/project/...` and the mocks wouldn't match.

CaptureIo

```
struct CaptureIo {
    opts: Option<Options>,
}

impl CaptureIo {
    fn new(opts: Options) -> Self {
        Self { opts: Some(opts) }
    }
}

impl Io for CaptureIo {
    fn warning(&mut self, _: &str) {}
    fn success(&mut self, _: &str) {}
    fn message(&mut self, _: &str) {}
    fn write_json(&mut self, _: &ApiResponse) {}
    fn set_options(&mut self, opts: Options) {
        self.opts = Some(opts);
    }
    fn options(&self) -> Option<Options> {
        self.opts.clone()
    }
}

#[allow(dead_code)]
fn _capture_helper_compiles() {
    let o = CaptureIo::new(Options::new());
    let _: &dyn Io = &o;
}
```

A custom `Io` implementation used to verify that the trait compiles correctly.

Teaching points:

- **`fn _capture_helper_compiles()`** — a function that exists only to assert the code type-checks. The `let _: &dyn Io = &o;` line forces the compiler to verify that `CaptureIo` is a valid `Io` impl.
- **`#[allow(dead_code)]`** — silences the warning that nothing else uses this function.

This is a common Rust pattern: write a "compile-only" test to ensure your trait implementations are correct without writing a full behavioral test.

Test: `fetch_returns_meta_envelope`

```
#[test]
fn fetch_returns_meta_envelope() {
    let server = start_server(|s| {
        Box::pin(async move {
            use wiremock::matchers::{method, path};
            use wiremock::{Mock, ResponseTemplate};
            let body = serde_json::json!({"meta": {"whoami": "marco@example.com", "now": "2017-05-09 11:57:00"}}).to_string();
            Mock::given(method("GET"))
                .and(path("/meta"))
                .respond_with(ResponseTemplate::new(200).set_body_string(body))
                .mount(s)
                .await;
        })
    });
    let mut client = client_at(&server);
    let options = opts(&[("url", "meta")]);
    let resp = do_fetch(&options, &mut client).expect("fetch succeeds");
    assert_eq!(resp.status, 200);
    assert_eq!(resp.reason, "OK");
    assert_eq!(resp.body["meta"]["whoami"], "marco@example.com");
}
```

A standard "happy path" test for `fetch`. The mock returns a small JSON object; the test verifies the response status, reason, and a specific field of the parsed body.

Test: `fetch_bad_response_propagates_error`

```
let err = do_fetch(&options, &mut client).expect_err("400 should error");
match err {
    Error::HttpStatus { status, reason, body } => {
        assert_eq!(status, 400);
        assert_eq!(reason, "Bad Request");
        assert!(body.contains("not supported"));
    }
    other => panic!("expected HttpStatus, got {other:?}"),
}
```

Verifies that a 400 response from the server becomes `Error::HttpStatus`. The `match` extracts `status`, `reason`, and `body` for assertions.

Test: `missing_action_returns_error_envelope`

```
let server = start_server(|_| Box::pin(async {}));
let mut client = client_at(&server);
let mut io = StderrIo::new();
io.set_options(opts(&[]));
let err = do_create_deployment(&io.options().unwrap(), &mut io, &mut client, None)
    .expect_err("empty opts should error");
assert!(matches!(err, Error::MissingOption(_)));
```

Empty options bag → `MissingOption`. Note the empty `start_server` setup — no mocks are mounted because the request errors before any HTTP call.

Test: `get_deployments_returns_sorted_data`

```

let server = start_server(|s| {
    let body = FIXTURE_GET_DEPLOYMENTS.to_string();
    Box::pin(async move {
        use wiremock::matchers::{method, path};
        use wiremock::{Mock, ResponseTemplate};
        Mock::given(method("GET"))
            .and(path("/project/stack/environment/uat/deploys"))
            .respond_with(ResponseTemplate::new(200).set_body_string(body))
            .mount(s)
            .await;
    })
});
let _env = setup_deploy_env(&server.uri());
let mut client = client_at(&server);
let options = opts(&[("stack", "stack"), ("environment", "uat")]);
let resp = do_get_deployments(&options, &mut client).expect("list succeeds");
assert_eq!(resp.status, 200);
let data = resp.body["data"].as_array().expect("data array");
assert_eq!(data.len(), 2);
assert_eq!(data[0]["id"], "64989");
assert_eq!(data[1]["id"], "64962");

```

Teaching points:

- `FIXTURE_GET_DEPLOYMENTS.to_string()` — convert the `&'static str` to a `String` (the mock expects a `String`).
- `resp.body["data"].as_array().expect("data array")` — downcast the `data` field to a JSON array. `expect` panics if it's not an array.
- `data[0]["id"]` — index into the array, then into the object. The result is a `Value` we compare with `assert_eq!`.
- **The sort assertion** — the IDs are 64989 then 64962, which is descending. The action sorts deployments by ID descending, so the first item should have the highest ID.

Test: `last_deployment_returns_first_record`

```

let original: serde_json::Value = serde_json::from_str(FIXTURE_GET_DEPLOYMENTS).unwrap();
assert_eq!(resp.body, original["data"][0]);

```

Verifies that `last_deployment` returns the same JSON object as the first item in the `data` array of the fixture.

Test: `create_deployment_success`

```

let fixture: serde_json::Value = serde_json::from_str(FIXTURE_CREATE_SUCCESS).unwrap();
let inner_body = serde_json::to_string(&fixture["body"]).unwrap();
let server = start_server(move |s| {
    let inner_body = inner_body.clone();
    Box::pin(async move {
        use wiremock::matchers::{method, path};
        use wiremock::{Mock, ResponseTemplate};
        Mock::given(method("POST"))
            .and(path("/project/stack/environment/uat/deploys"))
            .respond_with(ResponseTemplate::new(201).set_body_string(inner_body))
            .mount(s)
            .await;
    })
});
let _env = setup_deploy_env(&server.uri());
let mut client = client_at(&server);
let options = opts(&[
    ("stack", "stack"),
    ("environment", "uat"),
    ("ref_type", "branch"),
    ("ref", "develop"),
]);
let mut io = StderrIo::new();
io.set_options(options.clone());
let resp = do_create_deployment(&io.options().unwrap(), &mut io, &mut client, None)
    .expect("create succeeds");
assert_eq!(resp.status, 201);
assert_eq!(resp.reason, "Created");
assert_eq!(resp.body, fixture["body"]);

```

A round-trip test: send a real `createDeployment`, mock returns a fixture, verify the response body equals the fixture body.

Teaching points:

- `fixture["body"]` — the inner "body" object of the fixture (the envelope has `{status, reason, body}`).
- `options = opts(&[("stack", "stack"), ..., ("ref", "develop")])` — supply all required options.
- `assert_eq!(resp.body, fixture["body"])` — compare two `Value` s. They implement `PartialEq` so this works.

Test: `create_deployment_gone_bad`

```

let err = do_create_deployment(&io.options().unwrap(), &mut io, &mut client, None)
    .expect_err("400 errors");
match err {
    Error::HttpStatus { status, reason, .. } => {
        assert_eq!(status, 400);
        assert_eq!(reason, "Bad Request");
    }
    other => panic!("expected HttpStatus, got {other:?}"),
}

```

Teaching points:

- `Error::HttpStatus { status, reason, .. }` — destructures only `status` and `reason`. The `..` ignores `body`.

Test: `create_deployment_missing_ref_errors`

```
let options = opts(&[
    ("stack", "stack"),
    ("environment", "uat"),
    ("ref_type", "branch"),
]);
// note: no "ref"
let err = do_create_deployment(...).expect_err("missing ref");
match err {
    Error::Generic(m) => assert!(m.contains("Requires ref")),
    other => panic!(...),
}
```

The action explicitly errors with `"[Action:CreateDeployment] Requires ref option"` when no ref is supplied (and not redeploying). The test verifies the error message contains "Requires ref".

Test: `create_deployment_with_overrides_succeeds`

```
let overrides = DeploymentOverrides {
    ref_: Some("https://example.com/pkg.tar.gz".into()),
    ref_type: Some("package".into()),
    title: Some("[CD:Package] abc".into()),
    summary: Some("Branch:main".into()),
};
let mut io = StderrIo::new();
io.set_options(options.clone());
let resp = do_create_deployment(
    &io.options().unwrap(),
    &mut io,
    &mut client,
    Some(overrides),
)
.expect("create with overrides succeeds");
assert_eq!(resp.status, 201);
```

Verifies that `DeploymentOverrides` is correctly applied — the action uses the override values instead of what's in the options bag.

Teaching points:

- `ref_: Some(...)` — struct field with trailing underscore because `ref` is a reserved word.
- `Some(overrides)` — wrap in `Option::Some` because the parameter type is `Option<DeploymentOverrides>`.

Test: `create_deployment_prod_environment_ignores_bypass_and_start`

This is the **most subtle** test in the file.

```
use wiremock::matchers::body_partial_json;
...
Mock::given(method("POST"))
    .and(path("/project/stack/environment/prod/deploy"))
    .and(body_partial_json(json!({"bypass_and_start": false})))
    .respond_with(ResponseTemplate::new(201).set_body_string(inner_body))
    .mount(s)
    .await;
```

The action has a guard: `if bypass_and_start && !is_production(&environment) { ... }`. So when the environment is `"prod"`, the `bypass_and_start` flag in the payload should be `false` even if the user passed `--bypass_and_start=true`.

Teaching points:

- `body_partial_json(...)` — a wiremock matcher that checks the request body **partially matches** the given JSON. We don't need to match the entire body; we just need to verify that `bypass_and_start` is `false`.

- `json!({"bypass_and_start": false})` — the partial match.
- The mock path is `/project/stack/environment/prod/deploy`s — different from the uat path. The mock will only match if the production URL is hit.

Test: `git_fetch_returns_final_completed_body`

```
Mock::given(method("POST"))
    .and(path("/project/example/git/fetches"))
    .respond_with(ResponseTemplate::new(202).set_body_string(r#"{"data":{"id":42}}"#))
    .mount(s)
    .await;
Mock::given(method("GET"))
    .and(path("/project/example/git/fetches/42"))
    .respond_with(
        ResponseTemplate::new(200)
        .set_body_string(r#"{"data":{"attributes":{"status":"Complete"}}}"#),
    )
    .mount(s)
    .await;
```

Two mocks: the POST returns 202 with an ID, the GET returns the final status. The action should call both and return the final state.

Teaching points:

- **Two endpoints, one test** — the action makes a POST then polls with GETs. We mock both endpoints.
- `/git/fetches/42` — the GET path includes the ID from the POST response. This is the **dynamic URL** pattern: the second URL is constructed from data returned by the first call.

Test: `git_fetch_non_202_returns_error`

```
Mock::given(method("POST"))
    .and(path("/project/example/git/fetches"))
    .respond_with(ResponseTemplate::new(500).set_body_string("oops"))
    .mount(s)
    .await;
...
let err = do_git_fetch(&options, &mut client).expect_err("500 errors");
assert!(matches!(err, Error::HttpStatus { status: 500, .. }));
```

If the POST returns something other than 202, the action errors immediately (rather than trying to poll).

Test: `check_deployment_progress_polls_until_completed`

```
Mock::given(method("GET"))
    .and(path("/project/stack/environment/uat/deploy/42"))
    .respond_with(
        ResponseTemplate::new(200)
        .set_body_string(r#"{"data":{"attributes":{"state":"Completed"}}}"#),
    )
    .mount(s)
    .await;
...
let resp = do_check_deployment_progress(&io.options().unwrap(), &mut client)
    .expect("progress check succeeds");
assert_eq!(resp.status, 200);
```

The mock returns `Completed` immediately. The action polls once, sees `Completed`, and returns success.

Tail helpers

```
#[allow(dead_code)]
fn _imports_used() {
    let _h: HashMap<String, String> = HashMap::new();
    let _b: DeploymentOverrides = DeploymentOverrides::default();
}

#[allow(dead_code)]
fn _serde_imported() {
    let _: Option<serde_json::Value> = None;
}
```

Three small "compile-only" helpers that ensure all imports are referenced somewhere (so clippy doesn't complain). Common pattern at the end of test files.

Why Is It Written This Way?

- **Same env-lock pattern** as `bitbucket_test.rs`. Process-wide state must be managed carefully across parallel tests.
 - **One test per scenario**. Each test sets up the mocks it needs and asserts on the specific behavior. This makes failures easy to diagnose.
 - **Real wiremock servers**. Catches HTTP-level bugs that pure mocks would miss.
 - **Compile-only helpers**. A small cost to keep imports referenced and clippy happy.
-

Key Rust Concepts Used Here

Concept	Where	What it does
<code>tests/</code> directory	whole file	Integration tests live here
<code>#[test]</code>	every test	Marks a function as a test
<code>include_str!</code> macro	fixtures	Include a file as <code>&'static str</code>
<code>static MUTEX: Mutex<T></code>	env locking	Global mutex for serializing access
<code>MutexGuard<'static, ()></code>	<code>EnvLock</code>	Guard from <code>Mutex::lock</code>
<code>impl Drop for EnvLock</code>	bottom	RAII cleanup on scope exit
<code>for<'a></code> HRTB	<code>start_server</code>	Higher-ranked trait bound for async closures
<code>Pin<Box<dyn Future + 'a>></code>	<code>start_server</code>	Pinned, heap-allocated future
<code>Box::pin(async move { ... })</code>	setup closures	Pin a future on the heap
<code>tokio::runtime::Builder</code>	<code>start_server</code>	Build a tokio runtime
<code>rt.block_on(...)</code>	<code>start_server</code>	Run an async block to completion
<code>move</code> closure	several tests	Move captured vars into the closure
<code>.clone()</code> inside <code>move</code> closure	several tests	Clone outer vars for the inner closure
<code>Mock::given(method(...))</code>	every test	Start a request matcher chain
<code>.and(matcher)</code>	every test	Compose matchers with logical AND
<code>.respond_with(template)</code>	every test	Specify the mock response
<code>.mount(server)</code>	every test	Register the mock
<code>body_partial_json(...)</code>	prod-ignores test	Match request body partially
<code>body_string(...)</code>	several tests	Match request body exactly
<code>header("name", "value")</code>	several tests	Match a request header
<code>method("GET")</code>	every test	Match by HTTP method
<code>path("/foo")</code>	every test	Match by URL path
<code>serde_json::json!({...})</code>	several tests	Build a <code>Value</code> from JSON-like syntax
<code>serde_json::from_str(...)</code>	several	Parse a string as JSON
<code>serde_json::to_string(&value)</code>	several	Serialize a <code>Value</code> to a string
<code>.as_array().expect("data array")</code>	<code>get_deployments</code> test	Downcast a <code>Value</code> to an array
<code>Value</code> indexing <code>["key"]</code>	assertions	Index into a JSON value
<code>matches!(err, Variant)</code>	many	Pattern-match on an enum value
<code>..</code> in pattern	several	"Bind these fields, ignore the rest"

Concept	Where	What it does
<code>_env</code> binding	every setup	Hold the env lock for its <code>Drop</code> side effect
<code>let _: Type = ...</code>	helpers	Type annotation only
<code>#[allow(dead_code)]</code>	helpers	Suppress unused-code warnings
Custom <code>Io</code> impl	<code>CaptureIo</code>	Implement the trait for testing
<code>&dyn Io</code>	helper	Trait object reference

 Source: `tests/bitbucket_test.md`

This file contains integration tests for the `ci:bitbucket` actions defined in `src/commands/bitbucket.rs`. Like `http_test.rs`, it uses `wiremock` to spin up real HTTP servers and exercises the actual `ReqwestHttpClient`.

The Big Picture

The tests cover the three `ci:bitbucket` actions:

Action	Tests
<code>createAccessToken</code>	success returns the <code>access_token</code> field; 401 returns <code>Error::HttpStatus</code> .
<code>createTag</code>	tag without commit is sent as-is; tag containing commit is truncated; short commit errors.
<code>deployPackage</code>	full happy path with token + deploy mock; short commit errors.

The tests also demonstrate a recurring pattern: **environmental state must be carefully managed** because `do_create_access_token` and `do_deploy_package` read environment variables.

Walkthrough

Imports

```
use std::sync::{Mutex, MutexGuard};

use nautpie::commands::bitbucket::{do_create_access_token, do_create_tag, do_deploy_package};
use nautpie::error::{ApiResponse, Error};
use nautpie::http::{HttpClient, ReqwestHttpClient};
use nautpie::io::{Io, StderrIo};
use nautpie::options::Options;
```

- `std::sync::{Mutex, MutexGuard}` — for serializing environment-variable mutations across tests.
- `nautpie::commands::bitbucket::*` — the three public functions we test.
- `include_str!` macro is used at the top to embed JSON fixtures.

Fixture

```
const FIXTURE_CREATE_SUCCESS: &str = include_str!("fixtures/createDeploymentSuccess.json");
```

`include_str!` — a macro that includes the file's contents as a `&'static str` at compile time. This means the fixture file is part of the test binary; no runtime file I/O.

Helper: `start_server()`

Identical to the helper in `http_test.rs`. Briefly:

```
fn start_server<F>(setup: F) -> wiremock::MockServer
where
    F: for<'a> std::ops::FnOnce(&'a wiremock::MockServer)
        -> std::pin::Pin<Box<dyn std::future::Future<Output = ()> + 'a>>,
{
    let rt = tokio::runtime::Builder::new_current_thread()
        .enable_all()
        .build()
        .expect("tokio runtime builds");
    let server = rt.block_on(async { wiremock::MockServer::start().await });
    rt.block_on(async { setup(&server).await });
    drop(rt);
    server
}
```

Starts a tokio runtime, starts a mock server, runs the user's setup to mount mocks, then drops the runtime. Returns the running `MockServer`.

Helper: `client_at()`

```
fn client_at(server: &wiremock::MockServer) -> ReqwestHttpClient {
    let mut c = ReqwestHttpClient::new().expect("client builds");
    c.set_endpoint(&server.uri());
    c
}
```

Build a client and point it at the mock server.

Helper: `opts()`

```
fn opts(pairs: &[(&str, &str)]) -> Options {
    let mut o = Options::new();
    for (k, v) in pairs {
        o.set(*k, (*v).to_string());
    }
    o
}
```

Build an `Options` bag from a list of `(key, value)` pairs. Useful for tests that need to construct specific CLI option sets.

Teaching points:

- `pairs: &[(&str, &str)]` — a borrowed slice of tuples of borrowed string slices.
- `*k, *v` — dereference the tuple elements. `k` and `v` are `&&str`; `*k` gives `&str`, which we then pass to `set` via `.to_string()`.

Env-lock pattern

```
struct EnvLock(MutexGuard<'static, ()>);
static ENV_MUTEX: Mutex<()> = Mutex::new(());

fn lock_env() -> EnvLock {
    EnvLock(ENV_MUTEX.lock().unwrap_or_else(|e| e.into_inner()))
}

impl Drop for EnvLock {
    fn drop(&mut self) {
        for key in [
            "NAUT_ENDPOINT",
            "DASH_USER",
            "DASH_TOKEN",
            "BB_ENDPOINT",
            "BB_AUTH_STRING",
            "BITBUCKET_REPO_OWNER",
            "BITBUCKET_REPO_SLUG",
            "BITBUCKET_BRANCH",
            "BB_CONSUMER_KEY",
            "BB_CONSUMER_SECRET",
            "BB_OAUTH_ENDPOINT",
        ] {
            std::env::remove_var(key);
        }
    }
}
```

This is a **critical pattern** for any test that mutates process-wide environment variables.

Teaching points:

- `static ENV_MUTEX: Mutex<()> = Mutex::new(());` — a global mutex. `()` is the "type parameter" — we don't actually care about the protected data, just that only one test mutates env vars at a time.
- `MutexGuard<'static, ()>` — the guard returned by `Mutex::lock`. The `'static` lifetime comes from the `static` mutex.
- `unwrap_or_else(|e| e.into_inner())` — if the lock is **poisoned** (a thread panicked while holding it), recover by extracting the inner guard anyway. Tests should still run.
- `impl Drop for EnvLock` — when the `EnvLock` is dropped (e.g., end of test function), the env vars are cleared. This is **RAII**: the cleanup happens automatically when the lock goes out of scope.

`setup_bitbucket_env()`

```
fn setup_bitbucket_env(server_uri: &str) -> EnvLock {
    let g = lock_env();
    std::env::set_var("BB_OAUTH_ENDPOINT", server_uri);
    std::env::set_var("BB_ENDPOINT", server_uri);
    std::env::set_var("BB_CONSUMER_KEY", "BB_CONSUMER_KEY");
    std::env::set_var("BB_CONSUMER_SECRET", "BB_CONSUMER_SECRET");
    std::env::set_var("BITBUCKET_REPO_OWNER", "ssmarco");
    std::env::set_var("BITBUCKET_REPO_SLUG", "cd-test");
    std::env::set_var("BITBUCKET_BRANCH", "release/something");
    std::env::set_var("NAUT_ENDPOINT", server_uri);
    std::env::set_var("DASH_USER", "u");
    std::env::set_var("DASH_TOKEN", "t");
    g
}
```

Sets all the env vars that `do_create_access_token`, `do_create_tag`, and `do_deploy_package` need. Returns the `EnvLock` so the caller can hold it for the duration of the test (and cleanup happens at the end).

Teaching points:

- `BB_OAUTH_ENDPOINT` and `BB_ENDPOINT` are set to the same `server_uri` — both functions read either or both. Setting both is defensive.

- `BB_CONSUMER_KEY` set to `"BB_CONSUMER_KEY"` — the value is the same as the variable name, just for clarity.
- The `let _env = setup_bitbucket_env(...)` pattern — the test binds the returned `EnvLock` to `_env`. The underscore prefix tells the compiler "I know I'm not using this value; I'm just keeping it alive for its `Drop` side effect."

Test: `create_access_token_returns_response`

```
#[test]
fn create_access_token_returns_response() {
    let server = start_server(|s| {
        Box::pin(async move {
            use wiremock::matchers::{method, path};
            use wiremock::{Mock, ResponseTemplate};
            let body = serde_json::json!({
                "access_token": "R2uy6EHAKD7K1q3IG9FHf1B4hq9IprThiLsT0HnA=",
                "scopes": "repository",
                "expires_in": 7200,
                "refresh_token": "Sdyr6UewYGxsmgDH78",
                "token_type": "bearer",
            });
            .to_string();
            Mock::given(method("POST"))
                .and(path("/oauth2/access_token/"))
                .respond_with(ResponseTemplate::new(200).set_body_string(body))
                .mount(s)
                .await;
        })
    });
    let _env = setup_bitbucket_env(&server.uri());
    let mut client = client_at(&server);
    let options = opts(&[]);
    let resp = do_create_access_token(&options, &mut client).expect("token succeeds");
    assert_eq!(resp.status, 200);
    assert_eq!(
        resp.body["access_token"],
        "R2uy6EHAKD7K1q3IG9FHf1B4hq9IprThiLsT0HnA="
    );
    assert_eq!(resp.body["scopes"], "repository");
}
```

Teaching points:

- `use ...` inside the closure — the imports are scoped to just this test. This avoids polluting the file-level imports and makes it clear which test uses which dependency.
- `opts(&[])` — empty options. `createAccessToken` reads everything from env vars, not options.
- `resp.body["access_token"]` — index into the JSON body (a `serde_json::Value`) by key.

Test: `create_access_token_non_200_errors`

```
let err = do_create_access_token(&options, &mut client).expect_err("401 errors");
assert!(matches!(err, Error::HttpStatus { status: 401, .. }));
```

Teaching points:

- `matches!(err, Error::HttpStatus { status: 401, .. })` — the `..` is a "rest pattern." It matches any other fields in the struct variant without binding them. So we're asserting that the error is an `HttpStatus` with status 401, ignoring `reason` and `body`.

Test: `create_tag_truncates_commit_in_tag`

```
let options = opts(&[
    ("commit", "COMMIT_HASH_REQUIRES_40_CHARS_1234567890"),
    ("tag", "v1.2.34"),
]);
let resp = do_create_tag(&options, &mut client).expect("createTag succeeds");
assert_eq!(resp.status, 201);
assert_eq!(resp.body["name"], "v1.2.34");
```

The tag (`v1.2.34`) does not contain the commit string, so PHP's `strpos` returns false. The Rust port's `tag.rfind(&commit)` returns `None`. The tag is sent as-is.

Test: `create_tag_truncates_when_tag_contains_commit`

```
let commit = "COMMIT_HASH_REQUIRES_40_CHARS_1234567890";
let tag = format!("release/{commit}-extra");
let options = opts(&[("commit", commit), ("tag", &tag)]);
```

The tag contains the full commit string. PHP's `strpos` returns the position; the tag is truncated to `pos + 7` chars. The Rust port replicates this with `tag.rfind(&commit)` and `tag.truncate(pos + cap)`.

Teaching points:

- `format!("release/{commit}-extra")` — produces something like `release/COMMIT_HASH_REQUIRES_40_CHARS_1234567890-extra`.
- **The mock just returns 201** — we don't directly verify the truncation in the wiremock body. The test asserts the request succeeded; the truncation logic is tested indirectly via the unit tests in `bitbucket.rs` itself.

Test: `create_tag_rejects_short_commit`

```
let _env = setup_bitbucket_env("http://unused");
let mut client = RequestHttpClient::new().unwrap();
let options = opts(&[("commit", "abc"), ("tag", "v1")]);
let err = do_create_tag(&options, &mut client).expect_err("short commit errors");
assert!(matches!(err, Error::Generic(_)));
```

Teaching points:

- `setup_bitbucket_env("http://unused")` — the URI doesn't matter because `do_create_tag` errors before any HTTP call. We just need the env vars set.
- `Error::Generic(_)` — the `_` matches any inner data (we don't care which error message it is).

Test: `deploy_package_returns_single_create_deployment_response`

This is the **biggest** test in the file. It exercises the full `deployPackage` flow:

1. Mock the OAuth token endpoint to return a valid token.
2. Mock the DeployNaut deployment endpoint to return a successful fixture.
3. Call `do_deploy_package` with a 40-char commit, stack, and environment.
4. Verify the response.

```

let token_body = serde_json::json!({
    "access_token": "R2uy6EHAKD7K1q3IG9FHF1B4hq9IprTHiLst0HnA=",
    ...
}).to_string();
let deploy_fixture: serde_json::Value = serde_json::from_str(FIXTURE_CREATE_SUCCESS).unwrap();
let deploy_body = serde_json::to_string(&deploy_fixture["body"]).unwrap();

let server = start_server(move |s| {
    let token_body = token_body.clone();
    let deploy_body = deploy_body.clone();
    Box::pin(async move {
        // mount two mocks: one for the token endpoint, one for the deployment
    })
});

```

Teaching points:

- `serde_json::from_str(FIXTURE_CREATE_SUCCESS).unwrap()` — parse the fixture JSON file (included at compile time) into a `Value`.
- `&deploy_fixture["body"]` — index into the parsed fixture. `deploy_fixture["body"]` is itself a `Value`. We pass `&Value` to `to_string`.
- `move |s| { ... }` — the outer closure takes ownership of `token_body` and `deploy_body`.
- `let token_body = token_body.clone();` — clone before the inner `move` takes ownership.
- **Two mocks mounted** — one for the OAuth endpoint, one for the deployment endpoint. The deployment flow makes both calls in sequence.

```

let mut io = StderrIo::new();
io.set_options(options.clone());
let resp = do_deploy_package(&io.options().unwrap(), &mut io, &mut client)
    .expect("deployPackage succeeds");

```

Teaching points:

- `StderrIo::new()` — construct a real `Io` impl (not a mock).
- `io.set_options(options.clone())` — stuff the options into the `Io` (the side-channel pattern from `io/mod.rs`).
- `&io.options().unwrap()` — pull the options back out and pass them to `do_deploy_package`.
- `options.clone()` — `set_options` consumes the `Options`, so we clone before passing in.

```

assert_eq!(resp.status, 201);
assert_eq!(resp.reason, "Created");
assert_eq!(resp.body["data"]["attributes"]["ref_type"], "package");
assert!(resp.body["data"]["attributes"]["package_link"]
    .as_str()
    .unwrap_or_default()
    .starts_with("https://"));

```

Teaching points:

- `resp.body["data"]["attributes"]["ref_type"]` — chained JSON indexing. Each `[]` returns a `Value`. We assert it's `"package"` (because the override sets `ref_type` to `"package"`).
- `.as_str().unwrap_or_default()` — try to extract a string; default to empty if the field is missing or not a string.
- `.starts_with("https://")` — substring check at the start.

Test: `deploy_package_rejects_short_commit`

Same pattern as the `create_tag` short-commit test: set env vars to a dummy URI, call with a short commit, expect `Error::Generic`.

Tail utility

```
#[allow(dead_code)]
fn _serde_used() {
    let _: Option<ApiResponse> = None;
}
```

A small helper that exists only to keep the `serde_json` and `ApiResponse` imports alive (so clippy doesn't complain about unused imports). Common pattern at the bottom of test files.

Why Is It Written This Way?

- **Real HTTP via wiremock.** Same philosophy as `http_test.rs` : catch issues that pure mocks would miss.
 - **Env-lock pattern.** Process-wide env var mutations are dangerous in parallel test runs. The mutex serializes them, and the `Drop` impl cleans up after each test.
 - **Fixture files via `include_str!`.** The JSON fixtures (in `tests/fixtures/`) are unchanged from the PHP repo. They live alongside the tests, not in source code.
 - **Test names describe the scenario.** `create_tag_truncates_when_tag_contains_commit` immediately tells you what scenario is being tested. The Rust convention for test names is `snake_case` , often describing the behavior under test.
-

Key Rust Concepts Used Here

Concept	Where	What it does
<code>tests/</code> directory	whole file	Integration tests live here
<code>#[test]</code>	every test	Marks a function as a test
<code>include_str!</code> macro	fixture loading	Include a file's contents as <code>&'static str</code>
<code>static MUTEX: Mutex<T> = Mutex::new(...)</code>	<code>ENV_MUTEX</code>	Global mutex for serializing access
<code>MutexGuard<'static, ()></code>	<code>EnvLock</code>	The guard type from <code>Mutex::lock</code>
<code>unwrap_or_else(e e.into_inner())</code>	<code>lock_env</code>	Recover from a poisoned mutex
<code>impl Drop for EnvLock</code>	bottom	RAII cleanup when the lock goes out of scope
<code>for<'a> FnOnce(&'a ...) -> Pin<Box<dyn Future + 'a>></code>	<code>start_server</code>	HRTB for the async setup closure
<code>tokio::runtime::Builder</code>	<code>start_server</code>	Build a tokio runtime
<code>Pin<Box<dyn Future>></code>	setup closures	Pinned, boxed, type-erased future
<code>Box::pin(async move { ... })</code>	setup closures	Pin a future on the heap
<code>move</code> closure	several tests	Move captured vars into the closure
<code>.clone()</code> inside <code>move</code> closure	several tests	Clone outer vars for the inner closure
<code>use ...</code> inside a closure	several tests	Localized imports
<code>serde_json::from_str(...)</code>	fixture loading	Parse a string as JSON
<code>serde_json::to_string(&value)</code>	deploy test	Serialize a <code>Value</code> to a string
<code>serde_json::json!({...})</code>	several tests	Build a <code>Value</code> from JSON-like syntax
<code>to_string()</code>	multiple	Convert values to owned strings
<code>from_str(FIXTURE).unwrap()</code>	several	Parse and panic-on-fail
<code>matches!(err, Error::Variant { field: x, .. })</code>	several	Pattern-match with rest-binding <code>..</code>
<code>_env</code> binding	every setup	Hold the env lock for its <code>Drop</code> side effect
<code>options.clone()</code>	deploy test	Clone the options before passing to <code>set_options</code>
<code>Value</code> indexing <code>["key"]</code>	assertions	Index into a JSON value
<code>.as_str().unwrap_or_default()</code>	assertion	Try to extract a string, default to ""
<code>.starts_with(...)</code>	assertion	Check a prefix
<code>#[allow(dead_code)]</code>	tail	Suppress unused-code warnings

CHAPTER 11: DOCUMENTATION AS CODE

 Source: `examples/README.md`

This directory contains **runnable examples** that demonstrate how to use `nautpie` as a library. The Azure SDK guidelines recommend providing one example per common use case (`[rust-repo-samples-examples]`), and these three examples cover the three most useful entry points.

The Big Picture

Cargo treats any `.rs` file in the `examples/` directory as a separate program. Each one is compiled and runnable via:

```
cargo run --example <name>
```

The example file **must** contain a `fn main()` because Cargo compiles it as a standalone binary. The examples use `nautpie` as a regular dependency — they import from the crate via `use nautpie::...` just like an external consumer would.

The Three Examples

1. `embed_as_library.rs`

Shows how to use `nautpie`'s `ApiResponse` envelope directly, without going through the CLI. This is useful if you're writing a Rust program that wants to emit responses in the same JSON-line shape `nautpie` produces.

Run it with:

```
cargo run --example embed_as_library
```

What it teaches:

- How to construct an `ApiResponse::ok` and `ApiResponse::error`.
- The `to_line()` method that produces a single-line JSON string (matching the binary's stdout contract).

2. `build_deployment_payload.rs`

Builds a `DeploymentDetails` payload using the consuming builder pattern, then renders it to JSON. **No network calls** are made — this is purely about understanding the wire format.

Run it with:

```
cargo run --example build_deployment_payload
```

What it teaches:

- The fluent builder: `DeploymentDetails::new().ref(...).ref_type(...).title(...)`.
- How `values()` strips null and empty fields (matching PHP's `array_filter($details, 'isNotNull')`).
- What the JSON payload actually looks like before it's POSTed to DeployNaut.

3. `parse_boolean_flag.rs`

Demonstrates the lenient boolean parser used by `nautpie` for flags like `--bypass_and_start`. The parser accepts `true` / `false`, `yes` / `no`, `1` / `0`, and treats missing or empty input as `false`.

Run it with:

```
cargo run --example parse_boolean_flag
```

What it teaches:

- The exact set of accepted truthy and falsy strings.
- That unknown values produce a typed `Error::Generic` rather than a panic.

Walkthrough: `embed_as_library.rs`

```
use nautpie::error::ApiResponse;
use serde_json::json;

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let resp = ApiResponse::ok(json!({"hello": "world"}));
    assert_eq!(resp.status, 200);
    assert_eq!(resp.reason, "OK");
    ...
}
```

Teaching points:

- `use nautpie::error::ApiResponse;` — the example pulls in the public crate like any external user would. This proves the library is genuinely reusable.
- `serde_json::json!({...})` — the `json!` macro builds a `serde_json::Value` from JSON-like syntax. Cheaper than constructing a `Value::Object` by hand.
- `Result<(), Box<dyn std::error::Error>>` — the return type. `Box<dyn std::error::Error>` is the standard "any error" type for examples and small programs. The Azure guidelines recommend this for samples ([\[rust-repo-samples-question-operator\]](#)).
- `println!("{line}");` — single-line output with no trailing newline. `println!` adds the newline; `line` is already newline-free.

Walkthrough: `build_deployment_payload.rs`

```
let details = DeploymentDetails::new()
    .ref_("abc123def456abc123def456abc123def4567890")
    .ref_type("sha")
    .title("[CD] v1.2.3")
    .summary("Branch:main")
    .bypass_and_start(true)
    .locked(false);

let json = details.values();
let pretty = serde_json::to_string_pretty(&json)?;
println!("{pretty}");
```

Teaching points:

- **Consuming builder.** Each method takes `mut self` and returns `Self`. This is a different style from the Azure-preferred `with_field()` pattern, but it works well for short-lived payload objects.
- `serde_json::to_string_pretty(&json)?` — render the `Value` as a multi-line pretty-printed JSON string. The `?` propagates any serialisation error (in practice, serialising a `Value` can never fail).
- **The full 40-character SHA.** The example uses a 40-character commit string because DeployNaut requires full SHAs in this position (see `do_create_tag` for the `len() != 40` check).

Walkthrough: `parse_boolean_flag.rs`

```
for input in ["true", "TRUE", "yes", "YES", "1"] {
    let v = check_boolean(Some(input))?;
    assert!(v);
    println!("{input:?} -> {v}");
}
```

Teaching points:

- `for input in [...]` — iterate over a static array of `&str`. No allocation per iteration.
- `assert!(v)` — for booleans, prefer `assert!(v)` over `assert_eq!(v, true)`. Clippy's `bool_assert_comparison` lint enforces this.
- `{:?}` for `Option<&str>` — the `Debug` formatter prints `Some("yes")` or `None`, which is what we want here.

Why Are Examples Useful?

- **Living documentation.** Unlike README examples (which can rot), `examples/` are compiled and run on every `cargo test`. If a public API breaks, the example fails to compile.
- **Discoverability.** A new contributor can `cargo run --example build_deployment_payload` and immediately see what the JSON payload looks like without reading the full library.
- **Reusable starting point.** Users embedding `nautpie` can copy an example as a starting template.
- **Tested doc snippets.** The Azure guidelines (`[rust-doc-samples]` and `[rust-repo-samples]`) recommend runnable examples in both doc comments (`///` blocks) and `examples/`. We do both.

How Examples Are Wired In

`Cargo.toml` doesn't need any extra configuration to enable `examples/`. Cargo picks them up automatically:

```
$ cargo run --example embed_as_library
Compiling nautpie v0.1.0
Finished `dev` profile [unoptimized + debuginfo] target(s) in 1.20s
Running `target/debug/examples/embed_as_library`
{"status":200,"reason":"OK","body":{"hello":"world"}}
```

The compiled binary ends up in `target/debug/examples/<name>`. You can run it directly, or run it through `cargo run --example <name>`.

Key Rust Concepts Used Here

Concept	Where	What it does
<code>examples/</code> directory	whole folder	Cargo treats each <code>.rs</code> file as a separate runnable program
<code>cargo run --example <name></code>	invocation	Build and run a specific example
<code>use nautpie::...;</code>	imports	Use the library from outside the crate (proves the lib is reusable)
<code>fn main() -> Result<(), Box<dyn Error>></code>	main signature	Required entry point; <code>Box<dyn Error></code> is the standard "any error" type
<code>serde_json::json!(...)</code>	value construction	Build a <code>Value</code> from JSON-like syntax
<code>serde_json::to_string_pretty(&v)</code>	output	Pretty-print a <code>Value</code> as multi-line JSON
Consuming builder	<code>DeploymentDetails</code>	Each <code>with_*</code> returns <code>Self</code> , enabling <code>a.b().c().d()</code> chains
<code>assert!(v)</code> for booleans	<code>parse_boolean_flag</code>	Clippy enforces <code>assert!(v)</code> over <code>assert_eq!(v, true)</code>
<code>Box<dyn std::error::Error></code>	return type	Type-erased error for small programs
<code>?</code> operator	error propagation	Early-return on <code>Err</code>

CHAPTER 12: LIBRARY DESIGN

 Source: `.kimchi/docs/azure-sdk-guidelines-review.md`

This report audits the `nautpie` codebase against the [Azure SDK Rust Guidelines](#) and surfaces what we can implement or learn from.

Important Caveat First

The Azure SDK guidelines are written for **client libraries that wrap Azure REST services** — they describe how to build a `SecretClient`, an `azure_core::Pipeline`, a `Pager<T>`, `Poller<T>`, and so on. `nautpie` is a **single-binary CLI** that talks to two non-Azure services (DeployNaut and Bitbucket Pipelines). Roughly a third of the guidelines (TypeSpec, `azure_core`, `TokenCredential`, namespace registration with the Architecture Board, Pager/Poller return types, ARM, default features) **do not apply** and are listed under "Not applicable" below.

The remaining guidelines — about idiomatic Rust, errors, documentation, packaging, linting, MSRV — are general enough to apply to any production-quality Rust project. Those are where the audit focuses.

Codebase Snapshot

Concrete numbers gathered before the audit:

- **58** public API items (`pub fn` / `pub struct` / `pub enum` / `pub trait` / `pub mod`).
- **135** doc-comment lines (`///`).
- **Sync only** in production code (`async` appears only inside `tests/*` to drive `wiremock`).
- **One** `serde::Serialize/Deserialize` derive (`ApiResponse` only).
- **No** `non_exhaustive`, `no missing_docs` lint.
- **Three** debug `eprintln!` statements (`http/request_client.rs:118`, `commands/bitbucket.rs:186`) and **one** legitimate `println!` (the JSON-line contract in `io/stderr_io.rs:56`).

- CI runs `cargo fmt --check` , `cargo clippy -D warnings` , `cargo build --release` , `cargo test` on **stable** only.
- `Cargo.toml` declares `rust-version = "1.85"` .

Section 1 — Design Principles

The five principles (Idiomatic, Consistent, Approachable, Diagnosable, Dependable) are general quality goals. Coverage:

Principle	Verdict	Notes
Idiomatic	✔ Strong	<code>Result<T, Error></code> , <code>Option<String></code> , traits for dependency injection, builder pattern in <code>DeploymentDetails</code> .
Consistent	✔ Strong	One error envelope, one client trait, one Io trait, consistent <code>do_*</code> action names.
Approachable	⚠ Mixed	README is thorough; but no <code>examples/</code> directory, no runnable doc examples, no <code>--help</code> -driven hero tutorial.
Diagnosable	⚠ Mixed	Errors are typed and have prefixes. But no structured logging, no tracing, three debug <code>eprintln!</code> s in library code.
Dependable	✔ Strong	PHP-parity contract preserved, semver-friendly (still 0.x), <code>rust-version</code> declared.

Section 2 — General Guidelines

✔ Already followed

- **Idiomatic Rust.** No anti-patterns that scream "translated from PHP." Builder, `Result` , traits, `Option` , all idiomatic.
- **`rust-version` declared** in `Cargo.toml` (`"1.85"`). Rust 1.85 was released Feb 2025, which is comfortably within the 6-month MSRV window from a 2026 vantage point. **TODO check:** as the codebase ages, this needs periodic review.
- **MSRV is tested in CI** — CI runs on `stable` , but not on the MSRV toolchain. This is a **gap**; Azure requires `[rust-engsys-stable]` testing on the MSRV specifically.
- **No panic in library code paths** — `expect()` is used only in test setup and in `main.rs` for the request client build (a process-level init that can never realistically fail). Zero `.unwrap()` in `src/` . **Strong adherence** to `[rust-general-unwrap]` .
- **No `prelude` module** ✔.
- **Cargo.lock committed** ✔ (verified by `ls`).
- **`cargo fmt --check` and `cargo clippy -D warnings` in CI** ✔.

⚠️ Gaps and improvements

Guideline	Current state	Recommendation
<code>[rust-general-dependencies]</code> — feature-gate request	<code>request</code> is unconditionally in <code>[dependencies]</code> with <code>default-features = false</code> . <code>ratatui</code> / <code>crossterm</code> are unconditional.	For a CLI binary this is acceptable, but if the lib were ever extracted as a reusable component, gate heavy deps behind features. Document the decision in <code>Cargo.toml</code> .
<code>[rust-general-idiomatic-code]</code>	✅ Idiomatic.	Keep current style.

Section 3 — Service Client (Most Not Applicable)

Azure expects `*Client` suffix, `pub fn new(&str, Arc<dyn TokenCredential>, Option<...>) -> Result<Self>`, `endpoint(&self) -> &Url`, fields with `pub(crate)` visibility, immutable clients, `clients` and `models` submodules, `with_{credential_type}` constructors, options structs with `Option<T>` fields, default `api_version`, `Default` impl, `Clone` derive, `SafeDebug` instead of `Debug` (PII), and `no env-` reads.

This codebase **deliberately diverges**:

- **Single-binary CLI**, not a reusable client library.
- **DeployNaut** and **Bitbucket** are unit structs (no fields, no state). The *configurable state lives on the `HttpClient` trait object* (`set_endpoint`, `set_username_and_password`, `set_content_type`). This is the antithesis of Azure's "immutable clients" rule, but it's the right choice for a CLI where the request sequence (set endpoint → set auth → POST → set content-type → POST) needs to mutate the same client.
- **NAUT_ENDPOINT**, **DASH_USER**, **DASH_TOKEN**, **BB_*** `env vars` are required. This violates `[rust-client-configuration-env]` and `[rust-client-configuration-env-sys]`, which Azure forbids because "WASM and many IoT devices won't have access to an environment block."
- **No `TokenCredential` abstraction** — basic auth is encoded directly.

What we can learn

Even though the rules don't directly apply, the **spirit** of "configuration should be explicit and discoverable, not implicit" is worth applying where it doesn't fight PHP-parity:

1. **Consider a `ClientOptions` struct** that bundles `endpoint`, `username`, `password` (and Bitbucket equivalents). Today, the constructor pattern is `setup_deploy_naut_env(http) -> Result<(String, String, String), Error>` which reads three env vars in one go. An explicit `DeployNautOptions` builder would make it clear what the client needs, and would let users skip `dotenvy::dotenv()` in tests/non-CI environments.
2. **Document why the client is mutable.** Today this design choice is buried in `http/mod.rs` and `commands/deploy_naut.rs`. A short ADR-style comment near `HttpClient::set_endpoint` explaining "the client is mutable because action sequences need to thread new auth state between requests" would make this discoverable for newcomers.
3. **No `Debug` for PII risk.** `Options` derives `Debug`. So does `RequestHttpClient`, which stores the basic-auth header. `Debug` output of these could leak credentials into panic messages and logs. **Recommendation:** add `#[derive(Debug)]` selectively (skip fields holding credentials) or implement `Debug` by hand to redact secrets.

Section 4 — Errors

Azure expects `azure_core::Error` and `azure_core::Result<T>`, with conversion `From<crate::Error> for azure_core::Error`. We don't use `azure_core`, but the principles translate:

Guideline	Current state	Verdict
One top-level <code>Error</code> type	✓ <code>nautpie::Error</code> re-exported from crate root.	Matches <code>[rust-errors-crate-definitions]</code> .
Implements <code>std::error::Error</code>	✓ via <code>#[derive(thiserror::Error)]</code> .	Good.
Variants carry meaningful context	✓ <code>MissingEnv(String)</code> , <code>HttpStatus { status, reason, body }</code> , etc.	Good.
Implements <code>From<azure_core::Error></code>	N/A — we don't depend on <code>azure_core</code> .	Acceptable for a non-Azure crate.
Concise, actionable messages	⚠ Messages have <code>[Required:ENV]</code> prefixes matching PHP. They are accurate but cryptic to a reader who has never seen the PHP original.	Consider adding a <code>#[error(...)]</code> doc-link style note in user-facing docs.
No <code>unwrap</code> / <code>expect</code> in non-test code	✓ Confirmed — only <code>main.rs</code> has <code>expect</code> , on a <code>request</code> client builder that effectively cannot fail.	Matches <code>[rust-general-unwrap]</code> .

What we can learn

1. Add `source()` chaining. `thiserror` makes this one line per variant if the variant holds an inner error. Today our variants carry `String`, not the original error, so error chains end at the wrapper. Switching to `#[source] Box<dyn std::error::Error + Send + Sync>` for `Json`, `Timeout`, `HttpStatus` would let `Error::source()` return the underlying cause for better diagnostics.
2. Categorise errors via `ErrorKind`. Azure's pattern is `Error { kind: ErrorKind, source: ... }`. Today we encode the kind directly in the variant. This is fine for an enum-of-fixed-cases, but if errors ever need to carry dynamic context (e.g., HTTP status + body), a `kind + data` split becomes more flexible.

Section 5 — Models and Builders

`DeploymentDetails` is the only model struct that gets close to Azure's "model type" rules:

Azure rule	DeploymentDetails	Verdict
Derive <code>Clone</code> , <code>Default</code>	<code>#[derive(Debug, Clone, PartialEq)]</code> ; <code>Default</code> impl is manual (could be <code>#[derive(Default)]</code> if all fields had defaults).	Partial — <code>#[derive(Default)]</code> would be cleaner.
Derive <code>serde::Serialize/Deserialize</code>	✗ None.	Gap.
Fields public	✓ all <code>pub</code> .	Good.
Optional fields as <code>Option<T></code>	✓ for <code>ref</code> , <code>title</code> , <code>summary</code> , <code>schedule*_unix</code> . But <code>bypass</code> , <code>bypass_and_start</code> , <code>locked</code> are plain <code>bool</code> , not <code>Option<bool></code> .	Diverges from <code>[rust-model-types-optional]</code> . Azure requires all non-vector fields to be <code>Option<T></code> . We use plain <code>bool</code> because we always want to serialise the field with an explicit <code>false</code> .
Vectors as <code>Option<Vec<T>></code>	N/A — no vector fields.	—
<code>#[serde(skip_serializing_if = "Option::is_none")]</code>	✗ Not used. We hand-roll the JSON in <code>values()</code> with <code>match Some/None</code> logic.	Diverges, but works. The hand-rolled <code>values()</code> matches PHP's <code>array_filter(\$details, 'isNotNull')</code> exactly.
<code>#[non_exhaustive]</code> on response-only structs	✗ Not used.	Gap.
Builder pattern via <code>Model::builder() -> ModelBuilder</code>	✗ We use a consuming-builder: <code>DeploymentDetails::new().ref_("...").ref_type("...")</code> .	Diverges from Azure's <code>with_</code> prefix convention, but is idiomatic Rust.

What we can learn

1. Add `#[derive(Serialize)]` to `DeploymentDetails` and let `serde` do the JSON work. The current hand-rolled `values()` is ~50 lines of careful `match` and `insert` logic. With derives + `#[serde(skip_serializing_if = "Option::is_none")]` + `#[serde(rename_all = "snake_case")]`, the body could shrink to a one-line `serde_json::to_value(&details)`. The PHP parity goal is preserved because the field names stay identical.
2. Rename builder methods to use `with_` prefix. Azure convention is `with_field_name(&mut self, value: FieldType) -> &mut Self`. Today we have `details.ref_("...")`; Azure would suggest `details.with_ref("...")`. This is purely cosmetic and would be a breaking change, so probably **not worth it**.
3. `#[non_exhaustive]` on response-only models. Today there are no response models in the library (`ApiResponse` is a generic envelope, not a service resource). If we ever add response models (e.g., a `Deployment` struct deserialised from DeployNaut's response), apply `#[non_exhaustive]` immediately.

Section 6 — Enumerations

`nautpie` has only one enum that approaches a domain model — `PollOutcome { Pending(String), Done }`. It correctly:

- ✓ Uses PascalCase variants.
- ✓ Derives `Debug` (implicitly, via usage; should add `Clone, Eq, PartialEq` for consistency with Azure's `[rust-enums-derive]`).

- ❌ Does **not** derive `serde::Serialize/Deserialize`. It doesn't need to today, but if the polling state ever crosses an FFI boundary it would.
- ⚠️ Azure says **DO NOT** add `#[non_exhaustive]` to enums. We haven't, which is correct.

The bigger enum is `nautpie::Error`. It does not derive `serde::Serialize` (errors don't currently appear in JSON). The `ApiResponse::body` is `serde_json::Value` which is forgiving enough that error-as-string works fine via `finish()` in `main.rs`.

What we can learn

Nothing actionable here. `PollOutcome` is the right shape for what it does.

Section 7 — Authentication

Azure's authentication guidance assumes `TokenCredential`, OAuth flows, `Arc<dyn TokenCredential>`, and forbids persisting credentials.

`nautpie` does none of that:

- Basic auth (`username:password`) over `Authorization: Basic base64(...)`.
- OAuth client-credentials grant (Bitbucket only) with a returned bearer token stored in a `String`.
- Credentials read from env vars at action-start time.

`set_username_and_password` stores the encoded header on the `HttpClient`. There is **no clearing** of that field after use.

What we can learn

1. **Add `Drop` on `RequestHttpClient` to zero the auth field.** Azure says `[rust-authentication-erase]` — securely free and zero credential data when no longer needed. Today the `authorization: Option<String>` is dropped normally; the bytes are freed but not zeroed. For a CLI that exits immediately after one HTTP call, this is a low-priority hardening item, but a good habit.
2. **Don't log auth.** The `eprintln!("{}", [request] {method} {url})` in `http/request_client.rs:118` is fine (URLs don't typically carry credentials), but worth checking the URL never contains the access token. **Confirmed safe** — the Bitbucket access token is sent as a query parameter on the download URL, which is built in `commands/bitbucket.rs:175` and used by `do_create_deployment`. The URL is constructed internally and not logged.

Section 8 — Packaging & Directory Layout

Azure rule	Current state	Verdict
<code>azure_*</code> crate name	❌ It's <code>nautpie</code> .	N/A — this isn't an Azure crate.
<code>sdk/<service>/azure_*</code> directory	❌ Single-crate repo at root.	N/A.
<code>[workspace.dependencies]</code> centralisation	❌ Single crate, no workspace.	N/A — would matter if/when this is split into <code>nautpie-core</code> and <code>nautpie-cli</code> .
No <code>build.rs</code>	✅ None.	Good.
<code>lib.rs</code> exports public APIs only	✅ All <code>pub mod</code> declarations are intentional. <code>ApiResponse</code> and <code>Error</code> are re-exported from the crate root.	Good.
Module-per-folder with <code>mod.rs</code>	✅ <code>commands/</code> , <code>http/</code> , <code>io/</code> , <code>tui/</code> all use the folder form.	Good.

What we can learn

- 1. **Consider a workspace split.** If `nautpie-core` (the API client logic without the CLI) became a separate crate, it could be reused by other tools. The `lib.rs` already provides this separation in spirit — the binary just calls into it.

Section 9 — Linting

Azure rule	Current state
<code>[workspace.lints.rust]</code> and <code>[workspace.lints.clippy]</code> centralised	❌ Not present. CI runs <code>cargo clippy --all-targets --locked -- -D warnings</code> but lints aren't declared in <code>Cargo.toml</code> .
<code>[lints] workspace = true</code> per crate	❌ Not applicable (single crate).

What we can learn

Add to `Cargo.toml` :

```
[lints.clippy]
all = { level = "warn", priority = -1 }
pedantic = { level = "warn", priority = -1 }
module_name_repetitions = "allow"
must_use_candidate = "allow"
missing_errors_doc = "allow"
```

This makes the lint policy **visible in source** and survives if anyone copies `Cargo.toml` into a fork. Today the policy is implicit in the CI command, which means a developer running `cargo clippy` locally gets a different (looser) set of warnings than CI does.

Section 10 — Documentation

This is the **largest gap** between `nautpie` and Azure's guidance.

Azure rule	Current state	Gap
Doc comments on all public APIs	⚠️ ~135 doc lines for 58 public items — most <code>pub fn</code> have doc comments, but a few <code>pub fn</code> are missing (e.g., <code>options_from_deploy_args</code>, <code>options_from_bitbucket_args</code>, <code>setup_deploy_naut_env</code>, <code>do_get_deployments</code>, <code>do_last_deployment</code>, <code>do_check_deployment_progress</code>, <code>check_deployment_progress</code>, <code>do_create_tag</code>, <code>do_deploy_package</code>, <code>do_create_access_token</code>).	Some action functions are missing doc comments.
<pre>#![warn(missing_docs)]</pre> in <code>lib.rs</code>	❌ Not present.	Easy win. Adding this attribute will surface every missing doc comment as a warning, and we can promote it to <code>deny</code> once the gaps are closed.
<pre>README.md</pre> included in <code>lib.rs</code> via <code>#![doc = include_str!("../README.md")]</code>	❌ Not done. <code>cargo doc</code> does not render the README.	Easy win — one line of code.
Module-level <code>README.md</code> via <code>#![doc = include_str!("README.md")]</code>	❌ Not done in <code>commands/</code> , <code>http/</code> , <code>io/</code> , <code>tui/</code> .	Optional, would help discoverability.
<code># Arguments</code> heading for each parameter	⚠️ Mixed. Some functions have it (e.g., <code>set_username_and_password</code>), most don't.	Standardisation.
Runnable doc examples (<code>/// ```rust no_run</code>)	❌ Zero doc examples anywhere.	Biggest gap. Azure requires <code>no_run</code> examples for client methods. Even minimal <code>/// ```no_run\n/// let _ = nautpie::Error::Generic(\"x\").into();\n/// ```</code> would be a start.
<code>examples/</code> directory	❌ No <code>examples/</code> directory.	Would help downstream users (if the lib were extracted).
Doc-test with <code>no_run</code> attribute	❌ N/A (no doc examples).	Follows from above.

What we can learn — Concrete Improvements

These are the **highest-leverage documentation changes**, ranked:

1. Add `#![warn(missing_docs)]` to `src/lib.rs`. This single line will catch every undocumented public item at compile time. Promote to `deny` once the gaps are closed.
2. Add `#![doc = include_str!("../README.md")]` to `src/lib.rs`. One line. Makes `cargo doc --open` render the README as the crate overview.
3. Add at least one `no_run` doc example per command. E.g., on `do_create_deployment`: `rust /// ```no_run /// use nautpie::options::Options; /// let mut opts = Options::new(); /// opts.set("stack", "example"); /// opts.set("environment", "uat"); /// opts.set("ref_type", "sha"); /// opts.set("ref", "abc123..."); /// ```` This compiles as part of `cargo test` and shows callers the expected shape.
4. Document the action functions that currently have no `///` comment. A one-line summary each is enough; the existing inline doc-comments in `cli.rs` can be cross-referenced.

Section 11 — Build System / CI

Azure rule	Current state	Verdict
<code>[rust-engsys-stable]</code> — test on MSRV toolchain	✗ CI tests on <code>stable</code> only.	Gap. Add a separate job that installs <code>1.85</code> via <code>dtolnay/rust-toolchain@1.85.0</code> and runs the same <code>cargo test</code> command.
<code>[rust-engsys-nightly]</code> — test on nightly	✗ Not done.	Optional — would catch regressions before stable catches them.
<code>[rust-repo-formatting-rustfmt]</code> — fmt in CI	✓ <code>cargo fmt --all -- --check .</code>	Good.
<code>[rust-repo-formatting-engsys]</code> — build pipeline verifies fmt	✓ Same job.	Good.
<code>Cargo.lock</code> committed	✓ (verified).	Good.
README at component root	✓ <code>README.md</code> at repo root.	Good.
<code>examples/</code> directory	✗ Missing.	Optional but recommended.

Section 12 — Settings That Are NOT Applicable

For completeness, these Azure guidelines do not apply to a non-Azure CLI:

- `azure_core::Pipeline`, `TokenCredential`, `Pager<T>`, `Poller<T>` — Azure-specific types.
- `SafeDebug` derive macro — Azure's PII-safe alternative to `Debug`.
- `TypeSpec`-emitted models and `@clientName` decorator.
- `azure_*` crate naming and Architecture Board registration.
- `with_connection_string` constructors.
- Service-method `Options` struct naming convention (`{Client}{Method}Options`).
- `Response<T, F>` return types and `ResultHeaders` traits.
- Header extension traits sealed with `private::Sealed`.
- ETag-related `if_match` / `if_none_match` options.
- `@azure/core-http` and similar central HTTP abstractions.

These are mentioned only so a reader doesn't think they were missed.

Prioritised Action List

If you adopt 5–10 items from this report, pick these (highest payoff first):

#	Change	Effort	Payoff
1	Add <code>#![warn(missing_docs)]</code> to <code>src/lib.rs</code> and fix any warnings it surfaces	30 min	Forces documentation completeness on every future change.
2	Add <code>#![doc = include_str!("../README.md")]</code> to <code>src/lib.rs</code>	1 min	<code>cargo doc</code> now renders the README.
3	Add <code>no_run</code> doc examples to the three or four most-used action functions	1 hr	Shows callers the expected shape, compiles in <code>cargo test</code> .
4	Add <code>[lints.clippy]</code> block to <code>Cargo.toml</code> so local <code>cargo clippy</code> matches CI	15 min	Removes the implicit/policy-as-CI-command gap.
5	Add an MSRV job to <code>.github/workflows/ci.yml</code> that installs Rust 1.85 and re-runs <code>cargo test</code>	15 min	Satisfies <code>[rust-engsys-stable]</code> .
6	Replace the hand-rolled <code>DeploymentDetails::values()</code> JSON building with <code>#[derive(Serialize)] + #[serde(skip_serializing_if = "Option::is_none")]</code>	1 hr	Removes ~50 lines of manual <code>match / insert</code> code. Keeps wire format identical.
7	Document the publicly-visible action functions that have no <code>///</code> comment today	30 min	Closes the doc gap surfaced by item 1.
8	Add an <code>examples/</code> directory with one runnable example per command	2 hr	Provides copy-paste starting points for downstream users.
9	Switch <code>HttpClient</code> 's <code>Debug</code> impl to redact the <code>authorization</code> field	30 min	Avoids leaking credentials in panic messages.
10	Consider introducing <code>*Options</code> structs (<code>DeployNautOptions</code> , <code>BitbucketOptions</code>) to bundle config rather than reading env vars on every action	2 hr	Makes config explicit and discoverable; aligns with Azure's options-struct philosophy.

Items 1–5 are low-risk, mechanical, and align the project with Azure's "library quality" bar without changing any behaviour. Items 6–10 are deeper refactors that trade PHP-parity strictness for Rust-idiomatic clarity.

Closing Note

The Azure SDK guidelines are an excellent quality bar for **any** Rust client library or CLI that aspires to be maintained long-term. `nautpie` already follows the most important general rules (no `unwrap`, idiomatic Rust, `Result`-based errors, `thiserror` for the error type, `cargo fmt` + `cargo clippy` in CI, MSRV declared, `Cargo.lock` committed). The biggest unforced errors are in **documentation coverage and discoverability**, which are also the cheapest to fix.